



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

The Algorithmic Emergence of Strategic Reasoning: An End-to-End Deep Reinforcement Learning Ap- proach to Chess and Italian Draughts

TESI DI LAUREA TRIENNALE IN
MATHEMATICAL ENGINEERING - INGEGNERIA MATEMATICA

Author: **Leonardo Plazzogna**

Student ID: 238204

Advisor: Prof. Marcello Restelli

Academic Year: 2025-26

Ai miei nonni

Abstract

Systems in the AlphaZero family learn to play solely from the rules of a game. The result is usually credited to the architecture, yet such a system rests on a long series of design choices, which may be shaped by the game at hand. One system built for one game cannot separate the two.

This thesis therefore defines the design once and instantiates it twice. A Monte Carlo tree search explores the continuations of a position. A policy–value network supplies the priors that guide the search and the evaluation that ends it. The network is trained on the games the system plays against itself, and a new version replaces the current one only when it beats it. Beneath this, a game-independent interface fixes what an environment must provide, and everything above that interface follows a single specification, realised separately for chess and for Italian draughts. The two games are played on the same board and share their formal properties, but they diverge precisely where an implementation is most sensitive: in the number of moves a position offers and in the rules that govern capture and draw. The experimental programme is carried out on draughts, the narrower of the two, whose scale allows full training runs to be completed and repeated and therefore yields the more consistent measurements; and because the two instances realise one specification, what those measurements establish about the method bears on the design they share.

Trained from no prior knowledge, the draughts instance outgrows every fixed opponent available to measure it. The learning method transfers between the two games intact. Almost every element of the surrounding engineering has to be rewritten, for the size of the game and for its rules. None of those rewrites, however, calls for a different way of learning. The boundary between the two is not where it is assumed to lie. The number of positions gathered before each evaluation of the network, initially chosen for throughput, governs playing strength. No comparison the system makes with itself can reveal this, since every one of them applies the value to both sides at once. Measuring strength in a self-referential system is a problem in its own right, and the protocol this thesis derives for it is the result that carries furthest beyond the two games.

Keywords: deep reinforcement learning; AlphaZero; self-play; Monte Carlo Tree Search; Italian draughts; computer chess; self-evaluation methodology

Abstract in lingua italiana

I sistemi della famiglia AlphaZero imparano a giocare dalle sole regole del gioco. Il merito viene di solito attribuito all'architettura, eppure un sistema del genere poggia su una lunga serie di scelte di progetto, che il gioco che si ha davanti può condizionare. Un solo sistema costruito per un solo gioco non permette di distinguere le due cose.

Questa tesi definisce quindi il progetto una volta sola e lo istanzia due volte. Una ricerca ad albero Monte Carlo esplora le continuazioni di una posizione. Una rete policy-valore fornisce le probabilità a priori che guidano la ricerca e la valutazione che la conclude. La rete è addestrata sulle partite che il sistema gioca contro sé stesso, e una nuova versione sostituisce quella in carica solo quando la batte. Al di sotto, un'interfaccia indipendente dal gioco fissa ciò che un ambiente deve fornire, e tutto ciò che le sta sopra segue un'unica specifica, realizzata separatamente per gli scacchi e per la dama italiana. I due giochi si giocano sulla stessa scacchiera e condividono le stesse proprietà formali, ma divergono proprio dove un'implementazione è più sensibile: nel numero di mosse che una posizione offre e nelle regole che governano la presa e la patta. Il programma sperimentale è condotto sulla dama, il meno ramificato dei due, la cui scala permette di portare a termine e di ripetere esecuzioni di addestramento complete e produce quindi le misure più consistenti; e poiché le due istanze realizzano un'unica specifica, ciò che quelle misure stabiliscono sul metodo riguarda il progetto che entrambe condividono.

Addestrata senza alcuna conoscenza pregressa, l'istanza della dama supera ogni avversario fisso disponibile a misurarla. Il metodo di apprendimento passa intatto da un gioco all'altro. Quasi ogni elemento dell'ingegneria circostante deve essere riscritto, per la dimensione del gioco e per le sue regole. Nessuna di quelle riscritture, però, richiede un modo diverso di imparare. Il confine fra le due cose non cade dove lo si dà per scontato. Il numero di posizioni raccolte prima di ogni valutazione della rete, scelto in origine per le prestazioni, determina la forza di gioco. Nessun confronto che il sistema compie con sé stesso può rivelarlo, perché ognuno di essi applica quel valore a entrambe le parti insieme. Misurare la forza in un sistema autoreferenziale è un problema a sé, e il protocollo che questa tesi ne ricava è il risultato di portata più generale.

Parole chiave: deep reinforcement learning; AlphaZero; self-play; Monte Carlo Tree Search; dama italiana; scacchi computazionali; misura in sistemi autoreferenziali

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
Introduction	1
From Samuel to AlphaZero: The Evolution of Computer Game-Playing	5
1 The Environment: State Representation and Game Mechanics	9
1.1 The Environment Layer	9
1.2 The Chess Environment	10
1.2.1 State Model	10
1.2.2 Move Lifecycle	12
1.2.3 Performance, Extensions and Tooling	14
1.3 The Italian Draughts Environment	14
1.3.1 State Model	14
1.3.2 Move Lifecycle	16
1.3.3 Tooling	18
2 The Search Agent: Monte Carlo Tree Search Implementation	19
2.1 The Search Agent	19
2.2 The Chess Agent	23
2.2.1 Search Control	23
2.2.2 Priors and the Policy Head	24
2.2.3 State Handling and Caching	24
2.2.4 Robustness	25
2.3 The Italian Draughts Agent	25
2.3.1 Search Control	25

2.3.2	Priors and the Policy Head	26
2.3.3	State Handling and Caching	27
2.3.4	Robustness	27
3	Feature Encoding: Bridging State Space and Neural Inputs	29
3.1	Feature Encoding	29
3.2	The Chess Encoder	31
3.2.1	Shape and Perspective	31
3.2.2	Plane Layout	31
3.2.3	Invariants and Contract	32
3.3	The Italian Draughts Encoder	33
3.3.1	Shape and Perspective	33
3.3.2	Plane Layout	33
3.3.3	Invariants and Contract	34
4	The Policy-Value Network: Residual Architecture	37
4.1	The Policy-Value Network	37
4.2	The Chess Network	39
4.2.1	Policy Head	40
4.2.2	Value Head	40
4.2.3	Numerics and Training Interface	41
4.3	The Italian Draughts Network	41
4.3.1	Policy Head	41
4.3.2	Value Head	42
4.3.3	Numerics and Training Interface	43
5	The Inference Bridge: Optimizing the Neural Adapter	45
5.1	The Inference Bridge	45
5.2	The Chess Adapter	46
5.2.1	Move Indexing and Logit Combination	46
5.2.2	Policy and Value Paths	47
5.2.3	Operational Considerations	48
5.3	The Italian Draughts Adapter	48
5.3.1	Move Indexing and the Shared Canonicalisation	48
5.3.2	Policy and Value Paths	48
5.3.3	Operational Considerations	49
6	Network Optimization: Training and Regularization	51

6.1	Learning from Self-Play	51
6.2	The Chess Training Loop	52
6.2.1	Data and Loading	53
6.2.2	Objective	53
6.2.3	Optimisation and Checkpointing	54
6.2.4	Contracts and Efficiency	55
6.3	The Italian Draughts Training Loop	55
6.3.1	Data and Loading	55
6.3.2	Objective	56
6.3.3	Optimisation and Checkpointing	56
6.3.4	Contracts and Efficiency	58
7	The Self-Play Pipeline: Autonomous Data Generation	61
7.1	The Self-Play Loop	61
7.2	The Chess Pipeline	63
7.2.1	Orchestration and Schedules	63
7.2.2	Parallelism and Portability	63
7.2.3	Inference and Search Behaviour	64
7.2.4	Data, Adjudication and Promotion	65
7.3	The Italian Draughts Pipeline	66
7.3.1	Orchestration and Schedules	66
7.3.2	Parallelism and Portability	67
7.3.3	Inference and Search Behaviour	67
7.3.4	Data, Adjudication and Promotion	68
8	Simulation and Results: The Emergence of Strategy	71
8.1	Experimental Setup	71
8.2	Evidence of Learning	73
8.3	The Limits of Relative Measurement	77
8.4	The Value of Additional Cycles	82
8.5	Placement on an Absolute Scale	87
8.6	Performance Settings and Playing Strength	91
8.7	Structural Limits of the Pipeline	95
8.8	Summary of Findings	99
9	Conclusions and Future Developments	103
9.1	Design and Experimental Scope	103
9.2	General and Game-Specific Components	104

9.3 Strength Measurement in a Self-Referential System 108

9.4 Results on Italian Draughts 109

9.5 Limitations 110

9.6 Future Work 111

9.7 Concluding Remarks 112

Bibliography 113

A Hyperparameters and Configuration Manifest 119

List of Figures 127

List of Tables 129

Acknowledgements 131

Introduction

Board games have served as benchmarks for computing since the earliest years of the field [1]. They are convenient for the purpose because their rules are stated exactly, both players observe the entire state, and a game terminates in an unambiguous result: two methods can be compared by playing them against each other, with no appeal to any judgement outside the game. The chapter that follows traces how those methods developed, from brute-force search with a handcrafted evaluation to systems given nothing beyond the rules.

This thesis takes two such games together, chess and Italian draughts. They are played on the same board and share the same formal properties — deterministic, perfect information, zero sum — but they differ in precisely those respects to which an implementation is most sensitive, and it is that combination which makes the pair useful.

Chess has long been considered an incredibly complex game, with countless openings, intricate tactics, and an almost overwhelming number of possible moves at each turn. This complexity is beyond human capacity to fully calculate, and, despite significant advancements in technology, even the most powerful computers are far from playing perfect chess. In fact, there are approximately 10^{46} possible positions in a chess game [2].

Italian draughts is smaller in every direction, and the difference is not one of degree alone. It uses only the board’s 32 dark squares, has 2 piece types instead of 6, and has no long-range sliding pieces; a position offers, on average, fewer than 6 legal moves against about 35 in chess, a figure reported in Chapter 8, where its consequences for the search are examined. It is also governed by a mandatory-capture rule with a “majority capture” priority that has no equivalent in chess [3]: in chess every legal move is available to the player, whereas in draughts a capture, where one exists, is compulsory.

Two main types of engine dominate the landscape today:

- **Traditional Engines:** Engines like Stockfish [4] rely on brute-force game-tree search. They explore huge numbers of move sequences by minimax or negamax with iterative deepening [5], and prune provably inferior lines with alpha–beta [6]

and supporting heuristics such as move ordering and transposition tables [7]. Leaf positions are scored by an evaluation function external to the search, handcrafted in the classical engines [8] and, in current versions, a small network evaluated incrementally on the CPU [9].

- **Neural Network-based Engines:** Exemplified by AlphaZero [10], these engines replace the handcrafted evaluation with a neural network trained by deep reinforcement learning [11] on games they play against themselves, and use it to guide a Monte Carlo Tree Search (MCTS) [12, 13] in place of alpha-beta.

This thesis follows the second approach, in which the engine learns from the games it plays against itself, and its goal is to build such a system and to understand how it learns. Applying one design to two games, rather than to one, gives that goal a test: it separates what the system can do because of its architecture from what it can do because of design choices tuned to the game it was written for.

The first step is to define a general environment interface for two-player, deterministic, perfect-information board games, covering state representation, legal-move generation and rule enforcement. The interface is then implemented twice: for chess, a high-performance hybrid C/Python implementation with all the classical pieces and rules; for Italian draughts, a lighter one, since its much smaller state and action spaces make many of the optimisations needed for chess unnecessary. Everything above the interface — the search, the encoder, the policy-value network, the inference boundary, the training loop and the conductor that runs self-play and evaluation — follows one specification and is realised once for each game. Chapters 1 to 7 are organised accordingly: each first describes the shared design, then its realisation for chess and for Italian draughts.

The agent chooses its moves with MCTS, the search on which AlphaZero is built. Each simulation descends the tree by a rule that favours moves with a high estimated value while granting an uncertainty bonus to moves visited less often [14], so that the search both deepens good lines and keeps testing alternatives: a balance between exploitation and exploration that is central to reinforcement learning [11]. During self-play, exploration is increased in two further ways: Dirichlet noise is mixed into the network’s prior at the root of the search, and the move actually played is sampled from the visit counts, with a temperature that decreases as the game proceeds [15]. Chapters 2 and 7 give the details.

The search is guided by a policy-value network, a residual convolutional network that gives, for each position, a probability for every move and an estimate of the outcome. It is trained only on the games the engine plays against itself. Each cycle generates self-play games, trains a candidate network on positions from the most recent games, and promotes

the candidate only if it beats the current champion in a match; the champion, old or new, then plays the next cycle's games.

Both engines are complete and described in full in the chapters that follow. The training runs that produced the results of Chapter 8, however, were carried out on Italian draughts alone: a run of the same kind for chess demands computational resources, and above all wall-clock time on dedicated hardware, well beyond what was available for this work.

Through this project, I aim to explore the intersection of game theory, machine learning, and computational efficiency. By developing a fully functional self-learning architecture and validating it on both chess and Italian draughts, I hope to strengthen both engines while gaining a deeper understanding of how machines can autonomously learn and adapt through reinforcement learning, and of which parts of that learning process are general, and which are specific to the game they were trained on.

From Samuel to AlphaZero: The Evolution of Computer Game-Playing

The quest to build a machine capable of playing chess began long before the hardware existed to support it. In 1950, Claude Shannon published his seminal paper, *Programming a Computer for Playing Chess* [1], in which he distinguished between two strategies: "Type A" strategies, which use brute-force search, and "Type B" strategies, which prioritise selective search mimicking human intuition.

Chess was not, however, where machines first learned to play a board game by improving through their own experience. In 1959, Arthur Samuel described a program for the IBM 704 that played checkers (English draughts) and updated its evaluation function by playing games against itself and against earlier versions of itself [16] — an explicit precursor of the self-play and temporal-difference ideas that reinforcement learning would later formalise [17], and that AlphaZero would scale up nearly 60 years afterwards. Draughts, in this sense, is not a late addition to this story but one of its starting points.

For decades, "Type A" dominated. As hardware capabilities followed Moore's Law, engines relied on minimax algorithms optimised by alpha-beta pruning [6]. These systems were defined by two characteristics:

- **Handcrafted Evaluation Functions:** Linear combinations of features (e.g., material count, pawn structure, king safety) weighted by human grandmaster knowledge.
- **Brute-Force Search:** The ability to calculate millions of positions per second to compensate for the lack of deep strategic understanding.

This approach culminated in Deep Blue, the IBM supercomputer that defeated World Champion Garry Kasparov in 1997 [8]. Deep Blue was a triumph of engineering and specific hardware, evaluating on average 126 million positions per second and up to 330 million. Following this, engines like Stockfish [4] refined this paradigm, achieving su-

perhuman ratings (3500+ Elo [18]) purely through algorithmic efficiency and massive handcrafted complexity.

The same paradigm reached an even more definitive conclusion in draughts. Chinook, developed by Jonathan Schaeffer and colleagues at the University of Alberta, contested the Man–Machine World Championship with Marion Tinsley in 1992 and 1994 [19, 20]. Tinsley won the first match; the second was abandoned after 6 drawn games when he withdrew for reasons of health, and the title passed to Chinook, which defended it successfully in 1995. The decisive result came later: after nearly 20 years of exhaustive search combined with endgame databases, the same project weakly solved English draughts in 2007 [21], proving the game a draw under perfect play — a milestone no chess engine has reached, or is expected to. Chinook plays English draughts, not the Italian ruleset studied in this thesis: the two differ in the obligation to take the numerically largest capture and in the rule that a man may not capture a king [3]. It is cited here as draughts’ traditional-engine counterpart to Deep Blue, not as a solved version of this thesis’ game.

While traditional engines were practically unbeatable, they were theoretically limited by their reliance on human domain knowledge. The break with that reliance came first in Go: AlphaGo [22] combined supervised learning from a database of human games with reinforcement learning from self-play, and in 2016 defeated Lee Sedol, one of the strongest players of the preceding decade. The following year AlphaGo Zero [15] removed the human games from the procedure entirely, learning from random initialisation by self-play alone, and surpassed the version that had won that match. In late 2017, DeepMind introduced AlphaZero [10, 23], a single algorithm applied to chess, shogi and Go alike. Unlike the traditional engines, AlphaZero began with no domain knowledge beyond the rules of the game (tabula rasa approach). It did not use an opening book, endgame tablebases, or handcrafted evaluation heuristics. Instead, it reached superhuman performance through deep reinforcement learning and self-play [11].

AlphaZero replaced the linear evaluation function of traditional engines with a deep ResNet (Residual Neural Network) [24]. This network, denoted as f_θ , takes the board state s as input and outputs a vector containing two distinct heads:

$$(p, v) = f_\theta(s)$$

- **The Policy Head (p):** A probability distribution over all legal moves, estimating the likelihood that a strong player would select a specific move.
- **The Value Head (v):** A scalar in the range $[-1, 1]$ estimating the expected outcome of the game from state s (where $+1$ is a win, 0 is a draw, and -1 is a loss).

This architecture allows the engine to evaluate a position not by counting material, but by recognising visual patterns and spatial relationships on the board, much like a human intuition but scaled to high-dimensional space.

The critical innovation of AlphaZero was replacing alpha–beta pruning with a variant of Monte Carlo Tree Search (MCTS) [12, 13]. While traditional MCTS relies on random “rollouts” to the end of a game to estimate value [25], AlphaZero uses its neural network to truncate the search. When the search reaches a leaf node s_L , the network evaluates it, providing a value v to backpropagate up the tree and a policy p to guide future exploration. The selection of nodes during the search is governed by the PUCT (Predictor + Upper Confidence Bound applied to Trees) algorithm [14]:

$$a_t = \arg \max_a (Q(s, a) + U(s, a))$$

Here, $Q(s, a)$ is the mean value of the simulations that have already passed through move a , and drives exploitation, while $U(s, a)$ drives exploration. The exploration term is proportional to $P(s, a)$, the probability the policy head assigns to move a in position s — that is, the component of p corresponding to that move — and it shrinks each time the move is visited. The prior therefore decides which moves the search tries first, before any of them has been tried at all, and the value observed in the tree decides which of them are worth trying again.

The training pipeline is a cycle of improvement. The agent plays games against itself, generating a dataset of positions. The network parameters θ are updated to minimise a composite loss function [15]:

$$l = (z - v)^2 - \pi^\top \log p + c \|\theta\|^2$$

where:

- $(z - v)^2$ is the mean squared error between the predicted value v and the actual game outcome z ;
- $-\pi^\top \log p$ is the cross-entropy loss attempting to make the network’s policy p match the improved search probabilities π generated by the MCTS;
- $c \|\theta\|^2$ is an L_2 regularisation term.

In its 2017 match against Stockfish 8, AlphaZero achieved a resounding victory (28 wins, 72 draws, 0 losses) after only 4 hours of training [23], demonstrating that self-learned intuition could outperform decades of human-refined heuristics.

The publication of the AlphaZero methodology sparked an open-source revolution. The Leela Chess Zero (Lc0) project [26] successfully replicated DeepMind’s results using distributed computing, proving that the approach was reproducible and scalable.

Furthermore, the "traditional" engines adapted. In 2020, Stockfish introduced NNUE (Efficiently Updatable Neural Networks) [9]. Unlike the deep convolutional networks of AlphaZero which run on GPUs, NNUE is a shallow architecture that runs efficiently on CPUs. This hybrid approach combines the strategic understanding of neural networks with the raw speed of alpha–beta search.

For the purpose of this work, the AlphaZero methodology serves as the fundamental reference point for both engines developed in this research. By restricting each system to learning solely through the mathematics of reinforcement learning and self-play, I can isolate the learning process itself, independently of the game it is applied to. This allows me to observe how an agent autonomously constructs complex strategic models not as hard-coded instructions, but as emergent properties of an optimisation process driven entirely by the rules of the game it is given, deducing advanced concepts such as piece activity, king safety, and space advantage in chess, or breakthrough timing and king control in draughts.

However, for chess specifically, the computational disparity between industrial-grade infrastructure and standard hardware resources necessitated a pragmatic adaptation: curriculum learning [27]. Recognising that a pure tabula rasa approach faces significant convergence hurdles in resource-constrained environments, the chess engine introduces decaying auxiliary penalties. By temporarily shaping the reward function to discourage immediate tactical failures — such as exposing the king or discarding material — the system bridges the initial “cold start” gap. These penalties act as transient scaffolding, designed to fade away as the network matures, ensuring that the engine ultimately relies on its own self-taught evaluation rather than permanent human bias.

The draughts engine, by contrast, needed no such scaffolding: trained directly from tabula rasa on a policy and value loss alone, with no reward shaping of any kind, it converged reliably. This is itself a useful data point, and the first of several in this thesis to point the same way: the need for curriculum learning turned out to be a property of chess’s branching factor and search depth under limited hardware, not of the underlying architecture. Section 8.6 measures a second instance of the same principle, in which a parameter that is harmless in a wide game proves costly in a narrow one.

1 | The Environment: State Representation and Game Mechanics

1.1. The Environment Layer

The environment layer implements the rules of the game. It is the one component that is entirely specific to the game being played: the search, the network and the training loop follow a common specification, realised once for each game, and access the rules only through the interface described here.

Its interface is small. It must enumerate a position's legal moves, return the position each of them leads to, and decide whether play has ended and with what outcome. It does not evaluate positions, order moves, or assign non-terminal rewards. Those are the responsibility of the components above it.

Two requirements follow from the use the layer is put to, and both are imposed by tree search [12, 13] rather than by the games themselves.

The first concerns state. A search visits a large number of positions that are never played, and must be able to enter and leave any of them without altering the official game state. State maintained implicitly — a cached evaluation, a counter updated as a side effect — can survive a branch that was abandoned, producing a discrepancy between the engine's internal state and the position it reports. Caches are therefore admitted only where they are transparent: they may reduce the cost of a query, but must not change its result.

The second concerns the cost of traversal. A search that spends more time constructing states than evaluating them is limited by the wrong quantity. How this is achieved is not fixed by the requirement, and it is the principal axis along which the two implementations differ. Chess is large enough to require an engineered representation: a compiled, bit-level board, an incrementally maintained hash, and a transition reversed by restoring a saved

snapshot rather than by copying the environment. Italian draughts is small enough for the same guarantees to be obtained from an interpreted representation that copies the state on every move. The sections below describe each in turn, and most of the differences between them follow from this difference in scale.

One convention is shared by both. Rewards are neutral at every non-terminal step [11], and terminal outcomes are adjudicated by the environment rather than claimed by a player. The second point departs from the written rules of both games, and is required because self-play has no player to make a claim.

1.2. The Chess Environment

1.2.1. State Model

The core logic is a bitboard implementation written in C and wrapped in Python. In support of efficient tree search, the implementation maintains an explicit 64-bit Zobrist hash [7]¹ updated incrementally in C, alongside a lightweight transposition table managed in Python. The table is a transparent cache and never affects the rules; the hash also identifies positions for the threefold-repetition rule.

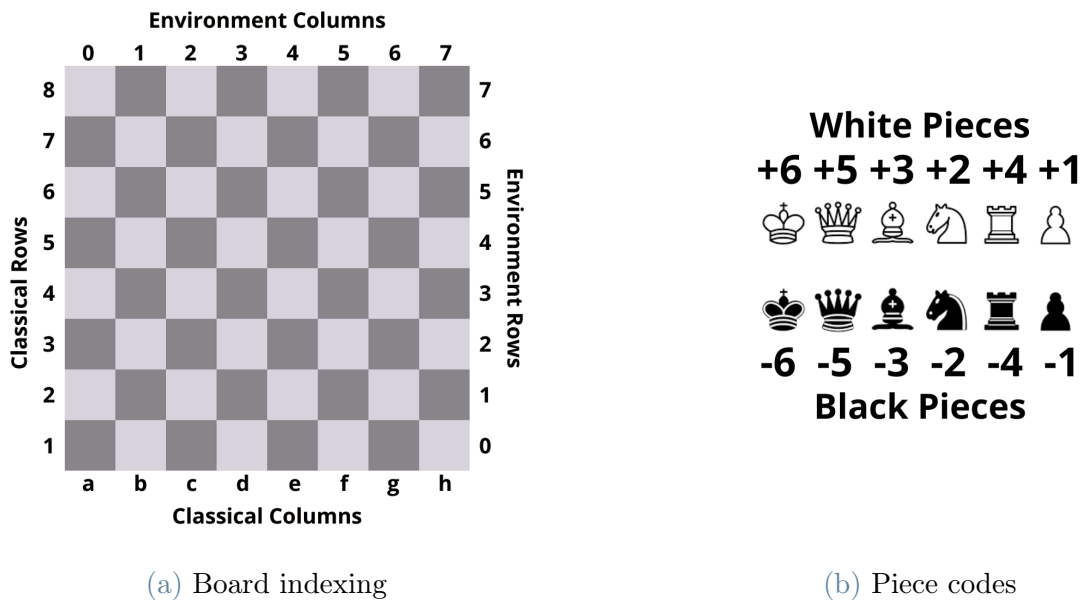


Figure 1.1: The board model the chess environment exposes to Python, as distinct from the bitboards it uses internally.

¹Zobrist hashing is a technique used in chess engines to uniquely represent board positions using a hash value. Each square on the board and each piece type is assigned a random number, and the position's hash is calculated by combining the corresponding values of all pieces on the board.

While the underlying C engine represents the board using 64-bit bitboards [5]² (12 for piece types and 2 for colours), the Python interface exposes a board model represented as an 8×8 integer matrix. Each square is encoded as either 0 (empty), a positive value (white pieces), or a negative value (black pieces). Piece codes are: pawn = 1, knight = 2, bishop = 3, rook = 4, queen = 5, king = 6. Figure 1.1 shows both conventions: the square indexing in panel (a) and the piece codes in panel (b). The rows are indexed from 0 (White’s back rank) to 7 (Black’s back rank), while the columns are arranged from the queenside (a-file, index 0) to the kingside (h-file, index 7). Moves are represented as simple four-tuples specifying the source and destination coordinates in (source row, source column, destination row, destination column) format. This compact representation mirrors typical reinforcement learning tensor layouts. For example, the bishop move f1→c4 is (0, 5, 3, 2): from row 0, column 5 to row 3, column 2, the convention illustrated in Figure 1.2.

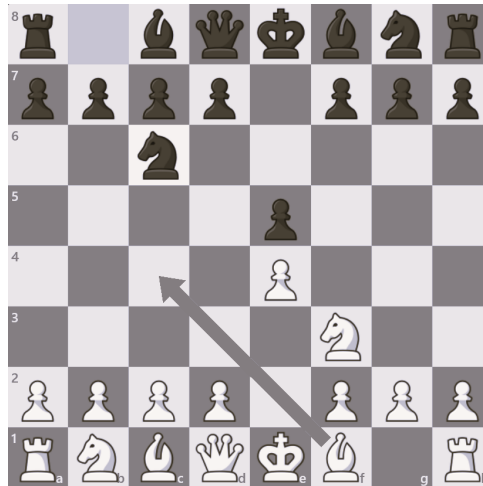


Figure 1.2: Move representation in the chess environment

In terms of state management, the C struct explicitly maintains a variety of essential elements: the side to move, the en-passant³ square index (ranging from 0 to 63, or -1 if none), 4-bit castling rights, a half-move clock enforcing the fifty-move rule⁴, and the full-move number. The game initialises from the standard setup, with all castling rights available (represented internally as the bits 1111), no en-passant target, counters reset, and a strict bitboard initialisation pass to ensure positional consistency before play begins.

²Bitboards are a data structure used in chess engines to represent the state of the board using bitmaps. Each piece type and colour is represented as a 64-bit integer, where each bit corresponds to a square on the chessboard.

³En passant is a special chess move where a pawn, after moving two squares forward from its starting position, can be captured by an opponent’s pawn as though it had only moved one square forward. This move must be done immediately after the two-square move.

⁴The fifty-move rule in chess states that if 50 consecutive moves are made by both players without any pawn movement or capture, either player can claim a draw.

For search and deduplication, each position carries a 64-bit Zobrist hash that is updated incrementally in the C core. This hash combines: (i) a 12×64 piece-square table (P, N, B, R, Q, K for White then Black), (ii) side-to-move, (iii) the 16 possible states of castling rights, and (iv) the exact en-passant square (or a “none” state). Threefold repetition is tracked directly via this C-computed Zobrist hash on the Python side, aligning perfectly with rule-level equality of positions while maximising lookup efficiency.

Transitions are reversible. The C core utilises a dedicated internal history array (capable of storing up to 2048 plies) that pushes and pops full board states, allowing constant-time state reversion. Promotion⁵ is automatically handled natively in C as an auto-queen, simplifying the action space. The fast push and pop methods allow search algorithms to explore branches deeply without the overhead of deep copies — in fact, cloning the environment is explicitly bypassed in favour of this push/pop mechanism. Reversible updates symmetrically maintain the position hash natively; position-repetition counters on the Python side are updated only when the official game state advances, ensuring search explorations do not pollute gameplay bookkeeping. Figure 1.4 shows the two operations and the state snapshot they exchange.

1.2.2. Move Lifecycle

Move generation is performed in two stages. First, the environment generates pseudo-legal moves, considering all piece rules, including special cases like double pawn pushes, en-passant captures, knight jumps, and sliding moves for bishops, rooks, and queens. Second, these candidates are filtered for legality by temporarily making each move and testing whether the moving side’s king would be in check. This check ensures that only legal moves are applied, and relies on the push/pop⁶ machinery. The en-passant rule follows FIDE⁷ semantics [28], with the en-passant square being valid for exactly one reply after a double pawn push. Both move generation and execution ensure that the captured pawn is properly removed. For simplicity and performance, promotion is automatically handled natively in the C core as a queen upon reaching the last rank.

Game termination is evaluated after each move. The environment checks for checkmate (if the opponent is in check and has no legal moves), stalemate (if the opponent has no legal

⁵Pawn promotion occurs when a pawn reaches the opponent’s back rank and is then converted into a queen, rook, bishop, or knight.

⁶The push/pop mechanism is a technique used in move generation and execution where the game state is temporarily modified (push) by making a move, and then reverted (pop) if the move is invalid or after checking for legality. This ensures that the environment can quickly test multiple move possibilities without permanently altering the game state.

⁷FIDE (Fédération Internationale des Échecs) is the international governing body for chess, responsible for overseeing chess competitions and establishing the official rules of the game.

moves and is not in check), and draws due to the fifty-move rule or threefold repetition.

To ensure robust king safety, attack maps are generated directly from the board, allowing for precise and reliable check detection. The environment has dedicated attack helpers for pawns, knights, and kings, while sliding pieces trace rays until blocked, including the first opposing blocker square. This separation of concerns ensures that legality checks remain robust and easy to reason about.

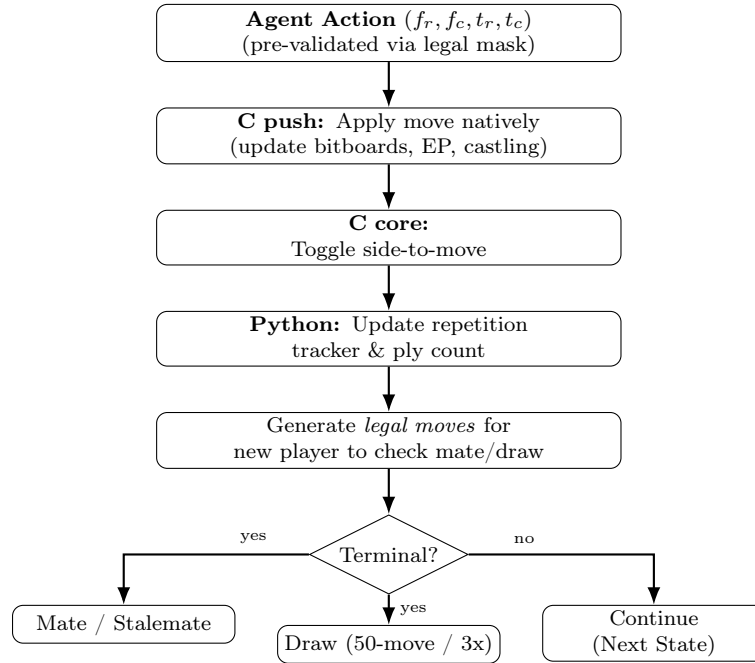


Figure 1.3: One-move lifecycle in the hybrid C/Python environment.

The environment’s step function is optimised for reinforcement learning agents that select actions from a precomputed list of legal moves. Rather than performing redundant boundary or format validation, it directly applies the move via the C extension. After executing the move, the C system internally updates special cases, such as en-passant, castling rights, and the half-move clock, while toggling the side to move. The Python wrapper then tracks the position count for repetition. Both non-terminal steps and terminal outcomes return the updated board with a neutral reward, alongside an info dictionary containing a result tag (e.g., “checkmate” or “stalemate”) for logging and training purposes.

To maintain compatibility with standard Python APIs [29], a compact exception hierarchy is defined, encompassing action errors and state errors. However, to maximise throughput during intensive search or exploratory machine learning workflows, the fast-path execution assumes invariants are respected by the agent’s policy mask rather than actively raising these exceptions.

Figure 1.3 summarises the one-move lifecycle. A candidate move — specified by source and destination coordinates — is passed directly to the engine. The position is updated natively and the side to move is switched, while a complete, lightweight snapshot of the entire board state is saved directly into an internal history buffer. This allows for instantaneous and robust state reversal without the risk of desynchronisation. The resulting position is then assessed for termination: checkmate, stalemate, or a draw by the fifty-move rule or threefold repetition yield a terminal outcome; otherwise, play proceeds with the new state.

1.2.3. Performance, Extensions and Tooling

Unlike implementations that rely solely on array manipulation, this core is engineered upon a highly optimised 64-bit bitboard architecture. To accelerate tree search and position deduplication, the implementation includes native Zobrist hashing calculated continuously in C, paired with a lightweight transposition table managed in Python.

For testing and developer experience, the environment supports FEN⁸ export, enabling the capture of arbitrary positions for curricula or edge-case validation. It also supports SAN⁹ and PGN¹⁰ formats for human-readable tracing and external tool interoperability. The environment includes a render helper for string-based terminal debugging, as well as factored policy helpers designed specifically to interface with tensor-based learning pipelines.

Three rules-level extensions are left open: under-promotion, the detection of dead positions, and the import of positions in FEN.

1.3. The Italian Draughts Environment

1.3.1. State Model

Italian draughts starts from the same two requirements — explicit state and cheap traversal — but meets them with far less machinery, because the problem itself is much smaller. The board is represented as a plain, interpreted list of 64 integers rather than a set of bitboards: with a much smaller state and action space than chess, the performance gained

⁸FEN (Forsyth-Edwards Notation) is a standard notation used to represent a chess position.

⁹SAN (Standard Algebraic Notation) is a widely used notation for recording chess moves. It specifies the piece moved, the destination square, and any captures, checks, or special moves in a concise format.

¹⁰PGN (Portable Game Notation) is a standard format for storing chess game data. It includes the game's moves in SAN, along with metadata such as player names, event details, and the result of the game.

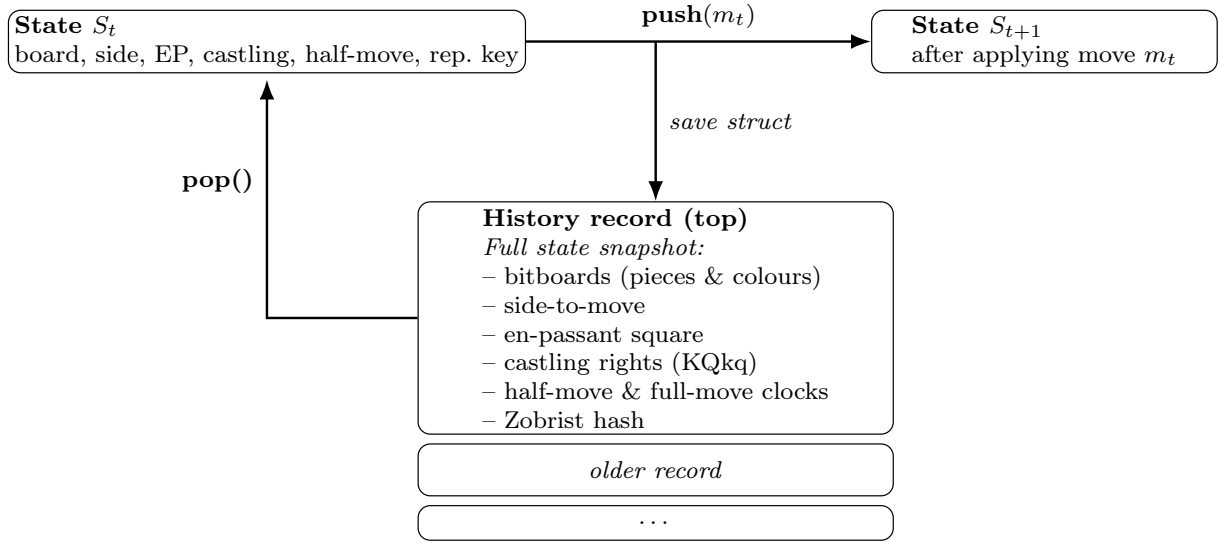


Figure 1.4: Reversible transitions: **push** applies m_t and saves a full state snapshot to the C history array; **pop** restores S_t instantly by overwriting the current state.

by a compiled, bit-level representation would not have justified the added complexity. The signed-integer encoding follows the same idea as in chess — the sign of the integer gives the colour, its magnitude gives the piece type — but there are only 2 piece types instead of 6, and, since draughts is played only on the dark squares, only half of the 64 indices are ever occupied; the other half exist purely so that row and column arithmetic stays identical to the chess implementation’s.

State is correspondingly minimal: a board, the side to move, and a counter of plies played without a capture or a promotion. The counter drives the only draw condition implemented, a draw after 80 consecutive such plies. This simplifies the federal rule [3], which grants a draw on request after 40 moves by the counted player, with at least one king per side, and restarts the count at every move of a man; the implemented rule therefore always adjudicates earlier than the federal one would, never later. There is no equivalent of castling rights or an en-passant target, because draughts has neither castling nor en passant.

A move is a richer object here than the 4 coordinates chess uses, and it has to be, because a single draughts move may traverse several squares and remove several pieces. Each move carries its origin and destination, the full sequence of squares it passes through, the list of pieces it captures, whether it promotes the moving piece, and whether it was made by a king rather than a man. Both of the last two are used by the move generator: promotion terminates the turn, and the type of the moving piece enters the capture-priority ordering described below. The index of a move in the fixed action space used by the network is

not stored with the move: it depends on the orientation in which the encoder presents the board, and is computed there (Chapter 3).

The two implementations part ways on how a move is applied. Chess mutates its state in place and can undo a move by restoring a saved snapshot, which avoids copying the board on every step of a deep search. Draughts does the opposite: applying a move produces an entirely new state, leaving the old one untouched. This is not an oversight but the same reasoning applied to a much smaller problem: with far fewer occupied squares and fewer piece types, copying the state on every move never became expensive enough to justify the added complexity of a reversible, in-place representation. Because a state is never modified once created, a search node can retain a reference to the state it represents rather than a copy of it.

Positions are also deduplicated differently: rather than maintaining an incrementally updated hash, equality between two states is established directly from their raw contents, cheap enough at this scale to be recomputed on demand instead of tracked continuously. The cost is a memory footprint that is not bounded in advance; the benefit is that a lookup either matches exactly or misses, so no collision handling is required.

One structural property of this design is worth stating explicitly, because later chapters rely on it. In the pipeline, every position is reached from the initial one through moves generated by this environment, so no illegal position can arise, and the layers above it perform no validation of their own.

1.3.2. Move Lifecycle

Move generation in Italian draughts follows a different logic from the two-stage pseudo-legal-then-filter approach used for chess, and the difference is forced by the rules rather than chosen. Captures are searched first, and the search is exhaustive along every capturing chain: a piece that can capture is followed recursively through every sequence of jumps it can make. Its starting square is treated as empty for the duration, so that a king's chain can pass through or end on the square it started from, while the pieces it jumps over stay on the board, blocking any second jump over them, and are removed only once the whole chain is fixed. If a piece reaches the last row in the middle of such a chain it is promoted immediately, and promotion always ends the turn on the spot — no further jumps are taken, even if the newly promoted piece could technically continue capturing. If any capturing move exists anywhere on the board, capturing is mandatory and non-capturing moves are not even generated; only when no capture is available does the engine fall back to generating single-step diagonal moves.

Because more than one capturing chain can be available at once, the rules require the player to take the numerically best one, a priority known as majority capture [3]: the engine first keeps only the chains that capture the largest number of pieces, then, among those, prefers any chain carried out by a king over one carried out by a man, then prefers the chain that captures the most kings, and finally, if a tie remains, prefers the chain that captures a king earliest in the sequence. This cascade has no counterpart in chess, where every legal move is equally admissible and the only filter is whether it leaves the king in check. A position in which a capture is available therefore often admits a single legal move, since the priority ordering frequently resolves to one chain. Figure 1.5 sets out the resulting order of operations.

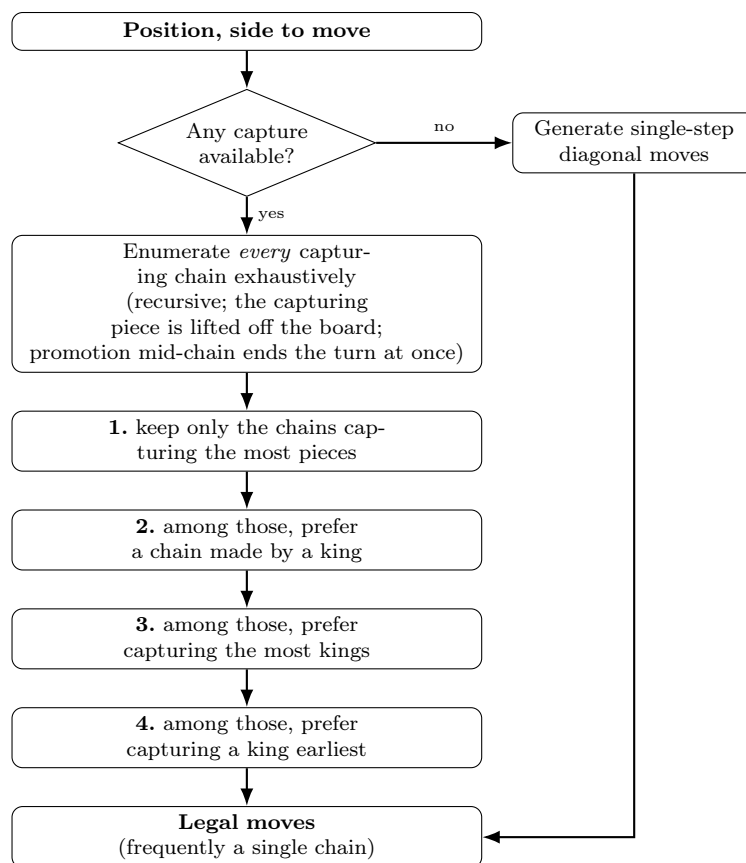


Figure 1.5: Move generation in Italian draughts: the test for a mandatory capture, the exhaustive enumeration of capturing chains, and the four priority filters applied in order.

Termination is correspondingly simpler than in chess. There is no separate notion of stalemate: a player who has no legal move, whether because all of their pieces have been captured or because every remaining piece is blocked, has simply lost. The only other way a game can end is the no-progress draw described above. There is no separate repetition rule to track: because that same ply counter increases on every move that is neither a

capture nor a promotion, any repeating sequence of quiet moves is already bounded by it, which makes a dedicated repetition detector unnecessary.

1.3.3. Tooling

Italian draughts needed a much lighter layer of tooling, again because the problem it addresses is smaller. There was no need for anything equivalent to algebraic notation, portable game records, or position-import formats, and a simple text rendering of the board for debugging was sufficient.

The one piece of tooling that does carry over conceptually is a compact, precomputed action space that maps each of the 32 playable squares to a small fixed number of possible moves from that square, serving the same purpose as the chess engine's factored policy helpers: giving the neural network a fixed, dense set of output slots rather than a space sized for the full generality chess needs. The exact layout of this action space is described together with the rest of the input and output encoding in Chapter 3.

2 | The Search Agent: Monte Carlo Tree Search Implementation

2.1. The Search Agent

The environment of Chapter 1 can say which moves are legal and what position each produces, but it has no notion of which move is better. Supplying that judgement is the task of the agent, and it does so by searching: it explores the tree of continuations, evaluates positions it reaches, and returns the move that the accumulated evidence favours.

The search used here is a Monte Carlo Tree Search [12, 25] of the kind introduced by AlphaGo Zero and generalised by AlphaZero [10, 15]. Two properties made it the appropriate choice. It is agnostic to how positions are evaluated, so the same search core operates unchanged whether the evaluation comes from a hand-written heuristic or from a neural network, and it improves as the evaluation improves rather than requiring redesign. And it is anytime: the quality of its answer degrades gracefully with the computational budget, which matters for a system that must search millions of positions during training and can afford only a few hundred simulations per move.

A single simulation consists of four phases [13], and the search repeats them until its budget is exhausted.

Selection

Starting at the root, the search descends the tree by repeatedly choosing among the children of the current node until it reaches one that has not yet been expanded. The

choice is governed by a PUCT-style score¹ [14], which combines two terms:

$$a_t = \arg \max_a (Q(s, a) + U(s, a))$$

The first, $Q(s, a)$, is the mean value accumulated for that move so far, and drives exploitation. The second, $U(s, a)$, is an exploration bonus that grows with the network’s prior probability for the move and shrinks as the move is visited, so that a plausible but untried move is examined before an implausible one, and any move eventually receives attention if the alternatives fail to justify themselves.

Expansion and Evaluation

On reaching an unexpanded node, the agent obtains the legal moves for that position, evaluates it with the network, and creates one child per legal move, initialised with the prior the network assigns it. Priors are restricted to the legal set and renormalised, so that no probability mass is ever assigned to a move that cannot be played.

Terminal positions are handled separately and never reach the network. A terminal position, won, lost or drawn under the rules of either game, receives its defined value directly and proceeds straight to backup. This is not only a saving: the network’s estimate of a position whose outcome is already determined is an approximation of a known quantity, and using it in place of the true value would introduce error where none need exist.

Positions already evaluated are served from a cache rather than re-evaluated; the two engines implement it differently, as described below. Because the same position can be reached by more than one sequence of moves, this saves a substantial number of network calls in games where transpositions are common.

Leaf Batching

Evaluating one position per network call leaves the hardware idle for most of the search, since a single position is far too small a workload to occupy a modern accelerator. The search therefore collects several unexpanded leaves before issuing one call for all of them, and processes the results together. Each such group is referred to here as a wave: with a budget of n simulations and a batch of b leaves, the search performs n/b waves, and reacts to its own results only that many times.

¹PUCT (Predictor + Upper Confidence bounds applied to Trees) extends the classical UCT rule with a prior supplied by a policy network, which biases the search towards moves the network considers plausible before any of them has been tried.

Collecting distinct leaves requires an additional device, because the selection rule is deterministic given the current statistics: without intervention, every leaf in a wave would be the same one. Before a selected path is handed to the batch, therefore, a temporary penalty — a virtual loss [30] — is added to each node along it, which makes that path less attractive to the next selection within the same wave. The penalty is removed when the true value returns and the backup is performed, so it never affects the statistics the search finally reports.

This mechanism is the reason the batch size is not a purely computational setting. The larger the batch, the more the search is pushed away from the line it would have chosen and the fewer opportunities it has to react to what it finds. Section 8.6 measures the effect of that trade-off.

Backup

The value obtained for a leaf is propagated back along the path to the root, incrementing each node’s visit count and accumulating its value with alternating sign, so that every node holds its value from the perspective of the player to move at that node. A position that is good for one player is by definition bad for the other, and the alternation encodes exactly that.

Move Selection

When the budget is exhausted, the move played is chosen from the visit counts rather than from the accumulated values. A move that the search has examined many times has survived repeated comparison against its alternatives, whereas a high mean value may rest on a single fortunate evaluation. By default the most-visited move is played, with random tie-breaking.

Two devices modify this during training [15]. A temperature parameter τ makes the choice stochastic, sampling in proportion to $N(a)^{1/\tau}$: a value near 1 early in the game diversifies the openings, and a value approaching 0 later recovers the greedy choice. And Dirichlet noise is mixed into the priors at the root only, so that the search is obliged to examine moves the network currently considers unpromising. Both are training devices: without them, self-play with a deterministic network would produce the same game repeatedly and the replay buffer would contain a single line of play. The pseudocode below brings the phases together in a single wave of simulations, from selection to backup.

One wave of simulations in MCTS (pseudocode)

```

def mcts_wave(root, batch_size):
    pending = []
    for _ in range(batch_size):
        node, path = select_to_leaf(root)    # PUCT; adds virtual
            loss on path
        if node.is_terminal():
            backup(path, node.terminal_value())    # no network call
        else:
            pending.append((node, path))

    misses = []
    for node, path in pending:
        hit = eval_cache.get(node.state)          # transposition
        if hit is not None:
            priors, value = hit
            node.expand(legal_moves(node.state), priors)
            backup(path, value)
        else:
            misses.append((node, path))

    if misses:                                  # ONE network
        call per wave
        results = f_theta([n.state for n, _ in misses])
        for (node, path), (priors, value) in zip(misses, results):
            eval_cache[node.state] = (priors, value)
            node.expand(legal_moves(node.state), priors)
            backup(path, value)

def backup(path, value):
    remove_virtual_loss(path)
    for n in reversed(path):
        n.visit_count += 1
        n.value_sum += value
        value = -value                          # alternate
            perspective

```

Dirichlet noise (root, training only): $P' = (1 - \varepsilon)P + \varepsilon \text{Dir}(\alpha)$. Applied to legal moves and renormalised.

Temperature (move choice): sample $\pi(a) \propto N(a)^{1/\tau}$; higher $\tau \approx 1$ early for diversity, then $\tau \rightarrow 0$ (argmax) later and at evaluation.

Everything described so far is shared by both engines as a specification, and realised once

for each. What remains to each of them falls into three categories: mechanisms that reduce the cost of the search and are worth their own complexity only above a certain scale; mechanisms that pursue the same goal by different means; and problems that arise in one game and not the other. The two sections that follow are organised alike, so that the corresponding decisions can be read against each other.

2.2. The Chess Agent

2.2.1. Search Control

At the start of each search the agent prepares the root. If the previous root's children remain legal and the same side is to move, it reuses that subtree and preserves the statistics accumulated in it; if children have become illegal or the environment has been reset, it rebuilds the root from scratch. This reuse policy is conservative by design: if any previously cached child has become illegal, the agent rebuilds the root rather than salvaging a subset of children, which avoids subtle desynchronisations. Root priors are refreshed every move regardless.

A quick mate-in-one check runs at the root, ending the search immediately in positions where a mate in one is available and there is nothing to deliberate about.

To avoid deterministic tie-breaking, children are shuffled before the best candidate is chosen. To manage the branching factor and focus computational resources, the search employs progressive widening [31]: the number of children introduced at a node grows with its visit count, so that a node visited few times considers only the moves the network rates most highly, and the remainder are admitted as evidence accumulates. This prunes the tree without relying on a fixed probability cutoff, which would discard a move that the prior underrates.

At the root during training, a small repetition penalty can be applied to discourage immediate twofold repetitions.

Early stopping watches the visit distribution: if a single move surpasses a dominance threshold — more than 60% of total visits, after a minimum number of simulations — the search halts and returns it, since further simulations are unlikely to change the answer. Figure 2.1 places these mechanisms on the course of a single search, from the preparation of the root to the move finally played.

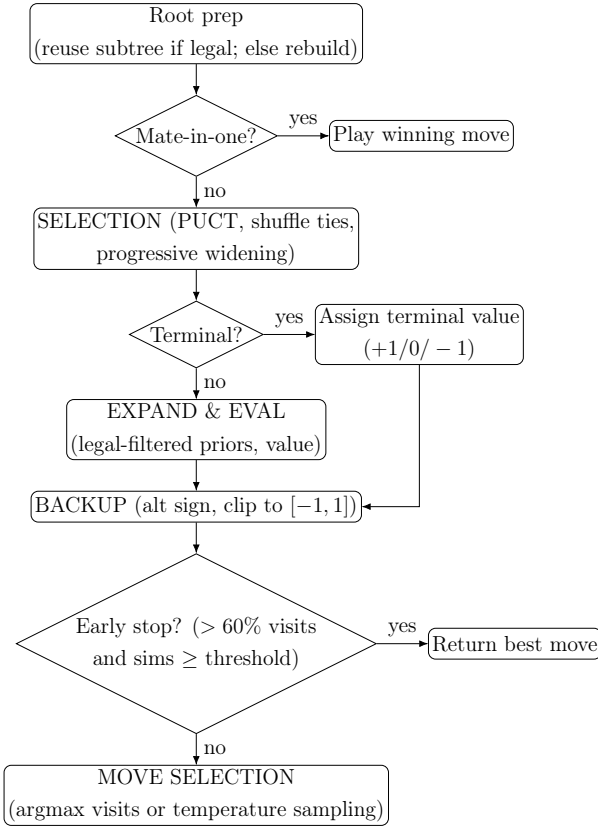


Figure 2.1: End-to-end search flow in the chess engine; Figure 2.2 gives the draughts equivalent.

2.2.2. Priors and the Policy Head

When a factored policy head is used, the agent consumes 64×64 from/delta logits, restricted to auto-queen promotions, and combines them with a masked softmax over the legal factors; a legacy flat policy head is also supported. The expansion phase relies entirely on the upstream neural adapter to provide a strictly legal and correctly normalised distribution. By delegating legality masking and normalisation to that boundary, the search core performs no validation of its own, which keeps expansion throughput high.

2.2.3. State Handling and Caching

Performance rests on minimising state allocations. The environment acts as a low-allocation engine that applies a move internally and reverses it afterwards, which eliminates the cost of deep-copying the environment at each node expansion; cloning is explicitly bypassed in favour of this mechanism.

Leaf evaluation is batched through a Central Inference Server. Requests from many

concurrent games are accumulated in queues and micro-batched, up to 256 states per flush, which allows high FP16 tensor throughput across parallel worker processes.

To prevent redundant evaluations across transposed paths, the system employs a fixed-size transposition table mapping the 64-bit Zobrist key [7] directly to the position's scalar value. Collisions are handled by a fast, lock-free replacement strategy that overwrites older entries. This prioritises throughput and low latency over precise eviction, at the cost of admitting, in principle, a false match between two positions that share a key.

2.2.4. Robustness

Robustness guards are actively enforced. The root is always a verified legal position, and illegal actions are excluded from the priors at expansion. Before any state change, candidate moves are re-validated against the current legal set. If a desynchronisation or a stale cache entry introduces an illegal child, that branch is pruned automatically, allowing the search to continue without bringing down the multiprocessing pool.

2.3. The Italian Draughts Agent

2.3.1. Search Control

Every search starts from a fresh, single-node root built from the current position, with no attempt to recycle statistics from the previous move. The bookkeeping required to reuse a subtree safely — verifying that its children are still legal, discarding it if any is not — costs more here than rebuilding, because the tree is shallow and each node is cheap to create.

There is no mate-in-one shortcut. A forced win is found by the search itself: with mandatory capture and a small branching factor, a winning capture tends to surface within the first few simulations regardless.

Selection uses the same PUCT-style score, but children are not shuffled before the best is chosen; ties are broken by the fixed order in which the moves were generated. With only a handful of legal moves in most positions, the practical effect of that choice is small.

Progressive widening is not used either. The branching factor is small enough that every legal move is expanded into a child as soon as its node is first reached, rather than introduced gradually: with roughly 6 children at a typical node, there is nothing for a widening schedule to withhold.

There is no repetition penalty at the root, consistent with the environment not tracking repetitions at all. As described in Chapter 1, the no-progress counter already bounds how long any repeating sequence can last.

Finally, there is no early stopping: every search runs its full simulation budget to completion. With a shallower tree and a much lighter network, the simulations that early stopping would skip are inexpensive enough that skipping them would save little. Figure 2.2 places these six differences on the flow of Figure 2.1.

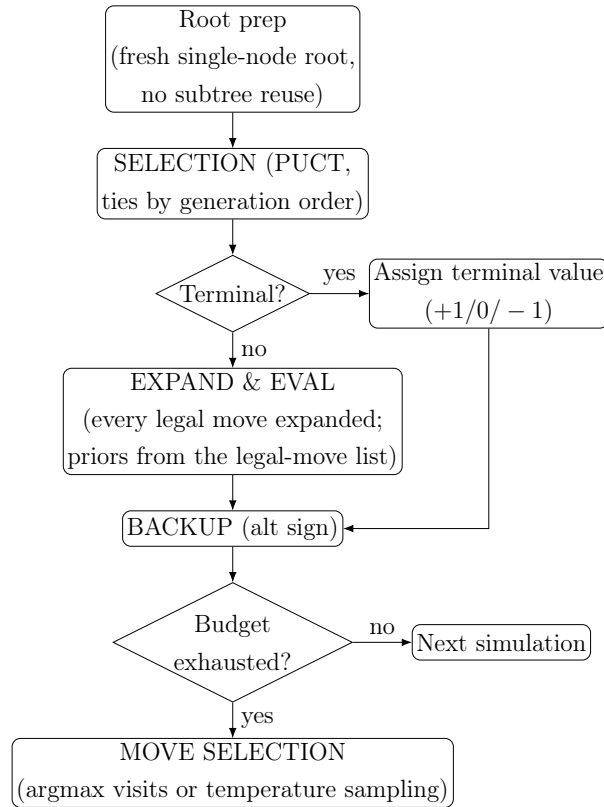


Figure 2.2: End-to-end search flow in the draughts engine, drawn on the same skeleton as Figure 2.1 so that the two can be read against each other.

2.3.2. Priors and the Policy Head

The policy head is always the flat kind. The action space is already small enough that a factored decomposition into a from-square and a conditional delta buys nothing: once the origin, direction and mode of a move are fixed, the destination follows from the rules rather than being a further choice for the network to weigh.

Illegal actions never enter the priors in the first place, because the priors are built directly from the position’s own legal-move list rather than filtered from a larger candidate set

afterwards.

2.3.3. State Handling and Caching

The environment provides no reversible transition to exploit, since every move produces a new state. The search compensates differently: each node holds a reference to the state it represents, and that state is computed once — the first time the search descends through that edge — rather than being recomputed by every simulation that later passes through it. Chess avoids the cost of copying entirely; draughts pays it at most once per edge of the tree, which is acceptable because each individual copy is already cheap.

Leaf evaluation is batched: each search accumulates its own pending leaves — 8 by default — before making a single network call. On the CPU, parallelism across games comes from running several independent search processes, each holding its own copy of the network; on a GPU, the same arrangement as in chess is used, with a single server process that owns the device and batches the requests of all workers, up to 256 positions per call.

The transposition cache makes the opposite trade-off to the chess table. Rather than a fixed-size structure indexed by a hash, it is a plain cache indexed by the exact contents of a position, allowed to grow for the duration of a single search. A lookup either matches exactly or misses, so there is nothing to evict and no possibility of two distinct positions being confused. The cost is a memory footprint that is not bounded in advance, which is acceptable because the number of distinct positions reachable within one search is small.

2.3.4. Robustness

The compact action space introduced in Chapter 1 creates a problem that does not arise in chess: because it maps each move to one of a small fixed number of slots per square, two structurally different legal moves can occasionally be assigned the same slot, most often when more than one capturing chain begins with an identical first step. Rather than letting one silently overwrite the other, the probability the network assigns to the shared slot is divided between the moves that claim it, so that the priors still sum correctly and no legal move is given zero weight by accident. The mechanism belongs to the adapter and is described in Chapter 5.

Separately, if a self-play game fails for any reason during a run across many parallel worker processes, that single game is discarded rather than allowed to bring down the rest of the pool, so a run of many games can absorb an occasional failure without losing the work already in progress.

3 | Feature Encoding: Bridging State Space and Neural Inputs

3.1. Feature Encoding

The encoder maps a position from the form the environment finds convenient for generating moves into the tensor the network requires, of fixed shape and with a fixed meaning assigned to every channel; it is the only component that knows both representations. Its contract is narrow. Given a position it must produce a tensor of a shape that never varies, with the same channel carrying the same meaning at every position and in every game; the mapping must be deterministic, so that the same position always yields the same tensor; and it must be free of side effects, so that encoding a position during search leaves nothing behind. Batching and device placement are deliberately handled outside this step, which keeps the encoder portable and allows the same code to serve training, self-play and evaluation without modification.

Both encoders describe a single static position rather than a history of recent ones. The network therefore receives an instantaneous snapshot, and any information about how the position was reached must either be summarised into a plane or be lost.

Perspective

A network trained on positions presented from an arbitrary point of view would have to learn each strategic pattern twice, once for each colour. Every encoder must therefore impose a consistent perspective, and there are two ways of doing so.

The first keeps the board in an absolute frame — one player's pieces always occupy the same planes — and adds a channel that declares whose turn it is. The network sees a fixed geometry and must learn to condition its judgement on that channel.

The second canonicalises the position before encoding it: the board is transformed so that the side to move always appears in the same orientation and in the same planes. The

same physical position then produces two different tensors depending on who is to move, and the network only ever learns what a good position looks like for the player about to act.

Both are legitimate. The first costs an extra plane and requires the network to learn the conditioning; the second costs a transformation at encoding time and requires that every downstream component agree on the convention. The two encoders described below make opposite choices, and the reasons are given in their respective sections.

Visibility and Permanence

The encoder determines what the network is able to see. A feature that is not encoded is invisible to the model regardless of how long it is trained: no amount of data will recover information that never entered the input. The choice of planes is therefore not a matter of convenience but a statement about what the designer believes is sufficient to judge a position.

The converse is equally consequential and less obvious. A convention that is encoded — a threshold, a counter, a normalisation — becomes part of the input distribution against which the weights are fitted. The network learns to interpret it, and it cannot afterwards be altered without invalidating what the network has learned about it. Choices made in this chapter are, in that sense, permanent for any model trained under them.

Shared Mechanics

Both encoders allocate and populate their planes as float32¹, and both build the tensor in a single linear scan of the board followed by a small number of constant-time writes for whole-plane flags. The cost is negligible relative to the search or the forward pass that follows.

Features are assembled in a standard array format [32] and adapted to the model’s tensor format [33] at the boundary; where that boundary supports it, the conversion preserves a shared view rather than copying, until the data is moved or modified.

One invariant is common to both and is the one worth checking: per-square exclusivity across the piece planes. For any square, at most one piece bit is set. A violation would present the network with a position that cannot exist, and would do so silently.

¹float32 refers to a 32-bit floating-point format used to represent real numbers. It provides a balance between range and precision and is commonly used in machine learning and numerical applications where memory usage and computational efficiency are important.

3.2. The Chess Encoder

3.2.1. Shape and Perspective

The encoder emits an unbatched feature tensor of shape $18 \times 8 \times 8$ with strictly binary entries for all base channels. It follows the scheme of AlphaZero’s input [10] — piece planes followed by planes for the state of the rules — without its history planes, so the policy/value network receives consistent planes without scattered preprocessing.

Perspective is handled by the first of the two methods described above. The board is kept in an absolute frame: White’s pieces always occupy planes 0–5 and Black’s always occupy planes 6–11, and a dedicated plane declares the side to move. The network is thus given a fixed geometry, and learns to read that plane as the condition under which the rest is to be interpreted.

3.2.2. Plane Layout

The 18 base planes follow a fixed layout, shown in Table 3.1. Indices 0–5 are White piece occupancy $\{P, N, B, R, Q, K\}$, 6–11 are Black $\{p, n, b, r, q, k\}$, each as one-hot bits on the squares they occupy. Plane 12 encodes the side-to-move as a full-board mask: all ones when White is to move, all zeros otherwise. This gives the network a fixed geometry without relying on parity or history. Planes 13–16 encode castling availability — white kingside, white queenside, black kingside, black queenside — as full-board binary flags, ones when available and zeros otherwise; a compact four-tuple of rights is preferred when present, with a graceful fallback to per-side booleans. Plane 17 encodes en passant with a “file-hot” mask: if an EP target exists, every square in that column is set to 1; otherwise the plane is zero.

Table 3.1: Input planes layout (chess)

Index range	Plane(s)	Description
0–5	White piece planes	$\{P, N, B, R, Q, K\}$ one/hot
6–11	Black piece planes	$\{p, n, b, r, q, k\}$ one/hot
12	Side/to/move	all ones when White is to move, all zeros otherwise
13–16	Castling rights	WK, WQ, BK, BQ as binary planes
17	En/passant file	one/hot file indicator if applicable

Figure 3.1 provides a concrete walk-through. Panel (a) shows the board after 1.e4 c5 — White to move, all castling rights intact, en-passant square c6. Panel (b) depicts the encoder’s response: 18 binary feature planes that disentangle this board state into machine-readable layers.

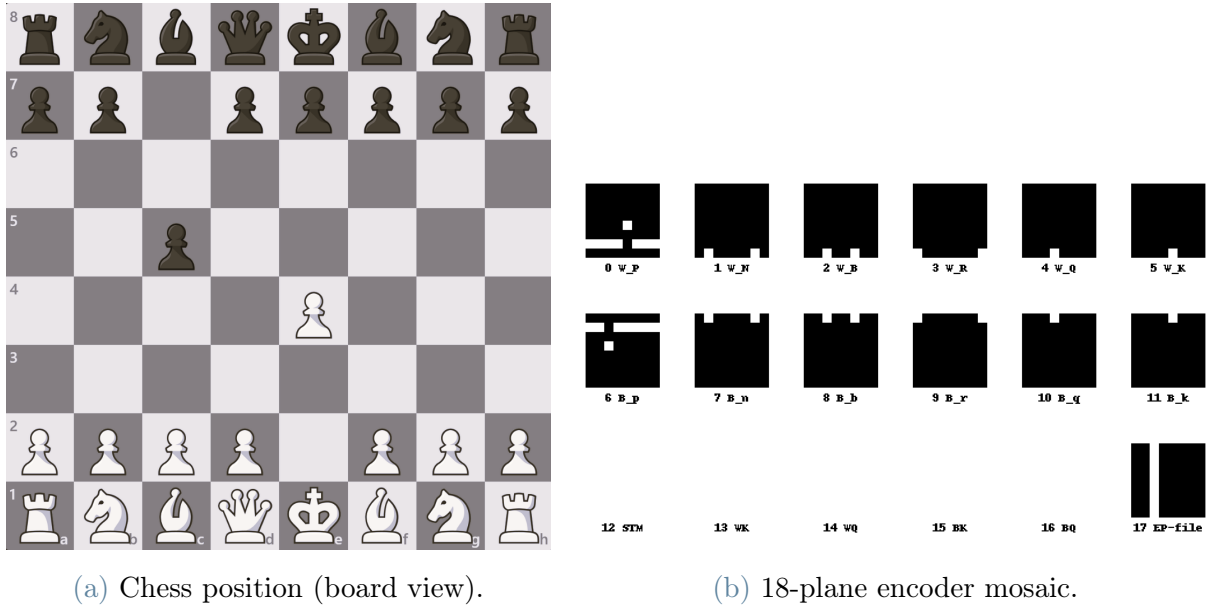


Figure 3.1: The chess encoder applied to the position after 1.e4 c5: (a) the board, (b) the 18 planes it produces.

Piece placement uses a small mapping from absolute piece IDs 1–6 to indices 0–5 for White, and the same mapping with a +6 offset for Black. During a single scan of the 64 squares, each nonzero signed entry — positive for White, negative for Black — sets exactly one bit in its corresponding plane, while empty cells are skipped.

3.2.3. Invariants and Contract

All base channels are binary one-hot indicators of presence, availability or eligibility; there is no scaling, no counts and no distances anywhere in the core design. The per-square exclusivity invariant holds across the 12 piece planes.

The environment contract is explicit: the board is an 8×8 grid of signed integers with $|\cdot| \in \{1, \dots, 6\}$; the side-to-move indicator uses +1 for White and -1 for Black; any EP file, when present, is a valid column index; castling rights are exposed either as a four-tuple or via per-side booleans.

Error handling in the hot path is strict and explicit, to prevent silent failures during self-play or training. The encoder actively validates the board shape, raises on unknown piece identifiers, and performs bounds checking on the EP file index to ensure it falls within the valid 0–7 range. This defensiveness is warranted because the chess environment supports position import, and can therefore be handed a position that did not originate inside the pipeline.

Strict adherence to the 18-plane convention keeps training, inference and self-play aligned with the network’s architecture, preventing off-by-one-channel errors.

3.3. The Italian Draughts Encoder

3.3.1. Shape and Perspective

The draughts encoder produces a $7 \times 8 \times 8$ tensor, and shares every structural property of the chess encoder: unbatched output, float32 planes, a single static snapshot, and a boundary conversion that avoids copying where it can.

It resolves perspective by the second of the two methods. The board is canonicalised before encoding: whenever it is the second player’s turn, the board is rotated a half-turn and every piece is reinterpreted relative to the side about to move rather than to a fixed colour. The same physical position therefore produces two different tensors depending on who is to move, and the network only ever has to learn what a good position looks like for the player about to act, never separately for each colour.

The choice is worth its cost here for a reason specific to this game. Draughts men move in only one direction, so the absolute frame would require the network to learn two mirror-image versions of every pattern involving advancement — precisely the patterns that matter most. Rotating once at encoding time removes that duplication, and the transformation is trivial on a board this small.

3.3.2. Plane Layout

The layout follows the same fixed-list philosophy on a much shorter list, shown in Table 3.2. Planes 0 and 1 hold, respectively, the men and the kings belonging to the side about to move; planes 2 and 3 hold the same two piece types belonging to the opponent. That is 4 planes in total against chess’s 12, both because there are only 2 piece types instead of 6 and because the relative framing collapses the White/Black split into a mover/opponent split.

There is no equivalent of castling or en passant, since draughts has neither, and no side-to-move plane, since canonicalisation makes it redundant; the 6 planes chess devotes to these are replaced by 3 of a very different character.

Plane 4 is a constant mask of the dark squares that are ever playable — the same at every position, included so that later stages of the network never have to special-case which squares matter. The mask remains correct through canonicalisation without any extra

work, because rotating the board by a half-turn does not change whether a square is dark.

Plane 5 is a single broadcast flag, 1 if the side to move has any capturing move available and 0 otherwise. It gives the network a direct signal for the mandatory-capture rule, which has no counterpart in chess and would otherwise have to be inferred indirectly from the piece planes.

Plane 6 is the no-progress count from Chapter 1, normalised to the $[0, 1]$ range and broadcast across the whole plane, so that the network can see how close the game is to being drawn. Chess’s 18 planes carry no equivalent signal for the fifty-move rule: that piece of state is tracked by the environment but never reaches the network as an input feature.

Table 3.2: Input planes layout (Italian draughts)

Index	Plane	Description
0	Own men	one/hot, mover’s perspective
1	Own kings	one/hot, mover’s perspective
2	Opponent’s men	one/hot, mover’s perspective
3	Opponent’s kings	one/hot, mover’s perspective
4	Playable squares	constant dark/square mask
5	Capture available	broadcast flag, 1 if a capture is mandatory this turn
6	No/progress	broadcast value in $[0, 1]$: plies since the last capture or promotion, divided by the draw threshold

Piece placement follows the same principle as chess’s: a single scan of the 64 squares, in which each occupied square sets exactly one bit in the plane matching its piece type and its relation — own or opponent’s — to the side about to move, while empty squares are skipped.

3.3.3. Invariants and Contract

The per-square exclusivity invariant holds across the 4 piece planes: at most one of them is set for any square. The environment contract is comparably simple: a 64-entry board of signed integers with magnitude in $\{1, 2\}$, the same 32-bit floating-point format used throughout, and the same negligible cost — one linear scan of the board plus a handful of constant-time writes for the 3 broadcast planes.

Two differences from the chess encoder are worth stating explicitly.

The first concerns what “binary” means. Chess’s design deliberately keeps every channel a strict zero-or-one indicator, with no scaling, counts or distances anywhere in the core. The draughts encoder breaks that rule for 1 of its 7 planes: the no-progress count is a continuous value in $[0, 1]$ — exactly the kind of scalar, distance-like signal chess’s design

goes out of its way to avoid. The capture flag, by contrast, is a broadcast binary flag of the same kind as chess's castling planes. The justification is that both encode rules the network cannot otherwise perceive, and that the alternative — omitting them — would leave the model unable to recognise a mandatory capture or an approaching draw.

That plane is also the clearest instance of the property described in Section 3.1. The count is divided by the draw threshold before being presented, so the threshold itself is embedded in the input: a network trained under one threshold has learned to interpret the plane against that denominator, and cannot be reused under a different one.

The second difference is in defensiveness rather than design intent. The chess encoder validates its input and raises on malformed piece identifiers or an out-of-range en-passant file. The draughts encoder performs no equivalent validation, relying instead on every position it is given having been produced by the environment of Chapter 1, which has no path by which it could construct an invalid one. The asymmetry reflects the fact that only the chess environment accepts positions from outside the pipeline.

4 | The Policy-Value Network: Residual Architecture

4.1. The Policy–Value Network

The search of Chapter 2 needs two things from a model, and they are different in kind. When it expands a node it needs a prior over the legal moves, to decide which continuations deserve attention before any has been tried. When it reaches a leaf it needs a value, an estimate of the outcome, so that the simulation can stop there instead of playing the game to its end. These replace, respectively, the move ordering and the hand-written evaluation function of a traditional engine.

Both are produced by a single network with two output heads [15]. Using one network rather than two is not merely an economy of parameters: the search is bounded by the number of evaluations it can afford, and a shared trunk means one forward pass per leaf instead of two. It also reflects the assumption that the features useful for judging a position and those useful for proposing a move are largely the same features.

Stem and Trunk

Encoded positions enter a 3×3 convolution with padding 1, followed by normalisation [34] and ReLU. This stem lifts the input planes to the trunk’s working width.

The trunk is a stack of constant-width residual blocks¹ [24]. Each block applies two 3×3 convolutions without biases, each followed by normalisation; ReLU follows the first normalisation, the skip connection is added after the second, and a final ReLU closes the block. No projection is required anywhere, because the channel count never changes.

¹The residual trunk refers to the sequence of residual blocks in a ResNet, where each block consists of two convolutions with a skip connection. These blocks allow the network to learn an identity function, which helps in training deeper models by enabling easier gradient flow.

Constant Spatial Resolution

A conventional vision network reduces spatial resolution as depth increases, on the assumption that precise location matters less than coarse structure. That assumption does not hold here. The board is not a picture of a scene but the state itself: which square a piece occupies is not a detail to be abstracted away but the entire content of the position, and two positions differing by one square are frequently one won and one lost.

The trunk therefore preserves the 8×8 grid at every layer, and the backbone it produces has shape $[B, C, 8, 8]$. The policy head requires this in any case, since it must produce an output attached to each square rather than a single vector for the position as a whole.

Squeeze-and-Excitation

The block admits one optional refinement: after the two convolutions and the skip connection, and before the final ReLU, the channels may be recalibrated by a squeeze-and-excitation step [35] — a global average over the spatial dimensions feeding two small fully connected layers that produce a per-channel gate, which is multiplied back into the feature map.

The two engines default differently, for the same reason. The chess network omits it, to keep the cost of an inner-loop evaluation as low as possible; the draughts network enables it by default, because on a much smaller board and with a much lighter network the additional cost is affordable. In both cases it is a switch rather than a fixed commitment, and the decision is a latency trade-off resolved in opposite directions rather than a disagreement about whether the mechanism helps.

Initialisation, Scaling and Restraint

Weights use He/Kaiming² initialisation [36] suited to ReLU activations: normal for convolutions, uniform with a small nonlinearity parameter for linear layers. Convolutions omit biases where a normalisation layer follows, since the normalisation’s own shift makes them redundant; the final logit layers, which have no normalisation after them, retain theirs. This scheme is used identically by both networks — one of the few places where the two were not designed independently so much as the same recipe applied twice.

Both designs stay lean: no attention, no dilations, no dropout, and, apart from the

²He/Kaiming initialisation is a method for initialising the weights in deep neural networks. It is specifically designed for ReLU activation functions and helps maintain the variance of activations throughout the network, making training more stable and improving convergence.

optional step described above, nothing beyond convolution, normalisation, ReLU and linear layers. Capacity is scaled through two knobs only — the channel width C and the number of blocks n — and no other part of the pipeline needs to change when they do.

The boundary is kept clean in one further respect: the network emits raw logits and never applies legality masking or normalisation to them. Restricting the distribution to the legal moves is the responsibility of the adapter described in Chapter 5. The network therefore never needs to know the rules, and the same trunk would serve a third game unchanged.

What remains specific to each engine is the two heads, and the differences there are substantial rather than cosmetic. The following sections describe each in turn.

4.2. The Chess Network

The chess network ingests batches of encoded positions of shape $[B, 18, 8, 8]$, with a trunk of default width $C = 64$ and $n = 6$ blocks. Figure 4.1 shows it whole, from the input planes to the two heads.

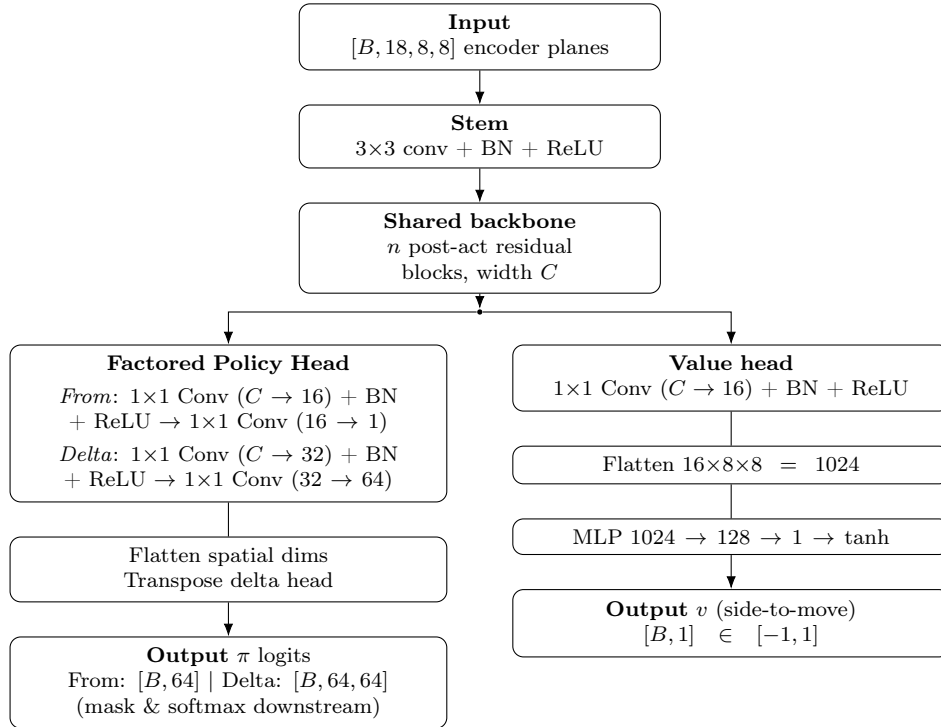


Figure 4.1: The chess policy-value network, with $C = 64$ and $n = 6$ in the configuration used here. Figure 4.2 gives the draughts network, which follows the same stem and trunk design and replaces both heads.

4.2.1. Policy Head

The policy head is parameterised in a fully convolutional factored form, which avoids the large fully connected layer a flat head over the full move space would require. A from-square branch compresses the backbone to 16 channels, then to a single channel, flattening to produce $[B, 64]$ logits. In parallel, a delta branch compresses the backbone to 32 channels, then to 64; after flattening the spatial dimensions and transposing, it yields a $[B, 64, 64]$ tensor mapping from-squares to to-squares.

The factorisation is what makes the head affordable. A chess move is a pair of squares, so a flat head would need 64×64 outputs produced directly; the factored form produces the marginal over origins and the conditional over destinations separately, and the joint logits are recombined downstream. Promotions are handled by the auto-queen convention of Chapter 1, so no further factor is needed. Table 4.1 lists the operations of all three outputs, the value head included.

Table 4.1: Chess heads: outputs and brief operations.

Head	Output	Operations & Notes
Policy (From)	$[B, 64]$	1×1 Conv ($C \rightarrow 16$) $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow 1 \times 1$ Conv ($16 \rightarrow 1$) $\rightarrow$ Flatten.
Policy (Delta)	$[B, 64, 64]$	1×1 Conv ($C \rightarrow 32$) $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow 1 \times 1$ Conv ($32 \rightarrow 64$) $\rightarrow$ Flatten & Transpose.
Value	$[B, 1] \in [-1, 1]$	1×1 Conv ($C \rightarrow 16$) $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ MLP ($1024 \rightarrow 128 \rightarrow 1$) $\rightarrow$ tanh.

4.2.2. Value Head

The value head mirrors the early structure of the policy head: a 1×1 convolution to 16 channels, normalisation and ReLU, then a flatten to $16 \times 8 \times 8 = 1024$. From there a small MLP³ ($1024 \rightarrow 128 \rightarrow 1$) with ReLU between layers and a final tanh⁴ produces $[B, 1] \in [-1, 1]$, the evaluation from the perspective of the side to move: positive favours that side, negative disfavors it.

³MLP (Multilayer Perceptron) is a type of fully connected neural network that consists of multiple layers of neurons, where each neuron is connected to every neuron in the next layer. It is typically used for tasks like regression or classification, where each layer learns progressively more complex features.

⁴tanh (hyperbolic tangent) is an activation function that maps input values to a range between -1 and 1 . It is used in neural networks to introduce nonlinearity and is particularly useful when the model requires outputs with a centred range.

4.2.3. Numerics and Training Interface

The model runs in FP32⁵ and supports mixed precision [37]; normalisation layers behave appropriately in training and in inference mode.

Training pairs cross-entropy (or KL)⁶ on the policy against the root-visit distribution with mean squared error (MSE or Huber⁷ [38]) on the value against outcomes in $[-1, 1]$.

The factorisation has a consequence for the loss that is worth stating: the joint cross-entropy equals the cross-entropy on the from-square marginal plus the expected cross-entropy on the to-square conditional, so the two branches can be trained against targets obtained by marginalising and conditioning the same root-visit distribution, with no approximation. The full derivation is given in Chapter 6.

The resulting split — input $[B, 18, 8, 8]$, backbone $[B, C, 8, 8]$, heads $[B, 64]$, $[B, 64, 64]$ and $[B, 1]$ — keeps shape errors rare and makes width and depth scaling straightforward.

4.3. The Italian Draughts Network

The draughts network ingests batches of shape $[B, 7, 8, 8]$, with a trunk of default width $C = 96$ and $n = 6$ blocks, and with the optional channel recalibration enabled. Figure 4.2 draws it on the same skeleton as the chess network, so that the two can be read against each other.

4.3.1. Policy Head

The policy head stays fully convolutional but never splits into two branches. A single stage reduces the backbone to 8 channels — one for each of the 8 direction-and-mode combinations a move can take from a given square — and the resulting $[B, 8, 8, 8]$ tensor is restricted to the 32 playable squares and reshaped into a flat $[B, 256]$ vector of logits, one per slot.

There is no second branch to combine because there is nothing analogous to factor. In chess a move is a genuine pair of choices, origin and destination, and the destination

⁵FP32 (32-bit floating-point) refers to a data type used to represent real numbers with 32 bits of precision. It is widely used in machine learning models for storing weights and activations due to its balance between precision and memory requirements.

⁶Cross-entropy and KL (Kullback–Leibler) divergence are loss functions used in training neural networks. Cross-entropy measures the difference between the predicted probability distribution and the true distribution, while KL divergence quantifies the difference between two probability distributions.

⁷Huber loss is a variant of MSE that combines the benefits of MSE and absolute error, being less sensitive to outliers by applying a quadratic penalty for small errors and a linear penalty for large ones.

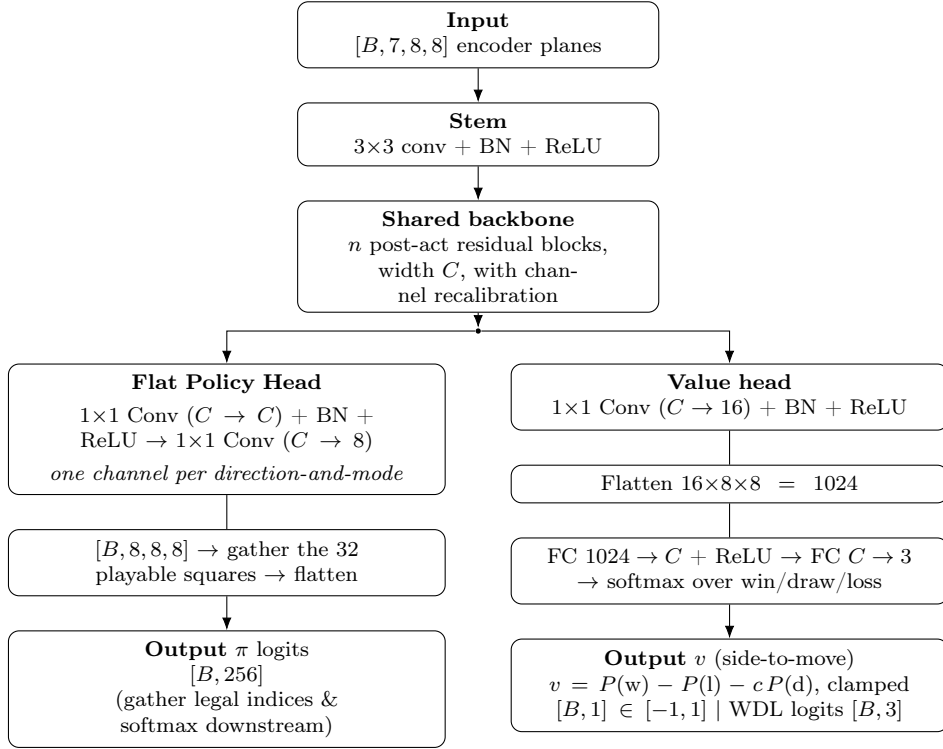


Figure 4.2: The draughts policy-value network, with $C = 96$ and $n = 6$, drawn on the same skeleton as Figure 4.1. The optional channel recalibration, omitted in chess, is enabled here, and the contempt term c is zero in every run reported in this thesis.

remains open once the origin is fixed. In draughts it does not: once the origin, the direction and the mode are fixed, the destination follows from the rules. Factoring the head would therefore introduce a conditional distribution over a quantity that is already determined. Table 4.2 lists the operations of both heads.

Table 4.2: Draughts heads: outputs and brief operations.

Head	Output	Operations & Notes
Policy	$[B, 256]$	1×1 Conv ($C \rightarrow C$) $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow 1 \times 1$ Conv ($C \rightarrow 8$) $\rightarrow$ gather the 32 playable squares $\rightarrow$ Flatten.
Value	$[B, 1] \in [-1, 1]$	1×1 Conv ($C \rightarrow 16$) $\rightarrow$ Norm $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ FC ($1024 \rightarrow C$) $\rightarrow$ ReLU $\rightarrow$ FC ($C \rightarrow 3$) $\rightarrow$ softmax $\rightarrow$ combine $\rightarrow$ clamp.

4.3.2. Value Head

The value head begins identically to the chess one: a 1×1 convolution to 16 channels, normalisation, ReLU, and a flatten to 1024 — the same numbers, not merely the same

shape. From there the two designs part ways. Instead of an MLP collapsing to a single tanh-bounded scalar, the draughts head passes through one fully connected layer down to the trunk’s own channel width, then a second down to three logits representing win, draw and loss. A softmax turns these into probabilities, and the scalar value used everywhere else in the pipeline is recovered from them as

$$v = P(\text{win}) - P(\text{loss}) - c \cdot P(\text{draw}),$$

where c is a contempt⁸ term. The result is clamped to $[-1, 1]$ as a safeguard: unlike a tanh output it is not automatically bounded once the contempt term is nonzero.

The difference from the chess head is one of modelling rather than of output format, and the reason is the status of the draw. A scalar regression treats a draw as the midpoint between winning and losing, which is a statement about utility but not about the game: a position whose true outcome is a certain draw and a position that is equally likely to be won or lost receive the same target, though they are entirely different positions. A three-way classification distinguishes them, because it predicts a distribution rather than a point. In a game where the drawn outcome is common, this distinction is worth a third output, and it is also why the value loss differs between the two engines, as Chapter 6 describes.

4.3.3. Numerics and Training Interface

The draughts network runs in FP32 only, with no mixed-precision path, on the CPU by default and on a GPU when one is available: the network is small enough that reduced precision was never worth the additional complexity.

The policy is trained with a flat cross-entropy directly against the masked root-visit distribution, with no marginal-and-conditional split to perform since the policy is never factored. The value head is trained with cross-entropy against the win, draw or loss class of the recorded outcome, rather than with a regression loss against a scalar. Both are described in full in Chapter 6.

The same clean split from encoder to network to agent holds here: input $[B, 7, 8, 8]$, backbone $[B, C, 8, 8]$, heads $[B, 256]$ and $[B, 1]$, with the three win/draw/loss logits also exposed so that the training loop can compute the value loss against them directly.

⁸Contempt is a deliberate penalty applied to the drawn outcome, used in engine play to make a program prefer a continued game over an early draw against an opponent it expects to outplay. It is available here but set to zero by default, so that the value is simply the difference between the win and loss probabilities.

5 | The Inference Bridge: Optimizing the Neural Adapter

5.1. The Inference Bridge

The network of Chapter 4 emits raw logits over a fixed index space and a scalar, and knows nothing about the rules: it cannot tell which of its outputs correspond to moves that are legal in the position it was just shown. The environment of Chapter 1 knows exactly that, but nothing about tensors or index spaces. The search of Chapter 2 must remain agnostic to both, or it would have to know the rules and the tensor layout of each game.

The gap between them is what this component fills. It is not an optimisation or a convenience layer: no other component is in a position to do the work, and the boundary is forced by the separation of concerns that the rest of the architecture depends on.

Three Obligations

The adapter is required to do three things, and deliberately nothing else.

It must map between representations: take a live position, produce the tensor the network expects, and translate the network's output indices back into moves the environment recognises.

It must restrict and normalise. The network assigns weight to every index it can express, including those that correspond to nothing legal in the current position. The adapter removes those, renormalises what remains, and returns a proper probability distribution over the legal moves alone. Doing this at the boundary is what allows the network to be trained without any notion of legality and the search to expand a node without validating anything.

It must fix the perspective. The search accumulates values across alternating turns and requires that every value it receives mean the same thing: the evaluation from the point

of view of the side to move. How that is achieved differs between the two engines, but the guarantee given to the search does not.

If the environment reports no legal moves, the adapter returns an empty mapping alongside the position’s terminal value, and the search handles the terminal state as described in Chapter 2.

Deliberate Omissions

The adapter does not evaluate positions, order moves, or decide what is worth caching. It applies no heuristic of its own and introduces no preference that the network did not express. This restraint is what makes it possible to state, in Chapter 8, that a measured change in playing strength originates in the model or in the search rather than somewhere in between.

It is also the only component in which the choice of device, the numerical precision and any ahead-of-time preparation of the computation graph appear. Everything above it runs unchanged on any hardware. Figure 5.1 places the adapter on the path from the environment to the search, with the three obligations attached to it.

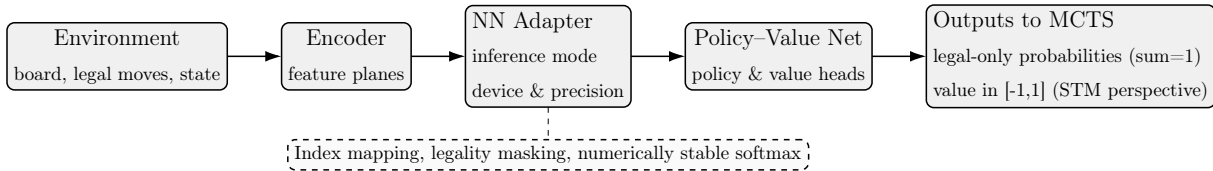


Figure 5.1: End-to-end inference path, shared by both engines. The plane count and the shape of the heads differ between them; the guarantees given to the search do not.

5.2. The Chess Adapter

The chess adapter assembles the 18-plane encoding of Chapter 3 directly from the environment before every inference, and hands the network a $[18, 8, 8]$ tensor.

5.2.1. Move Indexing and Logit Combination

Moves are indexed using a factored policy mapping. As shown in Figure 5.2, a move (f_r, f_c, t_r, t_c) maps to a unique pair of indices by computing the from-square $f_r \times 8 + f_c$ and the to-square $t_r \times 8 + t_c$. This mapping is canonicalised from the perspective of the player to move: if it is Black’s turn, the spatial coordinates are rotated by 180 degrees — $7 - f_r, 7 - f_c$ — before the indices are computed, so that the network always sees spatial

patterns from a consistent point of view.

The environment provides two mapping functions for this purpose: one returns the masks of the legal actions in the factored index space, the other converts a move into its pair of indices.

Example: $(f_r, f_c, t_r, t_c) = (1, 2, 3, 3) \Rightarrow i = 10, d = 27$

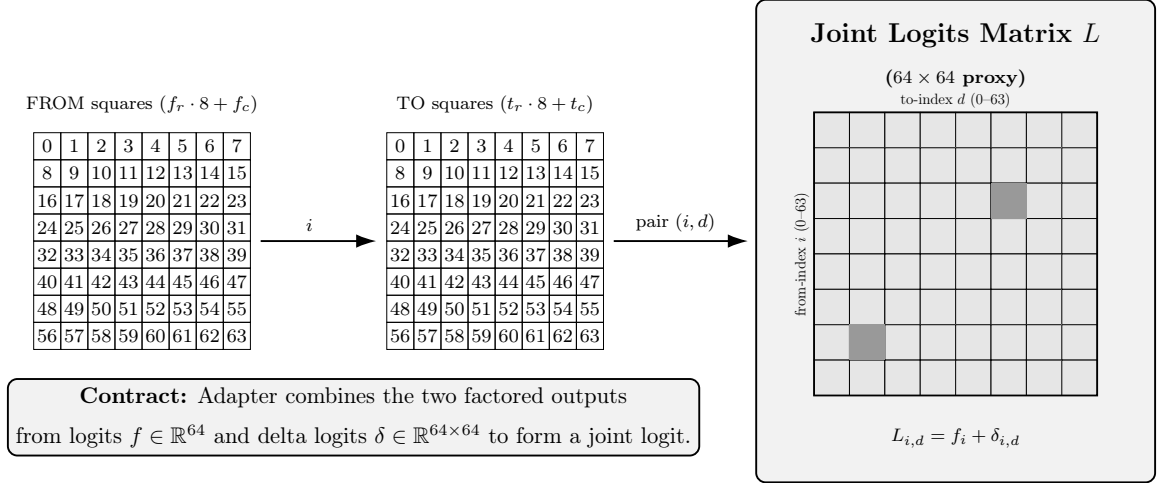


Figure 5.2: Move indexing and factored logit combination map.

5.2.2. Policy and Value Paths

On the policy path, the adapter receives the two factored tensors, the from-square logits f of shape $[64]$ and the delta logits δ of shape $[64, 64]$, and broadcasts and sums them to build the full grid of combined logits, $L_{i,d} = f_i + \delta_{i,d}$.

The environment’s factored masks are then applied to the joint grid, setting every illegal combination to $-\infty$ so that it contributes no probability mass. It is worth noting why the mask has to be applied here rather than to the two factors separately. A from-square may be perfectly legal — the piece standing on it has some legal move — while a particular destination from that square is not. Masking the factors independently would admit exactly those combinations, since neither factor is individually illegal. Only the joint mask distinguishes them.

Finally the adapter performs a numerically stable softmax over the surviving entries [39], subtracting the maximum logit before exponentiation, and returns a normalised distribution ready for the search.

On the value path, the network’s scalar is already bounded. Because the chess encoder keeps the board in an absolute frame, that scalar is expressed from White’s point of view;

the adapter returns it unchanged for White and with opposite sign for Black, which is what makes the perspective guarantee of Section 5.1 hold.

5.2.3. Operational Considerations

The environment contract required to support this is correspondingly structured: an 8×8 board of signed integers, exposed castling and en-passant state, and the two mapping functions described above.

The network is placed on the requested compute device, with GPU execution enforced in FP16 where supported, and kept in inference mode so that batch-normalisation statistics are frozen.

For high-throughput self-play the adapter does not issue one forward pass per position. It exposes a hook that extracts the feature tensor without evaluating it, so that the caller can enqueue the request to the central inference server described in Section 2.2; the batching, cross-process deduplication and caching performed there belong to the search’s throughput strategy rather than to this boundary.

5.3. The Italian Draughts Adapter

5.3.1. Move Indexing and the Shared Canonicalisation

Both halves of the previous section collapse into a single operation here. The 7-plane encoding of Chapter 3 and the compact move index of Chapter 4 are built from one and the same canonicalisation: the half-turn rotation applied whenever it is the second player’s turn.

Chess computes its input planes in an absolute, colour-fixed frame and reserves the 180-degree rotation for the move-index computation alone, so there are two symmetries to keep consistent with each other. Draughts applies the rotation once and reuses it for both the board and the moves, so there is only one.

5.3.2. Policy and Value Paths

The adapter reaches the same result on the policy path — a normalised distribution over the legal moves alone — by a shorter route. Rather than masking a full grid down to $-\infty$ and softmaxing over the whole space, it gathers only the logits at the legal indices directly out of the flat policy output and applies the softmax to that much smaller set.

The object produced is the same, and the illegal entries are never touched at all.

One complication has no equivalent in chess. Because several distinct legal moves can occasionally share the same compact index — most often when more than one capturing chain begins with an identical first step — the network cannot tell them apart, and its softmax produces a single probability for what is, from its point of view, one action. The adapter divides that probability evenly among every move mapping to the shared index, so the priors still sum to 1 and no legal move is silently assigned zero weight because it collided with another. In chess the situation does not arise, because with promotion fixed to the queen a pair of squares identifies at most one legal move, so no equivalent step exists there. Figure 5.3 shows the index layout and the case in question.

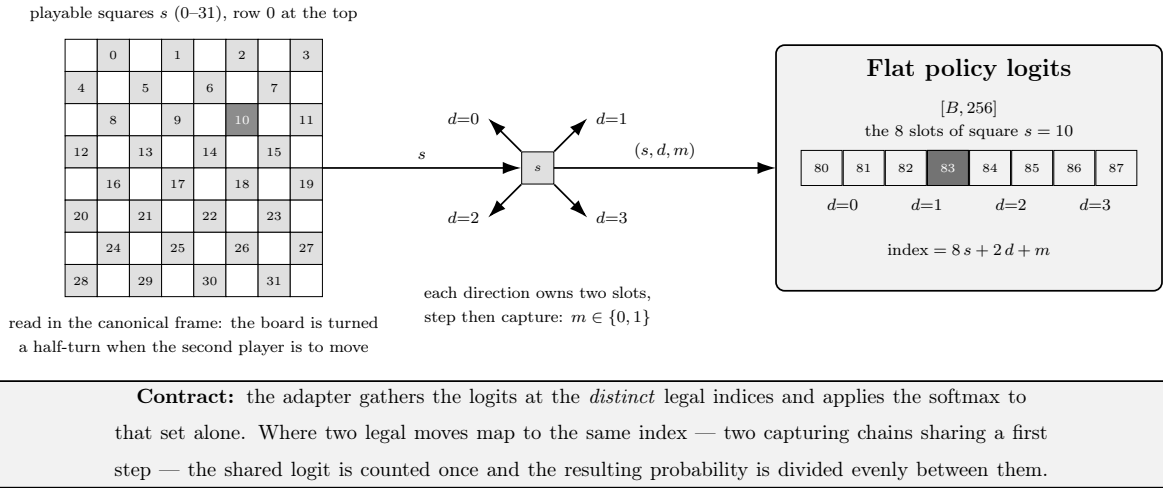


Figure 5.3: The compact move index in Italian draughts; the shaded slot is the up-right capture from square 10, at $8 \cdot 10 + 2 \cdot 1 + 1 = 83$. Where the chess adapter builds a 64×64 grid and masks the illegal entries to $-\infty$ (Figure 5.2), here the policy is a flat vector of 256 slots and the adapter gathers only the legal ones.

On the value path there is nothing to correct. The scalar the network produces is already relative to the side about to move, by construction of the canonical encoding, so the adapter passes it through unchanged.

5.3.3. Operational Considerations

The contract is much lighter, since there is no dual head and no castling or en-passant state to expose: a board of signed integers with magnitude in $\{1, 2\}$, and a list of legal moves, whose compact indices the adapter computes in the canonical frame, are enough.

There is no precision branch: the network runs in FP32 on whichever device it is given,

the CPU by default or a GPU when one is available. The optimisation that takes the place of reduced precision is a one-time preparation step: on construction the adapter traces the network’s computation graph [33], freezes it, and applies inference-time fusions such as folding batch-normalisation statistics into the preceding convolutions [34] — all before any position is ever encoded. If any part of the network turns out not to be traceable this way, the adapter falls back silently to the ordinary, unoptimised network. A fragile trace can therefore change how fast the engine computes, never what it computes.

Leaf batching across searches and processes is not handled here either; it is described in Section 2.3.

6 | Network Optimization: Training and Regularization

6.1. Learning from Self-Play

Turning the games recorded in self-play into a better network is the work of a single step, and one fact about that step organises everything else in this chapter, easy to miss because it differs from how a network is usually trained. A cycle does not train a model from scratch. It starts from the weights of the current champion and performs a short, directional fine-tune on recent data — 2 passes over a replay window in the draughts pipeline, 5 in the chess one — producing a candidate that is then judged against the champion it came from. The consequences run through the whole chapter: the learning rate must not disturb the starting weights in the first steps, which is what a warmup is for [40]; the number of epochs is small, because the point is to move rather than to converge; and, as Section 6.3 shows, a technique that averages over a trajectory can fail here for reasons that would not arise in ordinary training.

Samples and Targets

Each training sample records three things about one position visited during self-play: the encoded position, a policy target, and the outcome of the game it belonged to.

The policy target is the distribution of visit counts the search produced at that position's root, not the move that was actually played. This is the central idea of the method [15] and it is worth stating plainly. The search, guided by the network, is stronger than the network alone: it examines continuations the network merely guessed at and returns a judgement informed by what it found. Training the network towards the search's visit distribution therefore distils the search's improvement back into the network, which then guides a better search in the next cycle. Training towards the played move instead would discard everything the search discovered about the alternatives.

The outcome supplies the value target. It is the result of the whole game, taken from the perspective of the side to move at that position, so a single game contributes one label shared by all its positions.

The Objective

Both engines minimise a sum of two terms: a policy term that compares the network's move distribution against the search's, and a value term that compares its evaluation against the recorded outcome. The forms of both terms differ between the engines, and Sections 6.2 and 6.3 give each.

One element is common. Illegal actions are excluded at training time as well as at inference: their logits are set to a large negative constant before the log-softmax, so that they contribute nothing to the sum or to the normalisation. This is the same masking idea used at the inference boundary of Chapter 5, applied here so that the network is never penalised for the probability it would have assigned to a move that could not be played.

What remains specific to each engine is where the data lives, the form of the policy target, the loss used on the value, and the optimiser together with its schedule.

6.2. The Chess Training Loop

Figure 6.1 sets out the sequence a training call follows, from the stored shards to the saved checkpoint.

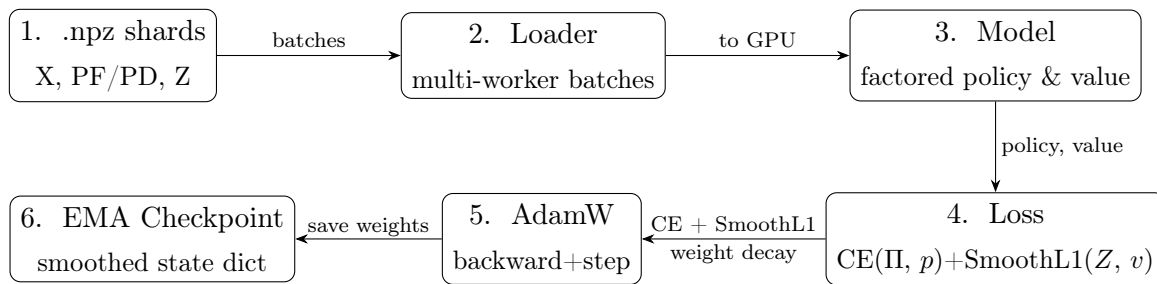


Figure 6.1: End-to-end training flow for the chess engine. Figure 6.2 gives the draughts equivalent.

6.2.1. Data and Loading

Training data lives in one or more .npz shards¹ [32], each containing arrays X , PF , PD and Z . To prevent host memory exhaustion during concatenation, the dataset retains the large factored policy arrays in float16 in RAM and casts them to float32 on the fly immediately before a sample is transferred to the GPU, halving the dataset’s memory footprint.

Features X use the model’s [18, 8, 8] encoding. Policy targets are factored: PF is a normalised from-square marginal over 64 squares, and PD provides per-from delta conditionals over 64 deltas. Values Z are game outcomes from the side-to-move’s perspective, with $Z \in [-1, 1]$.

A PyTorch² [33] data loader uses multiple persistent workers and pinned host memory so that the GPU is never starved for data. The model moves itself and all subsequent mini-batches to the accelerator if one is detected, falling back to CPU otherwise.

6.2.2. Objective

Moves are indexed by (i, j) , where $i \in \{1, \dots, 64\}$ is the from-square and $j \in \{1, \dots, 64\}$ is a delta conditioned on i . The policy distribution factorises by the chain rule,

$$\pi_{\theta}(i, j) = p_{\text{from}}(i) p_{\text{delta}}(j | i). \quad (6.1)$$

Given a flat target $\Pi(i, j)$, define the marginal–conditional pair

$$\begin{aligned} PF(i) &= \sum_j \Pi(i, j), \\ PD(j | i) &= \begin{cases} \Pi(i, j)/PF(i), & PF(i) > 0, \\ \text{any prob. row}, & PF(i) = 0, \end{cases} \end{aligned} \quad (6.2)$$

¹.npz shards are compressed binary files used by NumPy to store large arrays efficiently. In the context of training, they contain policy targets, feature planes, and game outcomes, and are loaded and concatenated during training to form mini-batches.

²PyTorch is an open-source deep learning framework developed by Facebook’s AI Research lab. It provides tools for building and training neural networks.

so that $\Pi(i, j) = PF(i) PD(j | i)$, with PF summing to 1 and each row $PD(\cdot | i)$ summing to 1. The flat 4096-way cross-entropy then rearranges exactly into two terms:

$$\begin{aligned} - \sum_{i,j} \Pi(i, j) \log \pi_{\theta}(i, j) &= - \sum_i PF(i) \log p_{\text{from}}(i) \\ &\quad - \sum_i PF(i) \sum_j PD(j | i) \log p_{\text{delta}}(j | i). \end{aligned} \quad (6.3)$$

In words, the factored policy loss is a cross-entropy on the from-square marginal plus a PF -weighted sum of cross-entropies on the per-from conditional deltas. The rearrangement is exact, so factoring the head costs nothing in the objective.

For value regression, with targets $Z \in [-1, 1]$ and prediction $v_{\theta}(X)$, the SmoothL1 (Huber) penalty³ [38] is used

$$\rho_{\delta}(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq \delta, \\ \delta(|e| - \frac{1}{2}\delta), & |e| > \delta, \end{cases} \quad e = v_{\theta}(X) - Z, \quad (6.4)$$

and the total objective scales the value loss by a constant factor ($\lambda_v = 2.0$),

$$L = L_{\text{policy}} + \lambda_v \mathbb{E}[\rho_{\delta}(v_{\theta}(X) - Z)]. \quad (6.5)$$

Optionally, targets can be temperature-adjusted — sharpened or flattened — before computing the cross-entropy [41].

6.2.3. Optimisation and Checkpointing

Parameters are optimised with AdamW⁴ [42, 43], so that weight decay is applied directly to the parameters rather than through the gradient. A cosine learning-rate schedule with a brief warmup helps the first epochs settle [44], and gradients are clipped to a maximum norm of 1.0.

The pipeline simultaneously maintains an exponential moving average (EMA) of the parameters, with decay 0.999 [45]. Loss is accumulated as a data-weighted mean, and the

³SmoothL1 (Huber) loss behaves like L_2 for small errors and like L_1 for large ones, providing stable gradients with improved robustness to outliers compared to pure MSE.

⁴AdamW (Adaptive Moment Estimation with decoupled weight decay) is an optimiser that keeps per-parameter adaptive learning rates via first- and second-moment gradient estimates, while applying weight decay as a separate L_2 regulariser rather than through the gradient update; this preserves the intended regularisation and often improves generalisation and training stability when weight decay is used.

smoothed EMA weights are atomically saved as a state-dictionary checkpoint for downstream use, so that an interrupted process cannot leave a corrupt file.

6.2.4. Contracts and Efficiency

A few contracts are assumed: the factored policy output must align with the loss semantics; PF and each row $PD(\cdot | i)$ are normalised; $Z \in [-1, 1]$ and broadcasts to the value head's shape; and all shards share consistent shapes, so that the float16-to-float32 concatenation scheme functions correctly.

Efficiency rests on masking I/O latency: persistent workers, pinned memory, asynchronous host-to-device transfers, and per-batch loss normalisation that keeps gradients comparable across batch sizes. Because responsibilities are cleanly separated — the dataset serves tensors, the training routine handles one epoch, the driver wires them together — extensions such as temperature smoothing, different schedule milestones or mixed precision in the forward pass require few changes.

6.3. The Italian Draughts Training Loop

Figure 6.2 sets out the same sequence for draughts, whose last step falls outside the training routine.

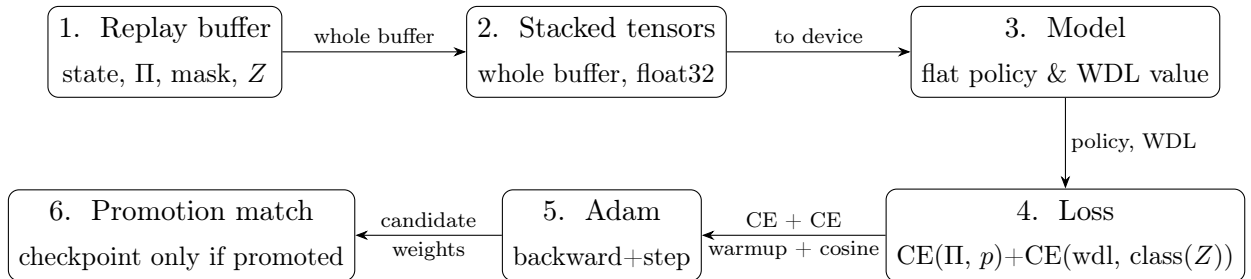


Figure 6.2: End-to-end training flow for the draughts engine, on the same sequence as Figure 6.1. The last step lies outside the training routine: the trained weights reach disk only if the candidate wins its promotion match.

6.3.1. Data and Loading

There is no on-disk shard format. The conductor keeps a bounded in-memory buffer of recent samples — position, visit-count target, legality mask, and outcome — and each training call converts whatever is currently in that buffer directly into stacked tensors. There is no float16 storage trick, since everything already fits comfortably in RAM at

full float32 precision, and no persistent workers or pinned memory: the buffer is already in memory, and each batch is moved to the training device as it is used—the CPU by default, the GPU when one is available.

The policy target is not factored. It is the search’s visit distribution over the compact action space of Chapter 4 — already a proper probability distribution over the legal moves — carried alongside its own legality mask rather than split into a marginal and a set of conditionals.

6.3.2. Objective

Writing $\Pi(a)$ for the target probability of action a under the visit distribution and $p_\theta(a)$ for the network’s predicted probability, the policy loss is an ordinary cross-entropy over the legal actions only:

$$L_{\text{policy}} = - \sum_a \Pi(a) \log p_\theta(a). \quad (6.6)$$

For the value head, instead of a regression against a continuous target, the outcome Z is first mapped to one of three classes — win, draw or loss, according to its sign — and the value loss is the cross-entropy of the network’s three win/draw/loss logits against that class:

$$L_{\text{value}} = \text{CE}(\text{wdl logits}, \text{class}(Z)). \quad (6.7)$$

The two terms are added with no weighting between them:

$$L = L_{\text{policy}} + L_{\text{value}}. \quad (6.8)$$

The absence of a weighting factor, where chess scales its value term by 2, is a consequence of the change of loss rather than a separate decision: two cross-entropies over comparable numbers of classes produce gradients of similar magnitude, whereas a cross-entropy and a Huber penalty do not.

6.3.3. Optimisation and Checkpointing

The optimiser is Adam rather than AdamW. The distinction is not cosmetic: Adam applies weight decay through the gradient, so its effective strength is rescaled by the adaptive denominator and is no longer uniform across parameters, whereas AdamW applies it directly to the weights. With a nonzero decay the two do not coincide. AdamW is implemented and selectable; Adam remains the default so as not to change, silently, the

behaviour of runs already completed under it.

The learning rate is not fixed. Two schedules operate at once. Within each training call, a linear warmup over the first 5% of steps is followed by a cosine decay to one tenth of the base rate; the warmup exists specifically to avoid disturbing the champion’s weights in the first steps of what is, as Section 6.1 noted, a short fine-tune rather than a fresh training run. Across the run as a whole, the base rate itself follows a second cosine, from its initial value to one tenth of it over the configured number of cycles. Because that second schedule is a function of the cycle index alone, it survives an interruption and resumption of the run unchanged.

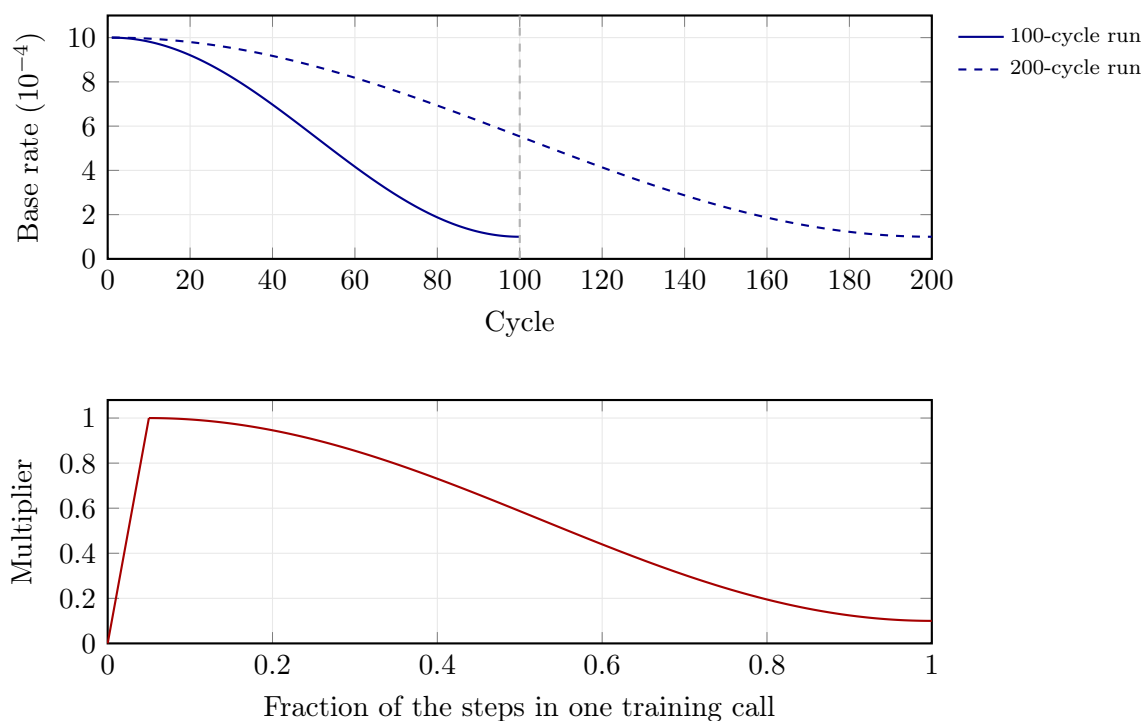


Figure 6.3: The two learning-rate schedules, which apply at the same time: the multiplier of the lower panel scales the base rate of the upper one. The two runs are held apart from their first cycle — at cycle 50 the longer trains at 1.6 times the shorter one’s rate, and at cycle 100, where the shorter one ends and the vertical line falls, at 5.5 times.

This second schedule has a consequence beyond training stability, and it is the reason the cycle count is not merely a stopping condition: a run configured for a different number of cycles follows a different learning-rate trajectory from its first cycle onward. Section 8.4 returns to what this means for comparing runs of different lengths. Figure 6.3 shows both schedules, and how far apart two runs of different lengths are held at the same cycle.

There is no gradient clipping.

The Exponential Moving Average

An EMA of the parameters is implemented and is enabled by default in the training routine itself, averaging over the last quarter of the steps in a call. The conductor nevertheless disables it, and the reason is a measured one. A controlled comparison over 6 cycles — identical configuration, identical seeds, a small network and 12 promotion games per cycle — gave the result shown in Table 6.1.

	Promotions	Mean arena score	Loss
EMA disabled	5 of 6	0.63	2.123 $\rightarrow$ 1.757
EMA enabled	1 of 6	0.48	stuck near 2.0

Table 6.1: Controlled comparison of parameter averaging in the draughts pipeline.

The explanation follows from the framing of Section 6.1. Each cycle is a short fine-tune that moves directionally away from the champion’s weights. Averaging a directional trajectory means lagging behind it, so the candidate that emerges is closer to the champion than the trained weights are — and therefore no better than the champion it was meant to improve on. The promotion gate then rejects it. Parameter averaging helps when a trajectory oscillates around a minimum [46]; it does not help when the trajectory is still advancing.

The mechanism is left available so that it can be reconsidered if the training regime ever changes — longer fine-tunes over more data per cycle, which is closer to the chess regime in which it is used.

Checkpointing happens one level up, outside this routine: the plain, non-averaged trained weights are written to disk only when the resulting network wins enough games in the promotion match to replace the champion, as described in Chapter 7.

6.3.4. Contracts and Efficiency

The contract is correspondingly smaller. The policy target and its legality mask must have the same length and refer to the same action indexing as the network’s policy output, and the outcome must be a scalar in $[-1, 1]$ from which a win/draw/loss class can be read off by its sign. There is no shard-shape consistency to maintain, because there are no shards. The indexing convention is checked once per training call, on the first batch, which costs nothing and catches a mismatch between whoever built the mask and whoever built the target.

Efficiency here is less about hiding I/O latency — there is very little of it, with everything already in memory — than about keeping the loop cheap enough to run every cycle without becoming the bottleneck. The routine returns its metrics rather than printing them, leaving logging to the conductor of Chapter 7. The simplicity is itself a form of extensibility: because the policy target is never factored, adding a curriculum-style adjustment to the loss or swapping the optimiser touches only a few lines, with no marginal-and-conditional bookkeeping to keep consistent elsewhere.

7 | The Self-Play Pipeline: Autonomous Data Generation

7.1. The Self-Play Loop

The components of the preceding chapters can generate a game, learn from a batch of positions, and judge one network against another. This chapter describes the process that puts them in a cycle and runs it unattended for days.

A Self-Reinforcing Cycle

The arrangement improves in three places at once, and each depends on the other two. The network supplies priors and evaluations that make the search stronger than it would be alone. The search, being stronger than the network that guides it, produces move distributions better than the network's own, and those become the training targets. Training on them moves the network towards the search's judgement, which makes the next search better still.

None of the three improves in isolation. A network trained on its own outputs would learn nothing; a search with a fixed evaluator would never improve; and data generated once would be exhausted. The loop is what makes each the source of the others' improvement.

Phases

Each cycle executes a fixed sequence. In the self-play phase the current champion plays a number of games against itself, with exploration noise so that the games differ from one another, and every position visited is recorded with the search's visit distribution and the eventual outcome. In the training phase a candidate network is fine-tuned on a window of recent positions, as described in Chapter 6. In the promotion phase the candidate plays a match against the champion and replaces it only if it wins by a sufficient margin. A fourth phase, measuring the champion against opponents outside the loop, runs on

a slower schedule in both pipelines, either as a separate phase or folded into the cycle. Figure 7.1 shows the sequence, with the fourth phase drawn as the separate step it is in chess.

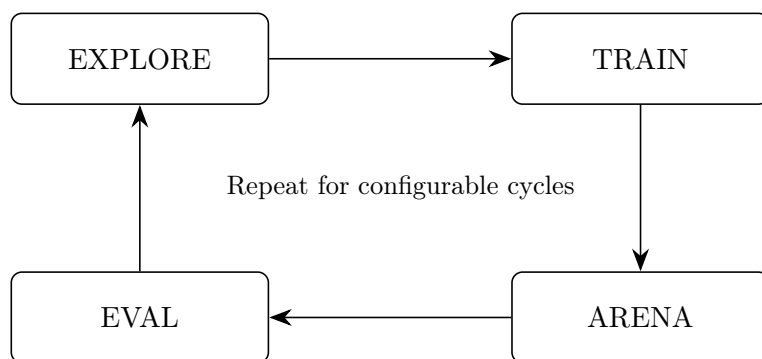


Figure 7.1: The four-phase loop of the chess pipeline. The draughts pipeline runs the same cycle without the separate EVAL phase.

The Promotion Gate

The third phase deserves separate mention, because it is what distinguishes the loop from a random walk.

Nothing in self-play or training guarantees that a candidate is stronger than the model it came from. Training reduces a loss on a finite sample; the relation between that loss and playing strength is indirect, and Chapter 8 examines how indirect. Without a gate, the sequence of champions would be a sequence of changes, not of improvements, and a run would wander rather than climb.

The gate is what makes the sequence monotone — or rather, what is intended to. Its guarantee is only as strong as the evidence it demands, and how much evidence each pipeline demands is one of the principal decisions described below.

Divergences and Resumability

Four questions are answered differently by the two engines: where self-play data is kept and in what form; how exploration is diversified so that the games are not all the same; on what evidence a promotion is granted; and what survives a cycle for later inspection. The sections that follow take each pipeline in turn.

Both must also be resumable. A run of many hours cannot be required to complete in one uninterrupted execution, so any schedule that varies across the run is defined as a function

of the cycle index rather than of elapsed steps, and survives an interruption unchanged.

7.2. The Chess Pipeline

7.2.1. Orchestration and Schedules

The conductor coordinates data generation, training, promotion and logging. Play can be driven by either a lightweight heuristic backend or the neural network, chosen independently for the exploration and evaluation phases; the arena then pits the current evaluation network against the current exploration backend with alternating colours for fairness.

Each phase writes its own artefacts to phase-specific directories, and a manifest records seeds, devices, configurations and thresholds so that a run can be reproduced. Every cycle emits per-game and combined PGNs, memory-optimised float16 dense datasets as .npz [32], CSV summaries for the arena, a line-oriented JSON¹ log of all games, and that manifest.

Schedules and annealing are explicit and traceable. As shown in Figure 7.2, root exploration noise — both the Dirichlet concentration and the mixing weight — shrinks by cycle, the temperature for policy sampling cools across opening plies, and only a controlled fraction of exploration games use MCTS rather than policy-only play. Key schedules are recorded in the per-cycle manifest.

7.2.2. Parallelism and Portability

Self-play is parallelised through an OS-aware process pool that stays stable on Linux, macOS and Windows. Worker counts are capped relative to available cores, with dynamic adjustment depending on whether the GPU or the CPU is handling inference, which protects against out-of-memory errors. Chunking is tuned so that progress advances per game even under concurrency, and if a platform rejects the preferred multiprocessing start method, execution falls back to a sequential path.

To prevent oversubscription, BLAS² thread pools are limited through environment variables, and hot paths avoid deep copies by relying on the stack-style state updates of

¹JSON (JavaScript Object Notation) is a lightweight, text-based data format used for storing and exchanging structured data. It is human-readable and commonly used for configuration files, logs, and data interchange between systems.

²BLAS (Basic Linear Algebra Subprograms) is a set of standardised low-level routines used for performing common linear algebra operations such as matrix multiplication, vector addition, and dot products.

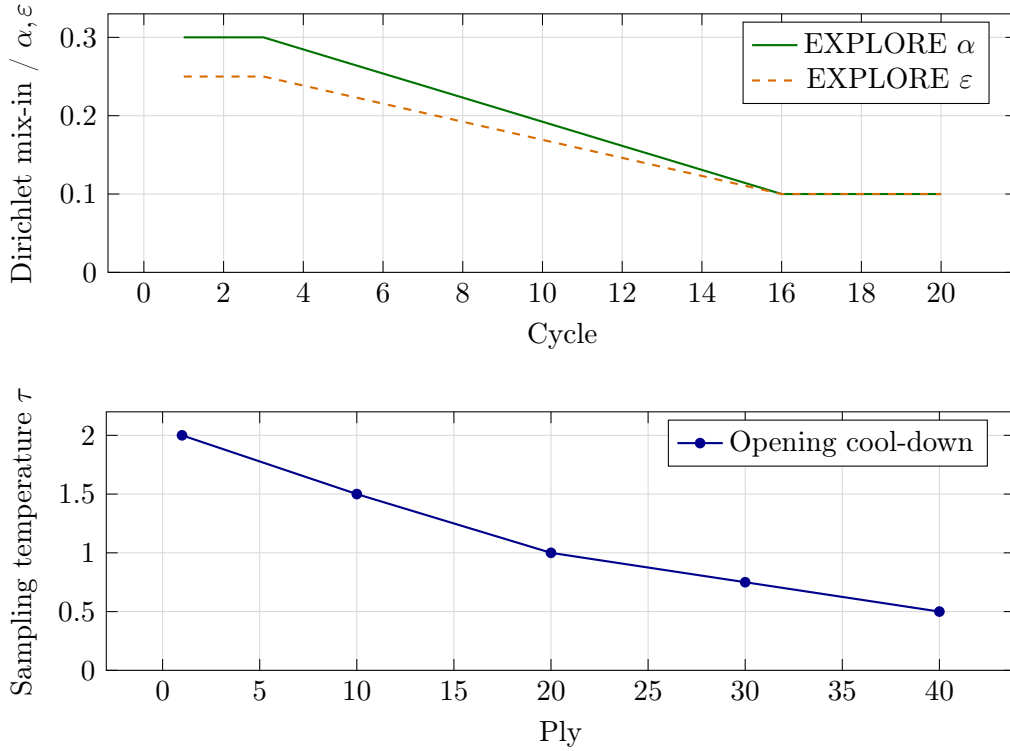


Figure 7.2: Chess pipeline: EXPLORE noise by cycle (both α and ϵ) and policy temperature by ply. Figure 7.3 compares the second of these with the draughts equivalent.

Chapter 1.

Portability guards are built in: the system detects Windows and notebook environments, chooses safe start methods, and wraps parallel sections in fallbacks. All filesystem writes are atomic and directories are created up front, so that an interrupted run cannot leave a partially written file behind.

7.2.3. Inference and Search Behaviour

When a GPU is available, a separate central inference process serves neural evaluations to all workers [33]. Requests flow through queues and are micro-batched, with adaptive flushing every few milliseconds or when a maximum batch size is reached. On non-CPU devices the model runs in FP16, applies the softmax only over the legal policy indices, and returns compact priors. A light LRU³ cache, keyed by BLAKE2b⁴ hashes [47] of the $18 \times 8 \times 8$ state planes plus the legal-index tuple or the evaluation perspective, deduplicates in-flight and repeated queries. If centralised inference is disabled, workers fall back to

³LRU (Least Recently Used) is a cache eviction algorithm that removes the least recently accessed items first when the cache reaches its limit.

⁴BLAKE2b is a cryptographic hash function designed for high-speed hashing and security.

local, device-aware inference.

Search behaviour is configured in three profiles, for evaluation, exploration and arena play. Each supports the early stopping and PUCT selection [14] described in Chapter 2, and root Dirichlet noise that anneals by cycle for exploration; the arena uses near-deterministic settings with a very small or zero mix-in. To guarantee opening diversity during exploration, the system probabilistically samples from a hard-coded opening book for the first 12 plies before handing full control to the agent’s search.

When the neural backend is off, or in early cycles, when baseline data is deliberately collected without it, a handcrafted heuristic policy supplies fast, usable priors. Rather than relying on static logic, the heuristic employs an exponential decay over cycles to phase itself out, transitioning control to the network while injecting biases for material gains, centre control, king safety and promotions. Its value estimate blends bounded material balance with in-check adjustments, mobility differences and mate-in-one detection.

7.2.4. Data, Adjudication and Promotion

During self-play every position is recorded with its encoded state, the player to move, and a factored policy target: PF over 64 from-squares and PD over 64×64 conditional deltas. The large conditional arrays are not stored densely: the pipeline compresses them into a sparse format, recording only the nonzero probabilities alongside their coordinate indices, which shrinks the dataset footprint by orders of magnitude while allowing exact reconstruction during training. All shard writes are atomic, and shards concatenate correctly even when rows are ragged. Training on this data is described in Chapter 6 and is not repeated here.

Targets and adjudication are training-aware. Terminal outcomes map to $+1/-1$ for checkmates or resignations from the side-to-move perspective and to 0 for other terminals. Draws receive a strict, symmetric penalty applied to both players regardless of material advantage, so that the agent does not seek a “winning draw”. Policy targets are lightly smoothed towards the uniform distribution over legal moves according to the branching factor. Games may end early by resignation after a minimum move count if evaluations stay below a threshold, or by quickdraw logic based on a configurable half-move clock.

Promotion is decided by a SPRT⁵ [48], testing the hypothesis that the candidate is meaningfully better — $p = 0.55$ against $p = 0.50$ — with controlled error rates. The system

⁵SPRT (Sequential Probability Ratio Test) is a statistical method for hypothesis testing. Applied here, it tests whether one network is meaningfully better than another, controlling type-I and type-II errors while using no more games than the evidence requires.

computes the score, an approximate relative Elo [18] and a 95% confidence interval, and tracks an absolute Elo rating across all cycles, writing the updated rating to an external ledger on a successful promotion.

Following the main match, a secondary mini-arena benchmarks the candidate against the baseline heuristic, giving an independent measure of progress against an opponent that is not part of the loop.

Operational hygiene supports auditability: a live-logging utility tees output to timestamped files, enables a fault handler, periodically flushes its files to disk, and writes a per-cycle JSON manifest capturing seeds, devices, configurations, quickdraw thresholds and SPRT parameters.

7.3. The Italian Draughts Pipeline

7.3.1. Orchestration and Schedules

The loop runs with one phase fewer: self-play, training and the promotion match repeat each cycle, with no separate evaluation phase. The promotion result itself is the per-cycle signal, and a lighter periodic strength check, described in Section 7.3.4, stands in for the separate evaluation phase of chess.

There is only ever one backend — the network together with its search — so there is no choice to make between a heuristic and a neural exploration source. The engine trains from a randomly initialised network from cycle one, consistent with the absence of curriculum learning noted in the introductory chapters. The promotion match pits the freshly trained candidate directly against the current champion, alternating colours in the same way.

Exploration noise is not annealed by cycle. The Dirichlet concentration and mixing weight are fixed constants for the entire run, and only the sampling temperature changes. It changes once, within each game rather than across cycles: the first 12 plies are sampled from the visit counts, and everything after them is played greedily. Figure 7.3 puts that schedule beside the chess one.

Because the self-play buffer is held in memory rather than in dataset shards, as established in Chapter 6, a cycle produces few of the file artefacts of the chess pipeline: no PGNs, since there is no move notation to write; no dataset shards; no JSON game logs and no manifest. What survives a cycle is one row of a metrics file, with game statistics, losses, phase timings and, when measured, the ladder rating and the scores against the fixed opponents; a resumption checkpoint holding the cycle index and the buffer, written

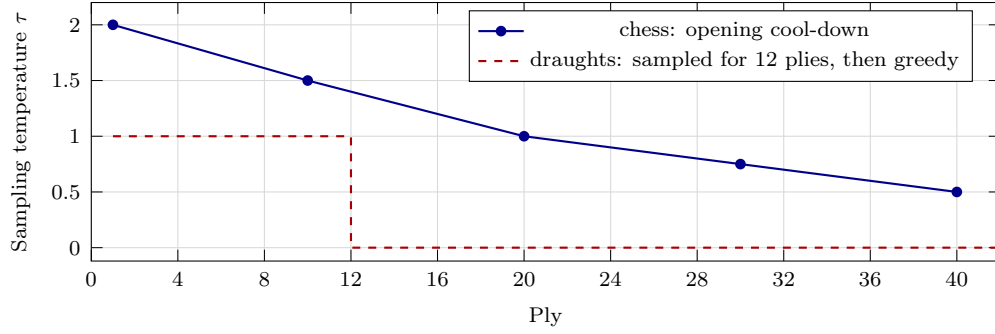


Figure 7.3: Sampling temperature by ply in the two engines: chess cools gradually over its first 40 plies, while draughts samples at $\tau = 1$ for 12 plies and then plays greedily.

atomically; and, when the candidate is promoted, an updated set of weights.

7.3.2. Parallelism and Portability

Self-play is parallelised through a process pool using the same spawn start method, but a simpler one: the worker count defaults to one fewer than the number of available cores, a static choice, not adjusted for whether a GPU is in use.

Oversubscription is guarded against through a different lever. Rather than relying on external thread-count environment variables, each worker process pins its own copy of the numerical library to a single computation thread on startup.

There is no push-and-pop low-copy hot path to lean on, since the environment itself has none. The pipeline’s own speed comes instead from two decisions: keeping the worker pool alive for the whole run rather than rebuilding it every cycle, and reloading a worker’s copy of the weights only when the file on disk has actually changed. Portability guards such as frozen-executable detection and notebook detection were not needed here and are not present.

7.3.3. Inference and Search Behaviour

Most of the centralised-inference machinery does not apply: there is no FP16 path and no cross-process cache, and on the CPU, the default device, there is no server at all; on a GPU, the single inference process described in Chapter 2 is used. Each search keeps its own transposition cache for its own duration and discards it afterwards. The softmax-over-legal-indices behaviour does carry over, in the form described in Chapter 5.

Search is not split into named profiles. A single configuration is reused for both self-play and promotion play instead of three separately tuned parameter sets; the only differences

are that root Dirichlet noise is switched off in the promotion match and that sampling there covers the first 10 plies rather than 12.

There is no opening book. With a small, largely symmetric starting position and no equivalent of chess's opening theory to seed, the exploration noise and temperature sampling already described are relied on for diversity from the first move. And, as established, there is no heuristic backend to fall back on: self-play always uses the current network.

7.3.4. Data, Adjudication and Promotion

Each self-play position is recorded in the same spirit — encoded state, player to move, a policy target, an outcome — but the target is not split into a from-square and delta pair. It is the single flat distribution over the compact action space of Chapter 4, already small enough that no sparse compression scheme is needed to store it. Training is covered in Chapter 6.

Adjudication follows a shorter list, built on the terminal conditions of Chapter 1. A side with no legal moves loses outright, mapping to $+1/-1$ from the side-to-move perspective. The no-progress counter reaching its limit is a true, unpenalised draw mapping to exactly 0: there is no equivalent of chess's symmetric penalty against a “winning draw”, since the mandatory-capture rule already makes a quiet, drawish game harder to sustain. One rule has no counterpart above: a game truncated by the move-count limit before reaching a natural terminal state is scored by material balance rather than treated as a draw, so that a game cut off for practical reasons still yields a training signal that leans the right way. There is no policy-target smoothing towards the uniform distribution, and no resignation or quickdraw logic — every self-play game runs until it reaches one of the two terminal conditions or the move-count limit.

Promotion

Promotion is decided by a sequential probability ratio test, as in the chess pipeline, testing the same hypothesis with type-I and type-II error rates both set to 0.05 and a cap of 400 games. The test stops as soon as the evidence is decisive in either direction; the cap exists for the case in which it never becomes decisive, and Chapter 8 reports how often that case arises.

A fixed win-rate threshold of 0.55 remains implemented as a fallback for when the sequential test is disabled. It was the original mechanism and was replaced because a fixed threshold over a modest number of games can promote or reject a candidate on the

strength of noise that a sequential test rejects.

Rating and Independent Measurement

The pipeline maintains an Elo scale of its own, though it is constructed differently from chess's absolute ledger. Every 10 cycles the current champion is frozen and never discarded, and each new frozen generation is played against the 3 preceding ones, its rating being derived from those results and appended to a persistent ladder. The scale is therefore relative and chained: each generation's figure depends on the figures of those before it. Section 8.3 examines how accurate that chained estimate turns out to be.

A second, independent yardstick exists for the same reason as chess's mini-arena: to catch a champion that is winning its own promotion matches without actually getting better. Every 20 cycles the champion plays 60 games against each of 2 fixed alpha-beta opponents, and on a coarser schedule a smaller number against a deeper one. Unlike the chess baseline, which is an internal heuristic that decays over the run, these opponents never change, so a score against them means the same thing at cycle 200 as at cycle 20.

The pipeline also implements, but leaves disabled, a further condition on promotion: a candidate approved by the match can be required to hold its own against a generation frozen some tens of cycles earlier, and be refused if it regresses against that fixed reference. Chapter 8 discusses why it was left inactive for the runs reported there.

Most of the operational hygiene described for chess is absent: no timestamped log files, no fault handler, no periodic flushing to disk and no per-cycle manifest. What survives a run for later inspection is the metrics file, the ladder with its frozen generations, and the weights.

8 | Simulation and Results: The Emergence of Strategy

8.1. Experimental Setup

Every result reported in this chapter comes from the Italian draughts instance. Draughts is the narrower of the two games, and its scale allows full training runs to be completed and repeated, which is what makes the measurements that follow consistent enough to compare with one another.

Two complete runs form the experimental basis of this chapter. They share every design choice and every hyperparameter except one: the number of self-play cycles, 100 in the first and 200 in the second. The comparison between them is the central question of Section 8.4, and the reason the two are presented side by side throughout rather than in separate sections. Table 8.1 gives their totals.

	Run A	Run B
Training cycles	100	200
Wall-clock duration	31.0 h	68.3 h
Mean cycle duration	18.6 min	20.5 min
Self-play games	40,000	80,000
Evaluation games	27,712	54,288
Total games played	67,712	134,288
Training positions	6,071,456	12,465,266

Table 8.1: The two runs analysed in this chapter. Every parameter other than the cycle count is identical between them.

Run B costs slightly more than twice Run A for twice the cycles: the mean cycle grows from 18.6 to 20.5 minutes, because the promotion match lengthens as the models converge and become harder to tell apart. Section 8.7 quantifies that effect, which turns out to be the dominant cost of the pipeline.

Some words on what a cycle is, since the unit recurs throughout the chapter. One cycle

consists of three phases executed in sequence, with a fourth measurement running on a periodic schedule rather than every cycle: the current champion plays a fixed number of games against itself, with search-guided move selection and exploration noise at the root; the positions so generated enter a replay window and a candidate network is trained on it; the candidate then plays a match against the champion, and replaces it only if it passes the promotion criterion; finally, on a periodic schedule, the champion is measured against opponents external to the learning loop. The first two are the learning; the third is the decision; the periodic fourth is the measurement. Sections 8.2 and 8.3 show that the fourth stops discriminating once the agent outgrows its opponents, and Section 8.7 that the third is what limits the system as a whole.

The configuration is the one arrived at through the calibration work of the preceding chapters, and Appendix A lists it in full. The values that matter for reading the results are collected in Table 8.2. Two of them deserve a comment here rather than in the appendix, because they behave differently from what their names suggest.

Parameter	Value
Self-play games per cycle	400
Simulations per move	400
Replay window	600,000 positions
Training epochs per cycle	2
Network	96 channels, 6 residual blocks
Network parameters	1.13 M
Exploration constant	1.5
Value-target discount	0.99
Search batch size	8

Table 8.2: Configuration shared by both runs. Appendix A gives the complete manifest.

The first is the search batch size: the number of leaf positions the search collects before querying the network. It reads as an implementation detail, a way of keeping the hardware busy, and it was originally chosen as one. Section 8.6 shows that it is not: it changes how the search behaves, and at values only slightly larger than the one used here it costs a substantial amount of playing strength. It is listed among the model hyperparameters, not among the performance settings, for that reason.

The second is the cycle count itself. It is not merely a stopping condition: the learning rate follows a cosine decay whose length is set by the total number of cycles, so a run configured for 200 cycles follows a different learning-rate trajectory from its very first cycle, not only from cycle 100. The two runs therefore differ in one setting but in two respects, and Section 8.4 returns to what this means for the conclusion that can be drawn

from their comparison.

Both runs were executed on the same machine: a single NVIDIA GeForce RTX 4090, with 32 logical processor cores, under Windows. Search and evaluation ran through the fast execution path and training in Python, both on the GPU. The fast execution path was checked against the Python implementation, with identical move generation and feature encoding, and the samples it produces are identical, bit for bit, regardless of how many parallel workers are used.

8.2. Evidence of Learning

The agent learns from nothing, and it learns quickly. The evidence is the periodic measurement against opponents that do not learn: alpha-beta searches at fixed depth, deterministic and identical from one cycle to the next, described in Chapter 7. Against depths 4 and 6 the champion plays 60 games. Depth 8 costs several times as much per game and was therefore sampled only occasionally during the runs; its column was completed afterwards, 30 games per entry, by replaying the frozen champions saved every 10 cycles. Those snapshots can be played against any fixed opponent at any time without retraining anything, and the same technique underpins Sections 8.3 and 8.4.

Cycle	Run A (100 cycles)			Run B (200 cycles)		
	depth 4	depth 6	depth 8	depth 4	depth 6	depth 8
20	1.000	0.867	0.917	0.950	0.975	0.800
40	1.000	0.983	0.967	0.983	0.900	0.867
60	0.975	1.000	0.983	0.983	0.992	0.967
80	0.992	1.000	0.967	0.992	0.983	1.000
100	1.000	0.983	1.000	1.000	0.958	0.950
120	—	—	—	1.000	1.000	0.967
140	—	—	—	1.000	0.992	0.983
160	—	—	—	1.000	0.992	0.917
180	—	—	—	1.000	0.917	1.000
200	—	—	—	1.000	0.992	0.983

Table 8.3: Score against fixed-depth opponents, from the learning agent’s perspective; 1.000 means every game was won. Depths 4 and 6: 60 games per entry. Depth 8: 30. A dash means Run A had already ended.

The trajectory is steep and short. By cycle 20 both runs already take between 9 and 10 points in 10 against depth 4, and by cycle 40 depth 6 is beaten almost without exception. Depth 8 — the deepest opponent measured, and the one that in earlier work had never been convincingly beaten — falls to 0.983 in Run A by cycle 60 and to 0.967 in Run B

at the same point. From there onward the three columns say the same thing in three different ways: the agent wins.

This is a real result and also, immediately, a problem. A measuring instrument that returns the same answer for every model it is shown has stopped measuring. The exhaustion is not quite simultaneous in the two runs: in Run A the three columns are statistically indistinguishable from one another after cycle 40, while in Run B depth 8 still separates the models at cycles 20 and 40 from every later one, and goes flat only from cycle 60. The instrument, in other words, keeps working as long as there is something left for it to see — which is what makes its subsequent silence worth taking seriously rather than dismissing as a defect of the opponent. Everything after that point is consistent with the agent improving enormously, and equally consistent with it not improving at all, at the search budget the runs themselves used. Section 8.3 takes up that difficulty; the rest of this section looks at what can still be read from the run itself.

The Training Loss as a Diagnostic

The most direct internal signal — the loss being minimised — turns out to say remarkably little (Figure 8.1).

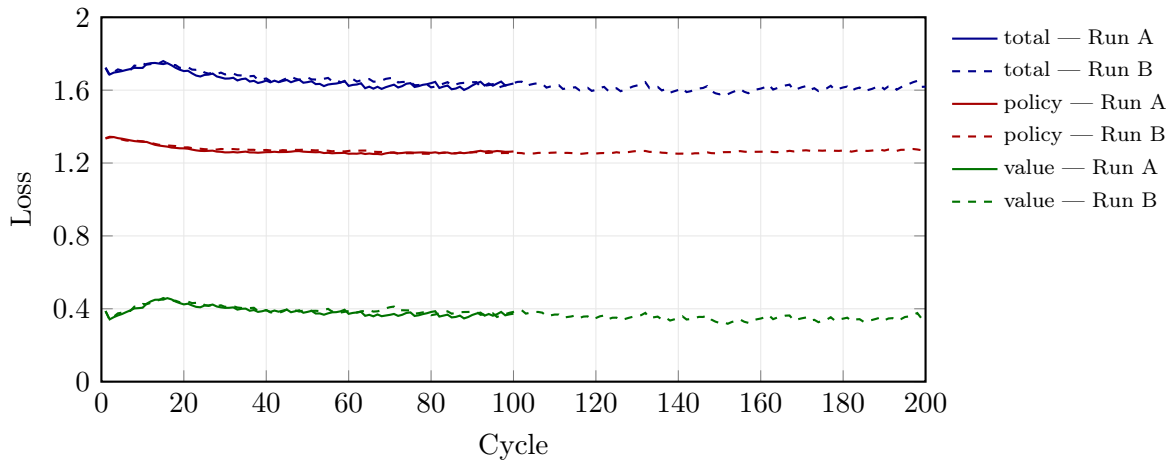


Figure 8.1: Total training loss and its policy and value components, per cycle, for both runs. The vertical scale is not truncated, because how little these curves move is the point of the figure.

Over 100 cycles the total loss falls from 1.724 to 1.636, and its policy component from 1.336 to 1.264, about 5% in both cases, with a transient increase around cycle 15. Run B behaves identically over twice the length: 1.722 to 1.619 in total loss and 1.335 to 1.271 in the policy component, 6% against a doubling of the training. This is difficult to reconcile

with the strength actually gained over the same interval. Table 8.3 cannot settle the point, since its first entry already shows the agent winning; the comparison that can is in Section 8.4, where the final champion is played directly against a snapshot of itself taken at cycle 20 and is found to be more than 200 Elo points stronger. Between those two cycles the objective being minimised moved by 4%.

They are reconcilable, and the reason was established in the diagnostic work of Chapter 6: a large share of the policy target is irreducible. The search distributes its visits over several moves of genuinely comparable value, and no network can predict which of them the search will favour on a given run. The loss therefore has a floor that has little to do with playing strength, and the agent spends most of the run moving within a narrow band above it. The practical consequence for this chapter is worth stating once and remembering throughout: in this pipeline the training loss is a diagnostic of optimisation, not a measure of strength, and no argument about the agent getting better will rest on it.

The Shape of the Games

The signals that do move are behavioural (Figure 8.2): properties of the games the agent plays against itself, rather than of the objective it minimises.

Games lengthen substantially: from 121 plies in the first cycle to 159 at cycle 100, with most of the increase concentrated in the first 15 cycles. Run B follows the same curve and continues past it, reaching 169 plies by cycle 200; its draw rate collapses to 0.318 at cycle 8 and recovers to 0.454 over the last 20 cycles, against 0.427 for Run A. Longer games are what one expects as play becomes sounder on both sides, since blunders that end a game early become rarer.

In both runs the games keep lengthening to the last cycle, and neither flattens: the 20-cycle averages climb from 144 to 157 plies in Run A and from 142 to 162 in Run B. Run B ends higher — 169 plies at cycle 200 against Run A's 159 — because it runs for longer rather than because it grows faster; from cycle 40 onward its games lengthen at about 6 plies per 100 cycles against Run A's 11.

This is worth noting precisely because it does not track strength. Section 8.4 establishes that Run A gained nothing measurable after cycle 50, and its games went on getting longer throughout that period. Game length is the one signal in this chapter that never plateaus, and it is not evidence that the player is still improving: a quantity can move steadily in the right direction while meaning nothing about strength.

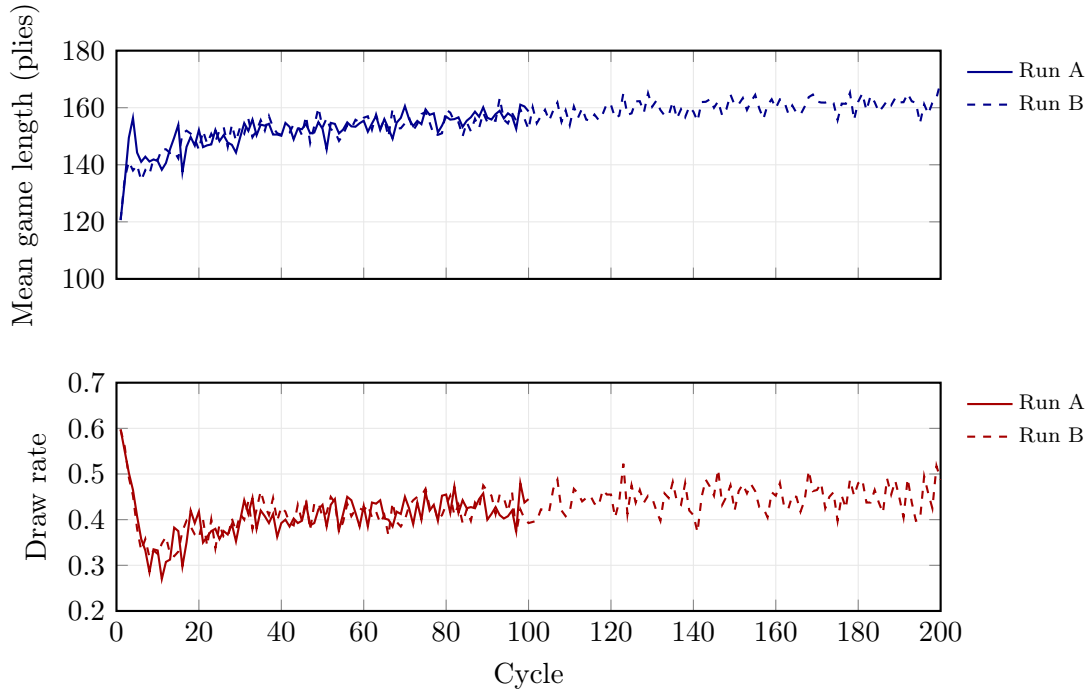


Figure 8.2: Mean game length and the proportion of self-play games ending in a draw, per cycle, for both runs. Unlike Figure 8.1, both vertical scales are truncated, because here the movement is the finding rather than its absence.

The proportion of draws is more interesting, because it does not rise monotonically — it collapses and then recovers. It begins at 0.598, falls to a minimum of 0.270 at cycle 11, and then climbs back to around 0.45, where it remains for the rest of the run. The shape is characteristic of self-play and admits a natural reading. Early on, neither side can force anything, and a large share of games simply exhaust the no-progress limit. The network then learns to convert an advantage well before it learns to prevent one, so decisive results become common and draws collapse. Finally the defensive side of play catches up, and the draw rate settles at a level that reflects two solid opponents rather than two weak ones. The initial and final draw rates are numerically similar and mean opposite things, which is a useful reminder that a single aggregate statistic can be read correctly only alongside the others. Run B traces the same curve with a shallower trough — a minimum of 0.318 at cycle 8 against Run A’s 0.270 at cycle 11 — and settles slightly higher, averaging 0.454 over its last 20 cycles against Run A’s 0.427.

The agent therefore learns fast, and by roughly cycle 40 in Run A and cycle 60 in Run B it has outgrown every opponent available to measure it. What it does after that, and whether 200 cycles produce a better player than 100, cannot be answered with the instruments used so far. The next section takes up that problem, and Section 8.4 returns to the

comparison itself.

8.3. The Limits of Relative Measurement

The difficulty that Section 8.2 ran into is not an accident of the opponents chosen, and it is worth treating in general terms, because it applies to any system that learns by playing against itself: this one, its chess counterpart, and the published systems both are modelled on.

A self-play system has no external teacher. It generates its own training data and it certifies its own progress: a candidate network replaces the champion when it beats that champion by a sufficient margin. Every quantity the pipeline computes about itself is therefore measured against a reference that moves along with the thing being measured. This is not a defect of the implementation, it is what makes self-play possible at all, but it has a consequence that is easy to overlook. A sequence of steps, each clearing a threshold only slightly above parity, resembles a random walk at least as much as it resembles a climb. The mean promotion score was 0.546 in Run A and 0.528 in Run B: each individual promotion carries very little evidence, and the 109 of them across the two runs carry less than their number suggests.

Establishing that the agent has improved therefore requires a reference that does not move. Over the course of this work four distinct ways emerged in which a strength measurement can appear to work while telling one nothing, and each was encountered in practice rather than anticipated. They are worth setting out before the results, because the remainder of the chapter is organised around avoiding them. Table 8.4 states each of the four with its mechanism and its symptom.

Four Modes of Failure

The first two are opposite faces of the same coin, and the symmetry is the useful observation: a reference loses discriminating power both when it is too weak and when it is too strong.

An opponent that always loses and an opponent that always wins return the same uninformative answer for any two models compared through them. Section 8.2 showed the first case arriving by cycle 40 in Run A and cycle 60 in Run B; Section 8.5 will show the second when the agent is placed against a competent external engine. Discrimination comes from references close to the level being measured, which — for a system that has outgrown everything external available to it — means the system's own earlier states.

Failure	Mechanism	Symptom
Saturation	The reference is too weak: the agent beats it almost always	Every model scores near the maximum; two models of very different strength are indistinguishable
Overwhelm	The reference is too strong: it beats the agent almost always	Every model scores near zero; differences compress into the few drawn games and become noise
Chained estimation	Ratings are propagated along a sequence of relative comparisons, so error accumulates rather than cancelling	The estimate drifts from the direct measurement, and the relative error grows as the true difference shrinks
Shared blind spot	A defect is applied identically to both sides of the comparison, so it cancels exactly	The measurement returns parity at any sample size; the instrument cannot see what it was built to see

Table 8.4: Four modes of failure for strength measurement in a self-play system. Each was observed in this project.

Those are kept: a snapshot of the champion is frozen every 10 cycles and never discarded, so any past version can be replayed against any later one at any time, without retraining anything.

The fourth failure is the least obvious and, in this project, the most consequential. Section 8.6 documents a search parameter that turned out to cost a substantial amount of playing strength, and which the promotion match could not have detected at any sample size: both the candidate and the champion played under the same parameter, so the handicap applied equally to both and the score returned parity. Detecting it required constructing a deliberately asymmetric comparison, in which the two sides differ in exactly the respect under suspicion. The lesson generalises beyond this parameter: a measurement that treats both contenders identically is blind, by construction, to anything that affects both contenders identically.

Three Verdicts on One Model

That these failures are not hypothetical is best shown by a case in which three measurements disagreed about the same model. It arose in an earlier run of the pipeline, not in either of the two analysed in this chapter, and it was the observation that prompted the methodology adopted here.

At one cycle of that run, the three instruments then in use reported the three verdicts of Table 8.5.:

Instrument	Verdict	Nature
Promotion match	0.590 over 144 games: promote	relative
Generation ladder	+819, the highest value of the run	relative, chained
Fixed-depth opponent, depth 6	0.433 over 60 games	fixed

Table 8.5: Three simultaneous verdicts on a single model: the two relative instruments agree, and the fixed one dissents.

The disagreement was resolved by replaying the games: three neighbouring generations were played against the same fixed opponent from the same starting positions, with nothing varying but the network. The generation in question scored 0.475, where the one from 10 cycles earlier scored 0.850 and the one from 10 cycles later scored 0.725. The difference between the first two is 0.375, corresponding to $z = 2.7$ and $p \approx 0.006$: the drop was in the model, not in the measurement, and the trajectory recovered on its own afterwards.

Two secondary observations point the same way. The model concerned produced 17 distinct games out of 20 against the fixed opponent, where its neighbours produced 20 out of 20 — it played more narrowly, leaving its opponent fewer equally-valued replies to choose between. And of every column recorded for that cycle, the only one lying outside 2 standard deviations of the surrounding cycles was the entropy of the search target, at 1.033 against a local mean of 1.073, $z = -3.54$.

What this establishes is not that the pipeline is broken. It is that a promotion decided by a relative criterion can make the champion worse in absolute terms, that the chained rating not only fails to notice but records its own maximum at precisely that point, and that only the fixed reference dissents. Any claim of progress resting on the first two measures is therefore unsafe on its own.

The Drift of the Chained Estimate

The magnitude of the third failure can be quantified directly, because the two endpoints of a chain can always be played against each other without the intermediate links. Tables 8.6 and 8.7 set the ladder’s figure beside that direct measurement, snapshot by snapshot.

Compared with	Ladder Elo	Score	Direct Elo	W–D–L	Error
cycle 20	+397	0.792	+232	15–8–1	1.7×
cycle 30	+318	0.792	+232	18–2–4	1.4×
cycle 40	+175	0.729	+172	15–5–4	1.0×
cycle 50	+160	0.562	+44	8–11–5	3.6×
cycle 60	+140	0.552	+36	14–25–9	3.9×
cycle 70	+113	0.594	+66	16–25–7	1.7×
cycle 80	+118	0.583	+58	16–24–8	2.0×
cycle 90	+37	0.479	–14	9–28–11	sign

Table 8.6: Run A: the final champion against each frozen snapshot of its own lineage. The ladder column is propagated along the chain of ten-cycle comparisons; the direct columns are measured by playing the two models against each other. 24 games per row up to cycle 50, 48 thereafter. The last column is the ratio of the ladder’s figure to the direct measurement, or sign where the ladder reports a gain that is really a loss.

Compared with	Ladder Elo	Score	Direct Elo	W–D–L	Error
cycle 20	+467	0.896	+374	19–5–0	1.2×
cycle 40	+255	0.750	+191	15–6–3	1.3×
cycle 60	+236	0.729	+172	13–9–2	1.4×
cycle 80	+173	0.729	+172	13–9–2	1.0×
cycle 100	+137	0.625	+89	19–22–7	1.5×
cycle 120	+107	0.635	+97	18–25–5	1.1×
cycle 140	+74	0.635	+97	21–19–8	0.8×
cycle 160	+71	0.615	+81	18–23–7	0.9×
cycle 180	+21	0.458	–29	8–28–12	sign

Table 8.7: Run B, measured in the same way as Table 8.6 and on the same scale. 24 games per row up to cycle 80, 48 thereafter. The ladder is far more accurate here, and at cycles 140 and 160 it understates rather than overstates; it fails in the same place as in Run A, at the final snapshot.

Figure 8.3 plots the two columns of each table against each other. Two regularities appear across the 17 paired comparisons, and together they identify the mechanism rather than merely its size. First, the overstatement is not uniform: it is close to 1.0 wherever the true difference is large, and it explodes exactly where the true difference approaches zero — the worst ratios in Run A, 3.6×

identifies, while Run B, which keeps genuinely improving for much longer, is never worse than $1.5\times$. Second, and more seriously, in both runs the chain gets the sign wrong at the last snapshot: the chain reports a gain of $+37$ and $+21$ Elo where the direct match finds losses of -14 and -29 .

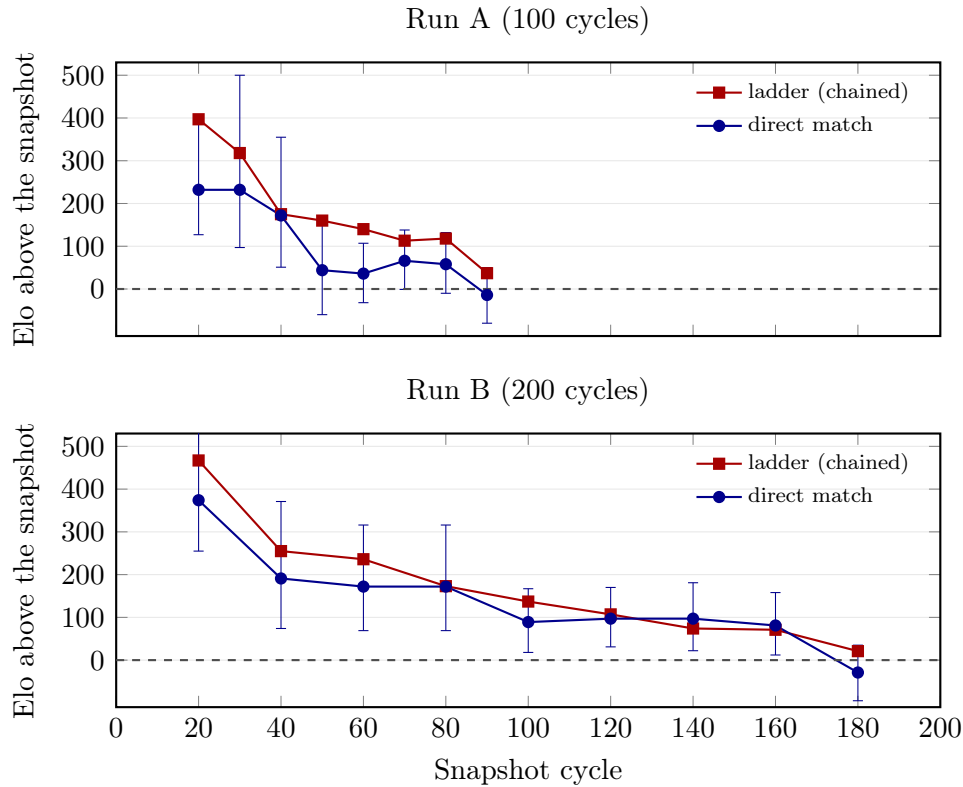


Figure 8.3: What the pipeline’s chained ladder claims for each frozen snapshot, against what the two models score when played directly. Error bars are 95% intervals on the direct measurement; the ladder carries none, being a propagated estimate rather than a match. Both panels share the same scales, so Run A occupies the left half of its own.

This is what a chain of relative estimates does. Errors along it accumulate rather than cancel, so the absolute drift is roughly constant while the quantity being estimated shrinks; the relative error therefore grows without bound as the true difference goes to zero, and the sign flips once the drift exceeds it. The practical consequence is precise: the chained rating is least trustworthy exactly where one most wants to trust it, which is at the end of a run, deciding whether the last cycles bought anything.

Both measurements use the search budget of the run itself, so the discrepancy cannot be attributed to differing conditions.

The Protocol Adopted

The four failure modes lead to a short list of requirements, which the remainder of this chapter follows and which is stated here in general form, since nothing in it is specific to draughts:

1. **Compare models directly.** Two networks, a large number of games between them, diversified openings, the result with its confidence interval. No intermediate links, and therefore no accumulated error.
2. **Choose references near the level being measured.** Frozen earlier states of the same lineage, rather than opponents that are certain to win or certain to lose.
3. **Make the comparison asymmetric when testing a shared setting.** Any parameter applied to both contenders is invisible to a symmetric match, and must be varied on one side only.
4. **Keep one fixed external reference regardless.** It cannot measure progress once saturated, but it is the only thing that answers where the system stands rather than how it has changed.

This protocol has one further property that matters in this particular game, and which Section 8.5 will make use of: the draw rule, a known source of ambiguity when comparing against an external opponent, applies identically to both sides of a direct match between two of the system's own models, and cancels out.

8.4. The Value of Additional Cycles

Whether a longer run produces a better player is the central question of the chapter, and the only one for which a prediction existed before the experiment was run. Earlier work on this pipeline had sampled a single long trajectory at cycles 50, 100, 150, 200 and 300, measuring each sample against the same fixed opponent, and had found no difference that the instrument could resolve beyond cycle 100: the comparison between the models at cycle 100 and at cycle 300 returned $p = 1.00$. The operational conclusion drawn at the time was that additional cycles beyond roughly 100 buy nothing measurable, and that two independent runs are worth more than one run of twice the length.

Two runs differing only in their cycle count put that conclusion to the test. The evidence comes in three parts: where progress stops within each run, how the two final champions compare when played against each other, and how both compare to the best model produced under an earlier configuration.

Progress Within a Single Run

The frozen generations make it possible to ask, of a finished run, at which point it stopped improving — a question that cannot be asked while the run is in progress, since the answer requires the final model. Each champion is played against every snapshot of its own lineage, at the search budget of the run itself. Tables 8.8 and 8.9 give the results.

Snapshot	Games	Score	Elo	95% interval	W–D–L	<i>p</i>
cycle 0	24	1.000	—	—	24–0–0	
cycle 10	24	0.979	+669	[+473, ∞)	23–1–0	
cycle 20	24	0.792	+232	[+127, +399]	15–8–1	
cycle 30	24	0.792	+232	[+97, +500]	18–2–4	
cycle 40	24	0.729	+172	[+51, +355]	15–5–4	
cycle 50	24	0.562	+44	[−60, +156]	8–11–5	0.41
cycle 60	48	0.552	+36	[−32, +107]	14–25–9	0.30
cycle 70	48	0.594	+66	[−1, +138]	16–25–7	0.05
cycle 80	48	0.583	+58	[−10, +132]	16–24–8	0.10
cycle 90	48	0.479	−14	[−80, +50]	9–28–11	0.66

Table 8.8: Run A: the final champion against each frozen snapshot of its own lineage; neither Elo nor interval is defined where the champion won every game. Scores are from the champion’s perspective, so a value above 0.5 means the final model is stronger. The *p*-value tests the hypothesis that the two are of equal strength; it is reported only where the outcome is close enough for the question to arise.

Snapshot	Games	Score	Elo	95% interval	W–D–L	<i>p</i>
cycle 0	24	1.000	—	—	24–0–0	
cycle 20	24	0.896	+374	[+255, +666]	19–5–0	
cycle 40	24	0.750	+191	[+74, +371]	15–6–3	
cycle 60	24	0.729	+172	[+69, +316]	13–9–2	
cycle 80	24	0.729	+172	[+69, +316]	13–9–2	
cycle 100	48	0.625	+89	[+18, +167]	19–22–7	0.01
cycle 120	48	0.635	+97	[+31, +170]	18–25–5	0.005
cycle 140	48	0.635	+97	[+22, +181]	21–19–8	0.01
cycle 160	48	0.615	+81	[+12, +158]	18–23–7	0.02
cycle 180	48	0.458	−29	[−95, +35]	8–28–12	0.37

Table 8.9: Run B, measured in the same way and on the same scale, sampled every 20 cycles because the run is twice as long.

Run A behaves, down to cycle 40, exactly as a steadily improving system should: the score falls as the reference approaches the final model, from complete dominance over the initial random network to a 0.729 advantage over the model of 60 cycles earlier. Then it

stops falling. The last five values — 0.562, 0.552, 0.594, 0.583, 0.479 — are not ordered, and the final champion loses outright to the snapshot taken 10 cycles before it.

The ordering is not, however, what the conclusion rests on. From cycle 50 onward not one snapshot of Run A is distinguishable from its final champion. Every confidence interval in that group contains zero, and the smallest p -value among the five is 0.05, at cycle 70 — which is what five comparisons would be expected to produce by chance alone. The statement this supports is simple and does not require reading a pattern out of individually inconclusive results: the last 50 cycles of Run A, half the run and some 15 hours of the 31, produced no improvement this measurement can detect.

Run B does not behave that way. Every snapshot up to and including cycle 160 is beaten significantly, and only the one taken at cycle 180 is indistinguishable from the final model. Where Run A stops improving at half its length, Run B keeps making detectable progress to roughly four fifths of its own. The shape differs too: Run A degrades into disorder, while Run B settles onto a plateau from cycle 100 to 160 — the four values there, 0.625, 0.635, 0.635 and 0.615, are indistinguishable from one another and all sit about 90 Elo below the champion — and then makes a further real gain between cycles 160 and 180.

Both tables also carry the material for Section 8.3’s third failure mode: the pipeline’s own generation ladder, propagated along the chain of comparisons, claims figures for these same pairs that overstate the direct measurement by up to a factor of 4 in Run A, and reverse sign at the last snapshot of both runs. Tables 8.6 and 8.7 give that comparison in full.

The Two Champions, Head to Head

The comparisons above are internal to each run, and they cannot be put side by side: each is measured against its own lineage, on its own scale. That Run B improves for longer does not by itself mean it finishes higher — it could climb for longer because it starts lower. The direct match of Table 8.10 settles it.

Games	100
Score, Run B’s champion	0.595 ± 0.035
95% interval	0.526 – 0.664
Elo difference	+67 [+18, +119]
W–D–L	36–47–17
Distinct games	100

Table 8.10: The two final champions played directly against each other, with diversified openings and the search budget used during training.

With 36 wins against 17 and an interval that excludes parity, the 200-cycle run produced the stronger player. The prediction that opened this section is therefore wrong, at least for runs of a correct configuration. The plateau that earlier work had read as a ceiling on useful training was, on the evidence of Section 8.2, largely the saturation of the instrument that had been used to look for it.

Two things are worth noticing about how the answer arrives. The first is that it is consistent with the internal evidence: Run B keeps improving for longer against its own lineage, and it is Run B that wins the direct match. Two independent measurements, one relative and one absolute in the sense that matters here, agree.

The second is the size. A difference of 67 Elo is real but modest, and Table 8.11 puts it beside the other two levers this chapter measures.

Lever	Measured effect
Doubling the cycles, 31 h $\rightarrow$ 68 h	+67 Elo
The corrections of Chapters 6 and 7, at a third of the compute	+164 Elo
Mis-setting the search batch size (Section 8.6)	−293 Elo

Table 8.11: The three levers this chapter measures, on a common scale.

Doubling the computation bought less than half of what correcting the machinery bought, and less than a quarter of what a single mis-set performance parameter would have cost. Compute is the least efficient of the three levers examined here, and it is also the only one whose price is paid in wall-clock hours.

A Prior Comparison and Its Confound

There is a third comparison available, and it points the same way while being less clean. The best model produced by an earlier configuration of this pipeline was trained for 300 cycles — three times Run A’s budget. Both champions were played directly against it, 100 games each with diversified openings, with the results in Table 8.12.

	Run A	Run B
Score against the 300-cycle model	0.720 $\pm$ 0.028	0.765 $\pm$ 0.026
95% interval	0.665–0.775	0.714–0.816
Elo difference	+164 [+119, +214]	+205 [+159, +259]
W–D–L	47–50–3	54–45–1
Distinct games	94	90

Table 8.12: Both champions against the best model of an earlier configuration, trained for 300 cycles.

With 47 wins against 3 losses, and 54 against 1, both of the shorter runs produce a substantially stronger player. That is a striking result, and it must immediately be qualified, because the models differ in far more than their cycle count. The earlier configuration predates a series of corrections to the pipeline’s measurement and training machinery, described in Chapters 6 and 7, and there is no way to attribute the gap to length rather than to those corrections. What the comparison establishes is that three times the compute, spent under a worse configuration, produces a worse player — which says something about the relative value of compute and correctness, and nothing directly about whether 100 cycles or 200 is the better budget.

It does, however, provide a check on the method itself. Run A is +164 over the earlier model and Run B is +67 over Run A, which predicts +231 for Run B over the earlier model; the direct measurement gives +205. The discrepancy of 26 Elo is comfortably inside the confidence intervals of all three matches. Three independent 100-game comparisons compose transitively to within their own error — which is precisely what the chained estimate of Section 8.3 fails to do.

One Setting, Two Respects

A caveat applies to the comparison between Runs A and B, and it should be stated rather than left for a reader to notice. The two runs were configured differently in exactly one place — the number of cycles — but they differ in two respects, because that number does more than decide when to stop. The learning rate follows a cosine decay whose length is set by the total cycle count, so a run configured for 200 cycles begins decaying more slowly from its very first cycle. Run B is not Run A continued; it is a different trajectory throughout.

That this is more than a theoretical concern can be seen in the early cycles. At cycles 20 and 40, measured against the same fixed depth-8 opponent with the same games and the same seeds, Run A scores 0.917 and 0.967 where Run B scores 0.800 and 0.867; neither difference is significant on its own, and combining the two gives $p = 0.05$. The behavioural signal is plainer than the score: at cycle 20 Run B needs a median of 169 plies to finish those games where Run A needs 89. Against the depth-6 opponent, however, the two runs trade places from one measurement to the next, so the evidence that Run B starts more slowly comes from one column of Table 8.3 and is not corroborated by the other.

The question this chapter can therefore answer is the practical one — is a 200-cycle run worth commissioning in preference to a 100-cycle one? — and not the scientific one, whether additional training would help if everything else were held fixed. Separating the

two would require a third run, in which the cycle count is raised while the learning-rate schedule is held to the shorter one, and that run has not been performed.

Additional cycles are therefore worth their cost, with two qualifications that matter as much as the answer. A run of 200 cycles produces a measurably stronger player than 100 — +67 Elo, with the internal evidence of both runs agreeing — so a longer run is worth commissioning where the wall-clock time is available. But the gain is small against the other levers this chapter measures, and it answers a question about this pair of configurations rather than about training length in the abstract. Section 8.7 argues that the reason the return is this modest lies not in the quantity of computation but in what each cycle is permitted to accept.

8.5. Placement on an Absolute Scale

Everything measured so far is relative: to fixed opponents the agent has outgrown, or to earlier versions of itself. Neither says where the resulting player stands against someone who plays this game seriously.

For Italian draughts there is one reference available. It is a mature engine combining a neural evaluation with search, an opening book of over a million positions, and endgame tables complete to ten pieces [49]. It is far stronger than anything produced here, so it was deliberately weakened for the comparison: 0.01 seconds per move, opening book disabled, transposition table reduced by three orders of magnitude. Each model played 50 games, with colours alternating and 800 simulations per move on our side — twice the budget used everywhere else in this chapter, because this is the one measurement in which the opponent does not scale down with us.

A Premise of Validity

A comparison across two independent implementations of the same game is worth nothing unless both are playing that game. Italian draughts has a hierarchy of capture obligations that few implementations reproduce in full, and a divergence on a single tie-breaking rule does not announce itself as an error: it produces perfectly ordinary games whose results mean nothing.

The correspondence between the two board representations was therefore measured, not assumed, before any game was played. On 120 positions reached by random play, including 24 in which a capture was obligatory, the two engines agreed on the complete set of legal moves in 120 cases out of 120.

The Result

Table 8.13 gives the four matches: the two champions of this chapter and the two models that preceded them.

Model	W	D	L	Score	Elo
62 cycles, defective promotion match	0	4	46	0.040 [0.002, 0.078]	−552
300 cycles, corrected	0	19	31	0.190 [0.123, 0.257]	−252
Run A, 100 cycles	0	27	23	0.270 [0.201, 0.339]	−173
Run B, 200 cycles	0	30	20	0.300 [0.232, 0.368]	−147

Table 8.13: Against the external reference, weakened as described. 50 games each, colours alternating.

The four point estimates rise monotonically across the four configurations, and not one of the adjacent pairs is significantly separated. Run A exceeds the 300-cycle model with a third of the compute, at $p = 0.107$; Run B exceeds Run A by $+0.030$ with a standard error of 0.049 , $p \approx 0.54$. At 50 games a side this instrument does not separate anything, and claiming otherwise would repeat exactly the error this chapter attributes elsewhere to relative measures.

The result that does stand out is in the first column. No wins, in any of the four models. All the progress is in draws: 4, 19, 27, 30. Split by colour, both champions take 16 draws and 9 losses as White (0.32); as Black, Run A takes 11 draws and 14 losses (0.22) against Run B's 14 and 11 (0.28). The asymmetry is consistent with a first-move advantage that survives even at this deficit.

Material Balance in Quiet Positions

A score obtained entirely through draws admits two very different readings, and they matter for what one concludes about the system. Either the agent reaches advantageous positions and fails to convert them, or it never reaches them at all. All the games were replayed move by move to find out.

One methodological point determines the answer. Material must be counted only in quiet positions — those in which the side to move has no capture available. Under a mandatory-capture rule the count swings within every exchange, and measuring after each ply produces the finding that the agent goes ahead in all 50 games, which is an artefact of the counting rather than a property of the play.

Table 8.14 counts quiet positions only, and contrasts the external match with the strongest fixed-depth opponent for reference.

	vs. external engine		vs. depth 8	
	Run A	Run B	Run A	Run B
Score	0.270	0.300	1.000	0.983
Median peak quiet advantage	0	+1	+4	+4
Games never ahead	15 of 27 draws	10 of 30 draws	0 of 30	0 of 30
Durable advantage of two or more	0 of 50	0 of 50	19 of 30	24 of 30
Median material at end	0 / -2	0 / -3	+4	+4

Table 8.14: Material balance in quiet positions, from the agent’s perspective, with a king counting as one piece. An advantage counts as durable if held for at least 10 consecutive quiet positions. Where two figures are given, the first is for drawn games and the second for lost ones.

The answer is unambiguous, and it is the same for both champions. In none of the 100 games against the external engine does either model hold a two-piece advantage for any length of time. The draws are not conversions missed; they are games held level and closed by the no-progress limit. Against an opponent within its reach the same agent behaves in the opposite way entirely, building an advantage in every one of 30 games and converting all but one. Figure 8.4 shows the two distributions whole, rather than through their medians.

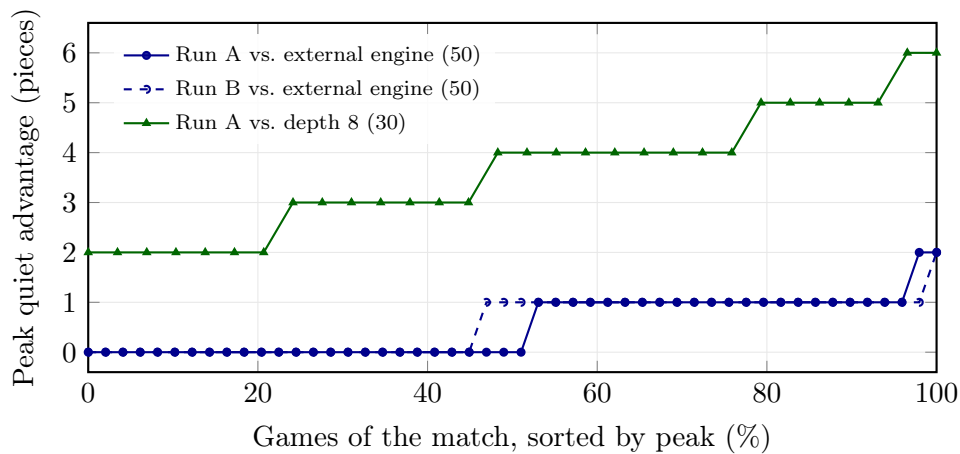


Figure 8.4: Peak material advantage reached in a quiet position — one in which the side to move has no capture available — game by game, with the games of each match sorted independently from the smallest peak to the largest. A king counts as one piece.

Between the two runs there is a difference of degree that the score does not show. Run B gets ahead more often — a median peak of one piece against Run A’s zero, and only a third of its draws spent never ahead against Run A’s 55% — and it also loses by more when it loses, a median of 3 pieces against 2. It plays a longer, more contested game and

is punished harder for it, which is what one expects of a player that has learned to hold on rather than to fold.

What has improved across the four runs, then, is the capacity to survive: to hold material against a stronger player. That is a real gain, and the draw counts measure it correctly, but it is a different gain from the one the phrase “progress is all in the draws” might suggest.

A Band Rather Than a Point

Every draw in both matches was closed by the no-progress limit — none by material, none by repetition. That limit does not coincide with the one in the federal rules, which counts moves by a designated player, requires a king on each side, and resets on any man move or change in material. The divergence was measured on both champions directly, on the same bench and with the same seeds: 24 self-play games each, all 24 distinct. Of the games closed by the implemented rule, 6 of Run A’s 12 and 5 of Run B’s 15 would not have been granted a draw under the official one.

Table 8.15 applies those rates to each model, treating the games that would have continued as having an unknown outcome somewhere between a draw and a loss, since the agent never wins against this opponent.

Model	Draws	Divergence	Corrected score	Corrected Elo
300 cycles	19	25%	0.142 – 0.190	–312 ... –252
Run A	27	50%	0.135 – 0.270	–323 ... –173
Run B	30	33%	0.200 – 0.300	–241 ... –147

Table 8.15: The effect of the draw rule. The divergence rates for the two runs were measured on the same bench; the rate for the 300-cycle model comes from an earlier, separate measurement and has not been redone.

The difference between the two rates is not significant — 6 of 12 against 5 of 15 gives $p = 0.45$ — so it should not be presented as a property of the two playing styles. The bands still overlap, and Run A’s can fall below that of the 300-cycle model: its apparent advantage, already not significant, does not survive the correction.

Run B is nonetheless the least ambiguous of the three. It draws more often and owes a smaller share of those draws to the house rule, so its band is narrower than run A’s and the highest of the three, and its lower bound of 0.200 sits above the whole band of the 300-cycle model. It remains overlapped with run A, so nothing is demonstrated against it.

The Limits of an Overwhelming Reference

The outcome is the one Section 8.3 predicted, and the match of Section 8.4 makes it possible to watch the instrument fail against a known answer: that comparison established, by direct match over 100 games, that Run B's champion is 67 Elo points stronger than Run A's, with an interval excluding parity. The external reference, given 50 games against each, returned 0.270 and 0.300 and could not distinguish them at $p \approx 0.54$. The difference was real and the instrument did not see it.

The mechanism is the one Table 8.4 names. A reference that beats the agent almost every time compresses all the information about relative strength into the handful of games it fails to win. Those games are few, they are noisy, and in this particular pairing they additionally depend on an arbitration rule that does not match the official one — so the residual signal is not merely small but partly spurious.

The external engine therefore answers the question this section asked — the system stands between 150 and 175 Elo points below a deliberately weakened version of a mature program, and does not beat it — and demonstrably cannot answer the question of Section 8.4. The two roles are distinct, and no single reference fills both.

8.6. Performance Settings and Playing Strength

Most of the configuration was settled before either run began, by the calibration work reported in the preceding chapters: the exploration constant, the search budget, the discount applied to the value target, the size of the replay window, the number of training epochs per cycle. Those settings are listed in Appendix A and none of them produced a surprise. One other setting did, and it was not among the hyperparameters at all — it had been chosen as a performance setting, on the reasonable assumption that it could not affect how the system plays — and it turned out to be the single most consequential value in the configuration.

Two Families of Parameter

A pipeline of this sort exposes two families of knob. Some govern how the computation is distributed: how many games run in parallel, which device carries the network, how the work is divided among processor cores. Others govern what the system computes: the depth of search, the shape of the network, the training objective. It is natural to assume the first family is free — that it changes only how long one waits.

For one of the two knobs examined here that assumption holds exactly. Raising the number of parallel workers from 13 to 24 cut the cost per ply by 46% in self-play and 38% in the evaluation matches, which the calibration bench projected would take a 100-cycle run from 67 hours to 43. It also leaves the training data untouched in the strictest possible sense: each game carries its own seed and its own search, so the number of workers changes only how games are distributed across cores. Samples produced under different worker counts were compared and found identical, bit for bit.

Those hours are projections and should be read as such. Run A, executed at the higher worker count, actually completed in 31 hours against the 43 projected — the bench overestimates by 39%, because it extrapolates from the game lengths of a mature network while the early cycles of a run play much shorter games. The ranking of the settings is reliable; the absolute figures are not, and the same caution applies to every projection in this section.

The second knob is the search batch size — the number of leaf positions the search accumulates before sending them to the network in a single evaluation. It belongs, by every appearance, to the same family. Raising it gives the hardware more work per call and less waiting, and the gain is substantial: at 400 simulations per move on the production machine, the cost per ply falls from 8.47 ms to 2.99 ms in self-play and from 17.66 ms to 2.98 ms in evaluation, which on the same projection would take a 100-cycle run from 43 hours to under 16. It does not, however, belong to that family at all.

The Cost in Playing Strength

Batching leaves changes the search itself: to collect several distinct leaves in one pass, the search must be discouraged from re-entering the branch it has just chosen, and the resulting exploration is not the one that sequential selection would produce. With 400 simulations per move, a batch of 8 gives the tree 50 opportunities to react to its own results; a batch of 64 gives it 6.

The effect was measured by playing Run A's champion against itself, with the two sides differing in nothing but this parameter. Each comparison was played over 48 games, with 400 simulations per move on both sides and diversified openings. In every row the finer of the two searches is the reference, so the Elo figure is always the handicap carried by the coarser one carried by the coarser one (Table 8.16)

Against 32 and 64 leaves the finer search does not lose a single game in 96: 62 wins, 34 draws. The disadvantage of 64 leaves is close to 300 Elo points — more than four times what doubling the length of a run buys, and comparable to the entire improvement the

Comparison	Score	Elo cost	95% interval	W-D-L
4 against 8	0.531	-22	$[-86, +41]$	11-29-8
8 against 16	0.719	-163	$[-253, -90]$	25-19-4
8 against 32	0.802	-243	$[-333, -175]$	29-19-0
8 against 64	0.844	-293	$[-402, -217]$	33-15-0

Table 8.16: One network played against itself, the two sides differing only in the batch size, at 400 simulations per move and 48 games per row. Scores and records are from the perspective of the finer search, so the Elo column is the disadvantage carried by the coarser one throughout.

system achieved over the best model of the previous configuration.

The first row is as informative as the others. Halving the batch from 8 to 4 changes nothing measurable: the point estimate favours the finer search by 22 Elo, with a confidence interval spanning $[-86, +41]$ that contains zero comfortably. The cost is therefore not a smooth function of the setting: it is negligible below the value in use and steep above it, so that the value inherited from the performance tuning happens to sit at the knee of the curve. That it does is fortunate rather than designed — nobody had checked.

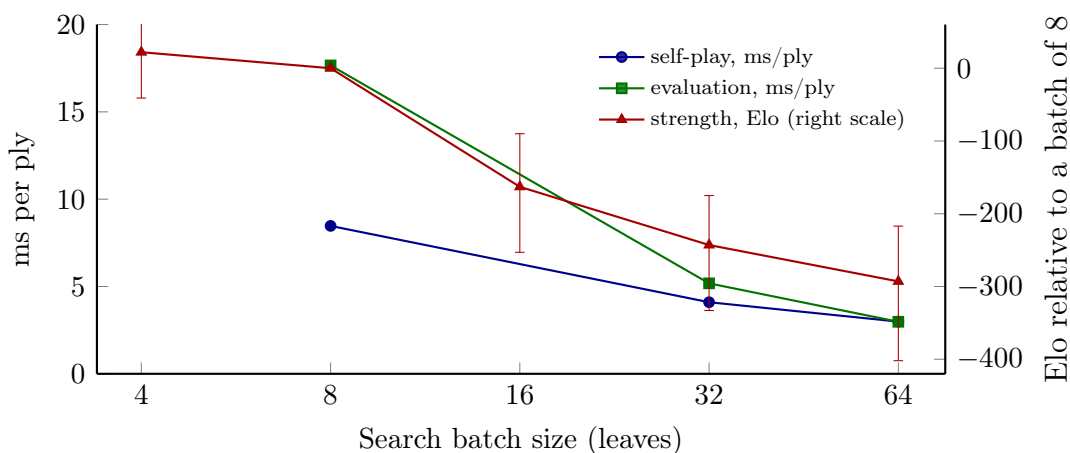


Figure 8.5: Search batch size against time per ply, measured on the production hardware, and against playing strength relative to a batch of 8. Error bars are 95% intervals. The speed series has no point at 4 and 16 leaves, because those values were never measured on the production machine.

The two axes of Figure 8.5 are not of equal standing, and the asymmetry is the practical point. The strength cost is measured, on 96 games with a result that never once goes the other way. The time saving is projected, by a bench known to overestimate. Trading a measured 300 Elo for a projected 27 hours is a worse bargain than the figure suggests.

The Blind Spot of a Symmetric Match

The pipeline runs a match before every promotion, expressly to detect a candidate that plays worse. It could not have detected this defect at any sample size, and the reason is structural rather than accidental: both the candidate and the champion play under the same batch size, so the handicap applies equally to both sides, cancels exactly, and the match returns parity.

The same holds for every measurement in the pipeline that compares two of the system’s own models, and for the fixed-depth measurements as well, since those too use the configured value on the learning side throughout. An instrument that applies a setting uniformly to everything it measures cannot see the effect of that setting. Detecting it required constructing a comparison that is deliberately asymmetric in exactly the respect under suspicion — the third requirement of the protocol in Section 8.3, which exists because of this case.

Branching Factor and the Cost of Batching

The magnitude raises the question the rest of this thesis asks: whether this is a property of the architecture or of the game it was applied to. The relevant measurement is of the game itself. Over 6 500 positions drawn from games played by a trained network, Italian draughts offers a mean of 5.84 legal moves per position, with a median of 6 and a maximum of 16. Only 9.5% of positions are forced, which is fewer than the mandatory-capture rule might suggest (Figure 8.6).

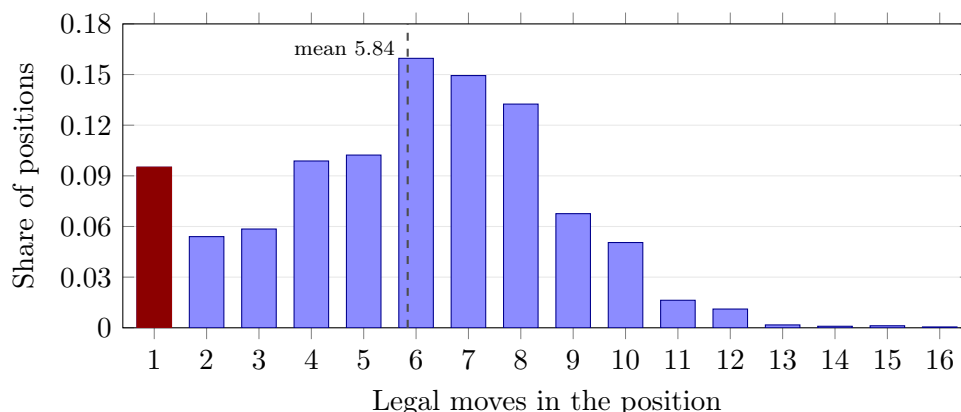


Figure 8.6: Distribution of the number of legal moves per position, over 6 498 positions from games played by a trained network. The highlighted column is the 9.5% of positions in which only one move is legal. The corresponding mean for chess, about 35, lies beyond the right edge of the axis.

A branching factor near 6 is what makes batching so destructive here. Collecting 64 leaves in a tree where each node has roughly 6 children forces the search to spread across essentially the whole reachable region rather than concentrating on the promising part of it — which is the entire purpose of the method. Chess, with a branching factor around 35 [2], absorbs the same batch far more comfortably: 64 leaves is a much smaller fraction of what the tree can offer.

This is an inference, not a measurement. No chess run was performed, so the claim that the same parameter would be nearly harmless there rests on the structure of the two games rather than on data from both. It is stated here because it is testable and because it points at something general: the cost of batching a tree search should be expected to scale inversely with the branching factor of the game, and a setting transferred unexamined from a wide game to a narrow one may carry a penalty its origin did not have.

8.7. Structural Limits of the Pipeline

The system works: it learns from nothing, it improves, and it reaches a level this thesis has measured against three independent references. This section reports what stops it from going further. Two limitations are established by the data of both runs; both are structural rather than incidental, and neither can be removed by running longer.

The Promotion Criterion

A candidate is promoted when it beats the current champion in a sequential test capped at 400 games. The criterion is the least examined part of the pipeline and, on the evidence collected here, the part that most constrains it. Figure 8.7 plots every promotion match of both runs against parity.

	Run A	Run B
Mean score of the promotion match	0.546	0.528
Promotions	45 of 100	64 of 200
Mean score, promoted candidates	0.593	0.588
Mean score, rejected candidates	0.508	0.500

Table 8.17: What the promotion decision is taken on. Scores are from the candidate’s perspective; the last two rows separate them by the outcome they produced.

The decision is being taken, cycle after cycle, on differences of a few percentage points, and in Run B the average rejected candidate scores exactly parity. Each accepted step therefore carries very little evidence, and the criterion guarantees only that the candidate

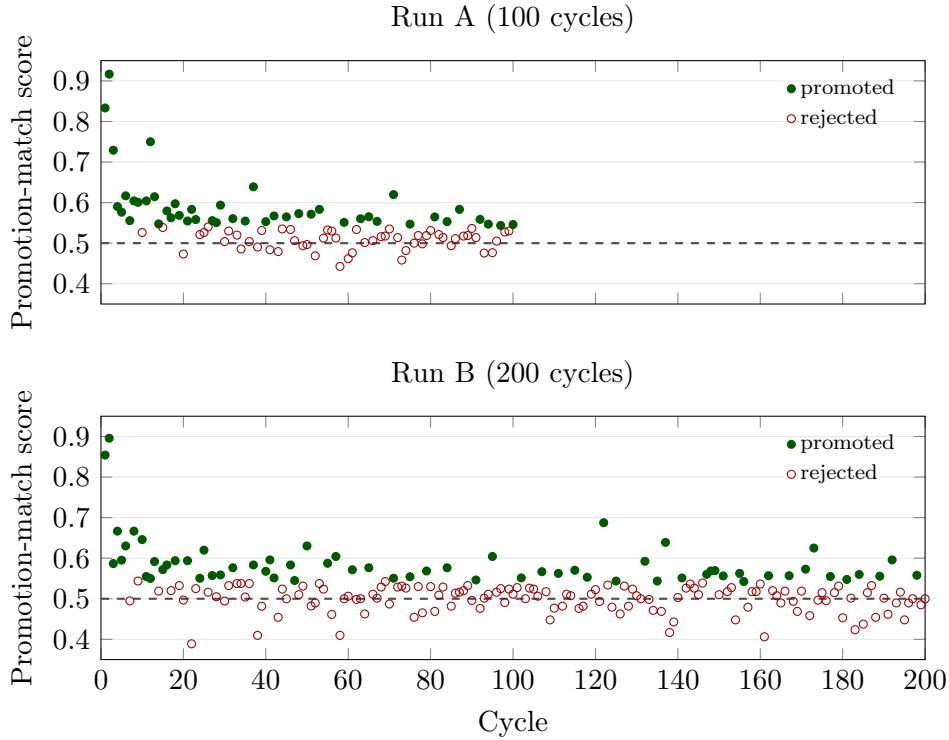


Figure 8.7: Score obtained by each candidate against the reigning champion, cycle by cycle; the dashed line is parity. Both panels share the same scales, so Run A occupies the left half of its own. The finding is the density immediately around that line: 89 of Run A’s 100 scores and 186 of Run B’s 200 fall between 0.4 and 0.6.

beats that champion — a reference that moves with the player it is meant to judge. As the models converge the criterion also accepts less: 45% of candidates in Run A against 32% in Run B.

The cost of extracting even that much is the dominant cost of the pipeline. The promotion match consumes 59% of Run A’s wall-clock time and 58% of Run B’s, against 30% for self-play and 3% for training. A large part of it decides nothing:

Promotion matches	Run A		Run B	
	cycles	promotions	cycles	promotions
Concluded before the cap	63	43 (68%)	125	59 (47%)
Reached the cap of 400 games	37	2 (5%)	75	5 (7%)

Table 8.18: Outcome of the promotion match, by whether the sequential test concluded on its own or exhausted its game budget.

In 37 cycles out of 100 in Run A, and 75 out of 200 in Run B, 400 games were played to establish that two models could not be told apart. That is 14 800 of Run A’s 27 712

promotion games and 30 000 of Run B's 54 288 — 53% and 55% of the most expensive phase of the cycle, spent on comparisons that do not conclude. What decides how much a match costs is not how late in the run it falls but how close it runs: across both runs the duration of a promotion match is strongly related to the distance of its final score from parity, at -0.59 in Run A and -0.62 in Run B. The quarter of matches whose scores ended furthest from parity took about 320 seconds each; the quarter that ended closest took about 850.

Three observations bear on the criterion, and they do not all carry equal weight in the two runs. Half the decision effort is spent without deciding, in both (Table 8.18); the promotions that do occur cluster just above parity, in both (Table 8.17); and the progression against frozen predecessors flattens at the middle of Run A, though not until four fifths of the way through Run B (Section 8.4). The third observation is therefore evidence from Run A and not from Run B, and it should not be presented as though the two agreed.

To these must be added the episode documented in Section 8.3, in which a candidate was promoted on a score of 0.590 while its strength against a fixed opponent fell significantly below that of its neighbours ($z = 2.7$). A criterion that compares a model only against its own immediate predecessor cannot detect that, because both of its readings measure the model against its own lineage.

The pipeline already contains a defence against this: a candidate approved by the promotion match can be required to also hold its own against a generation frozen some tens of cycles earlier, and the promotion is refused if it regresses against that fixed reference. A random walk frequently beats its own previous step; it does not keep beating a stationary one. The mechanism is implemented and verified, but it was left inactive for both runs reported here, because the two runs must differ in exactly one parameter for their comparison to mean anything. Testing it requires a dedicated third arm.

The Draw Rule as a Model Input

The second limitation concerns the rules themselves. The implementation adjudicates a draw after 80 consecutive half-moves without a capture or a promotion, with no condition on the material present. The federal regulation counts differently: it requires at least one king on each side, it counts the moves of one player rather than the half-moves of both, and — the difference that matters — it resets the count on any move of a man by either player, whereas the implementation resets only on a capture or a promotion.

The two criteria therefore diverge most in exactly the positions where men are still on the board, and the direction of the divergence is fixed by the rules rather than by the sample:

the implementation can only ever adjudicate earlier than the regulation would. Measured on 24 self-play games by each champion, on the same bench and with the same seeds, 12 of Run A's games and 15 of Run B's were closed by the implemented rule; of those, 6 and 5 respectively would not have been granted a draw under the regulation. In most of the divergent games the official count stood in the thirties, close to its own threshold; in one of Run B's it stood at 2, so that game was not remotely near a regulation draw.

The scope is larger than those proportions alone suggest. The no-progress limit closes 40% of Run A's self-play games and 42% of Run B's, and it accounts for some 98% of all the draws either run produced. Nearly every drawn game in nearly 18.5 million training positions of the two runs was ended by a house rule, and a measured share of those endings would not have been granted. That is the material the network learned its endgame behaviour from, and it is why the scores of Section 8.5 must be read as bands.

This cannot be repaired after the fact, and the obstacle is not in the code. The network receives, among its inputs, the no-progress counter divided by the threshold. The model has therefore learned with that denominator and plays accordingly: it knows when a game is about to be declared drawn. Replaying the recorded endgames under the official count would push that input past the only range it has ever seen, so the result would measure out-of-distribution behaviour rather than the rule. Adopting the correct rule changes the meaning of a model input and requires training from scratch — the cost of the correction is not the counting logic, which is trivial, but a complete run.

This is worth stating as a general observation, because it is not obvious: in this architecture an adjudication convention is not external to the game but an input to the model. Every such choice — the draw threshold, the ply limit, the material adjudication used in self-play — enters the weights and stays there.

The Experiment the Evidence Points To

Both runs reported in this chapter vary the duration and the schedule that follows from it, and Section 8.4 showed that doing so is not futile: 200 cycles produce a measurably stronger player than 100. But the return is small and the price is not. The additional 37 hours bought 67 Elo, while the promotion match consumed 58% of every one of those hours and spent more than half of that without reaching a decision.

The leverage is therefore not in the length of the run but in what each cycle is allowed to accept. The natural continuation is a run of the same length that changes the criterion — with the fixed-reference check active — and asks two questions the present data cannot answer: how many promotions it blocks, and whether the resulting champion is stronger,

measured by direct match against the champions of runs A and B. A criterion that refuses a candidate unable to hold its own against a stationary opponent would also, incidentally, cost nothing in the cycles where no promotion was going to happen anyway, which on the evidence of Table 8.18 is more than half of them.

8.8. Summary of Findings

The answers the preceding sections reached are collected here, so that the conclusions of the thesis can refer to them without re-deriving them. Table 8.19 states them section by section.

Table 8.19: The findings of this chapter, section by section.

Section	Finding
8.2 Evidence of learning	The agent learns from no prior knowledge: the untrained network loses every game to the trained champions, and both trained champions beat every fixed-depth opponent available.
8.3 The limits of relative measurement	Relative instruments are not sufficient. Four distinct failure modes were identified and a protocol was derived from them.
8.4 The value of additional cycles	Twice the compute helps, modestly: Run B's champion is +67 Elo over Run A's, less than half of what correcting the pipeline bought at a third of the compute.
8.5 Placement on an absolute scale	0.300 against a deliberately weakened reference engine, with no wins in 50 games — −147 Elo on the score as played, and as much as −241 once the house draw rule is discounted — and +205 Elo over the best model of the previous configuration.
8.6 Performance settings and playing strength	A setting chosen for performance alone was costing up to 293 Elo, and no symmetric measurement could have detected it.
8.7 Structural limits of the pipeline	The promotion criterion, and a draw rule that has been learned into the weights.

Established Findings

The pipeline learns, and the learning is real rather than an artefact of the instrument. The untrained network of cycle 0 loses all 24 games to either final champion, which is the negative control the rest of the series needs; by cycle 20 both runs already score 0.95 or more against the depth-4 opponent, and the final champions win 30 and 29 of 30 games against the deepest fixed-depth opponent the study could afford. Games lengthen as play improves, from 121 half-moves in the first cycle to 159 and 169 in the last, and the material record of those games shows advantages that are built and then converted rather than accumulated by accident.

Doubling the computation helps, and helps least of the three levers measured. Played directly against each other over 100 games, the 200-cycle champion beats the 100-cycle one by 67 Elo, with an interval excluding parity; the internal evidence agrees, since Run B keeps improving against its own lineage to four fifths of its length where Run A stops at half. Set against the 164 Elo that the corrections of Chapters 6 and 7 bought at a third of the compute, and the 293 Elo that a single mis-set performance parameter would have cost, additional computation is the least efficient of the three — and the only one whose price is paid in wall-clock hours.

The final model is the strongest the project has produced. Run B's champion is +205 Elo over the best model of the previous configuration, which was trained for 300 cycles. Against the external reference engine it does not win a single game, but it draws 30 of 50, where the earlier models drew 27, 19 and 4. The progress against an opponent of that strength lives entirely in the draws, and it is monotone across all four configurations.

Relative measurement is not sufficient, and the chapter shows where it fails. Two instruments that both compare the system to itself agreed with each other and were both wrong about the same model, at the same cycle, while the one instrument anchored to an unchanging opponent recorded a significant regression. Across 17 paired comparisons in the two runs, the pipeline's chained estimate overstates the direct measurement by up to a factor of 4, and reverses sign at the last snapshot of both runs; the overstatement is worst precisely where the true difference approaches zero, which is where one most wants to trust it. The external reference fails in the opposite way: given 50 games against each champion it could not separate the two, though a direct match over 100 games shows a real difference of 67 Elo. Each of these errors follows from a property of the instrument, and the protocol of Section 8.3 was derived from those properties rather than from the individual failures.

A performance setting was determining playing strength. The number of positions gath-

ered before each network evaluation — chosen to keep the hardware busy, and never suspected of anything else — costs 163, 243 and 293 Elo at successively larger values, while leaving no trace in any measurement the pipeline performs on itself. The reason is structural: every one of those measurements applies the setting equally to both sides of the comparison. The effect is amplified by a property of this particular game, whose mean of 5.84 legal moves per position is roughly a sixth of the figure for chess.

Boundaries of the Findings

Four claims that the data of this chapter do not support are worth stating explicitly, since each is close enough to a claim that it does support to be mistaken for one.

The comparison between the two runs answers a practical question — whether a longer run is worth commissioning — and not a scientific one. The number of cycles also governs the decay of the learning rate, so the two runs differ from their first cycle onward in more than the quantity under study.

Each configuration was run once. Both champions are single samples of a stochastic process, and nothing in this chapter separates the 67 Elo between them from the variation two runs of identical configuration might show. Only a repetition could, and none was performed.

The inference that the batching penalty would be mild in chess rests on the branching factors of the two games, not on a chess run. It is offered as a testable prediction.

Finally, the scores against the external engine are bands rather than points, because a measured share of the games were adjudicated drawn earlier than the federal regulation would allow. The direction of that bias is fixed by the rules and is always the same, but its magnitude on any particular set of games is estimated rather than known.

Taken together, the measurements describe a pipeline that learns strategic play from nothing and reaches a defensible level. More computation does improve it, but at a rate that two of the chapter's other findings dwarf: the same measurements that establish the gain from doubling a run also establish that correcting the machinery bought more, and that one unexamined performance setting could have cost four times as much. The ceiling is set less by the quantity of computation than by what each cycle is permitted to accept, and that distinction is the principal contribution of this chapter. It is where the conclusions of the thesis begin.

9 | Conclusions and Future Developments

9.1. Design and Experimental Scope

The deliverable of this work is one design, defined once and instantiated twice, together with the trained models it leaves behind. An environment interface specifies what a two-player, deterministic, perfect-information board game must provide: the legal moves in a position, the position that results from applying one of them, and whether a position is terminal and with what result. Everything above that interface — the search, the encoder, the policy–value network, the inference boundary, the training loop and the conductor — follows one specification, and each is realised twice, once for chess and once for Italian draughts. Chapters 1 to 7 are organised to keep the two apart: the shared design first, then each realisation in turn.

The draughts pipeline was trained to completion twice, over runs of 100 and 200 cycles, together comprising 202,000 self-play and promotion games and 18.5 million training positions: draughts is the narrower of the two games, and its scale allows full runs to be completed and repeated. The chess pipeline is complete and functional and stands as the structural counterpart.

That asymmetry bounds what this chapter can claim, and Section 9.5 states the bounds. It does not prevent the thesis from answering the question it set out to ask, because that question is answered chiefly by the design decisions the two implementations forced, and only in part by outcomes.

Both implementations are publicly available, in two repositories: PlazChess for chess and PlazDraughts for Italian draughts.¹ The draughts repository also carries the measurement scripts, the data behind the figures of Chapter 8, and a platform on which the trained models can be played against: the champion of the earlier 300-cycle configuration, Run A

¹PlazChess at <https://github.com/LeonardoPlazzogna/PlazChess>
 PlazDraughts at <https://github.com/LeonardoPlazzogna/PlazDraughts>.

at cycle 50 and at its end, and Run B at cycles 100, 150 and 200, together with an untrained network. The two final champions and the 300-cycle model are the one compared directly in Section 8.4.

9.2. General and Game-Specific Components

The introduction asked which parts of the learning process are general and which are specific to the game they were trained on. The material for answering it is distributed through Chapters 1 to 7, in the division every one of them observes: a section describing what both engines share, followed by two sections describing what each does differently and why. This section collects that division into a single statement.

The Ledger

Table 9.1 groups the elements by family rather than listing them individually, and groups the rewritten ones by the reason the technical chapters give for the divergence. The evidence column distinguishes three kinds of claim: $\blacklozenge$ marks a design decision documented and motivated in the technical chapters, $\blacktriangle$ an effect established by measurement in draughts, and $\circ$ an asymmetric observation that is suggestive but not controlled.

Table 9.1: What transferred between the two games and what had to be rewritten, with the rewritten elements grouped by the reason the technical chapters give for the divergence. The last block holds the three elements whose classification did not follow from a design argument: two were measured in draughts, and the third rests on an observation that was never controlled. One divergence is absent from the table because the chapters record it without arguing it: the schedules applied to root noise and to the move-selection temperature, annealed by cycle in chess and fixed in draughts, and a ramp against a step within a game.

Element	Shared	Rewritten	Evidence
Selection rule, expansion, backup, move selection; the root-noise and temperature mechanisms	✓		$\blacklozenge$ Ch. 2
Network recipe: stem, constant-width trunk, initialisation, scaling	✓		$\blacklozenge$ Ch. 4

continued on the next page

Table 9.1, continued from the previous page

Element	Shared	Rewritten	Evidence
Encoder contract: fixed shape, static snapshot, per-square exclusivity	✓		♦ Ch. 3
The inference boundary: index mapping, legality masking, perspective	✓		♦ Ch. 5
Training target: the search’s visit distribution	✓		♦ Ch. 6
The promotion gate, and the cycle it closes	✓		♦ Ch. 7
<i>Rewritten, adapted to the size of the game</i>			
State representation, reversible transitions, hashing, caching		✓	♦ Ch. 1
Subtree reuse, progressive widening, early stopping, mate-in-one		✓	♦ Ch. 2
Optional channel recalibration in the residual block		✓	♦ Ch. 4
Reduced precision, shared cross-process cache		✓	♦ Ch. 5
Data persistence; a separate evaluation phase; logging		✓	♦ Ch. 6–7
<i>Rewritten, adapted to the rules of the game</i>			
Absolute frame against canonicalisation; plane count		✓	♦ Ch. 3
Factored against flat policy head		✓	♦ Ch. 4
Scalar against three-class value head, and its loss		✓	♦ Ch. 4, 6
Opening book, resignation, draw handling		✓	♦ Ch. 7
<i>Rewritten, chosen for continuity with earlier runs</i>			
Optimiser and its schedule		✓	♦ Ch. 6
<i>Established by evidence rather than by a design argument</i>			
Initial curriculum scaffolding		✓	○
Exponential moving average of weights		✓	▲ §6.3

continued on the next page

Table 9.1, continued from the previous page

Element	Shared	Rewritten	Evidence
Batching of leaf evaluations		✓	▲ §8.6

The Reasons for the Rewrites

The rewritten block is long and, at first sight, heterogeneous: bitboards, reversible transitions, incremental hashing, progressive widening, subtree reuse, early stopping, a flat policy head, an opening book, sparse storage of policy targets. It becomes orderly once the rows are grouped by the reason each chapter gives, which is what the table does. Most are adapted to the size of the game, a smaller group to its rules, and one to nothing but continuity with runs already completed.

Not one of them replaces the method by which the system learns. The search still improves on the network’s own policy and that improvement is still the training target; the outcome of the game is still the value target; a candidate still replaces the champion only by winning a match against it. What differs is how those elements are parameterised for a game offering fewer than 6 legal moves per position instead of about 35, and for rules that make a capture compulsory and a draw common.

The learning method transferred from one game to the other intact. What was rewritten was adapted to the new game — most of it to the game’s size, some of it to its rules — and none of the rewrites required a different way of learning.

The thesis does not need to argue this: it has demonstrated it, chapter by chapter, in the reason given for every divergence, and those reasons are of two kinds. Some are about size. A compiled bit-level representation would not have justified the added complexity; a widening schedule has nothing to withhold where a node has 6 children. Others are about rules. Draughts men move in only one direction, which is what makes canonicalisation worth its cost; and there is nothing analogous to factor once the origin, direction and mode of a move are fixed, because the destination then follows from the rules. The single row explained by neither is the optimiser, and Chapter 6 says so plainly: Adam remains the default in draughts so as not to change, silently, the behaviour of runs already completed under it.

The Three Exceptions

Each of the three was placed in the rewritten column by evidence rather than by argument, and each by evidence of a different kind.

Initial scaffolding. The chess engine required decaying auxiliary penalties to bridge the cold start; the draughts engine converged reliably from tabula rasa on a policy and value loss alone, with no reward shaping of any kind. That is consistent with the requirement being a property of chess’s branching factor and search depth under limited hardware rather than of the architecture, but it is an asymmetric observation and not a controlled experiment — no chess run without scaffolding was performed — and it is marked accordingly.

Parameter averaging. An exponential moving average of the weights is standard practice, and the chess loop uses it. In draughts it was measured as harmful: across 6 cycles at identical configuration and identical seeds, 5 of 6 candidates were promoted with averaging disabled against 1 of 6 with it enabled, and the training loss stalled in the second case. The mechanism is explained in Section 6.3 and follows from a property of the regime rather than of the game: each cycle is a short fine-tune that moves directionally away from the champion’s weights, and averaging a directional trajectory means lagging behind it, so the candidate emerges weaker than the model it was meant to improve on. Averaging helps when a trajectory oscillates around a minimum; it does not help when the trajectory is still advancing.

Batching of leaf evaluations. The third exception is the most consequential, because unlike the other two it was not merely inherited differently — it was misclassified. The number of positions gathered before each network evaluation was chosen as a performance setting, on the reasonable assumption that it could not affect how the system plays. Section 8.6 shows that it costs 163, 243 and 293 Elo² at successively larger values; at the largest of them the finer search won 33 of 48 games and lost none.

The Misplaced Boundary

Taken together, the three refine the claim rather than contradicting it. The boundary between what is architectural and what belongs to the game is real, and most of it lies where a designer would expect. It does not lie exactly there, and two of the three exceptions show the ways it can be misplaced. A mechanism can be believed general and turn out to depend on the regime, as parameter averaging did. And a mechanism can be believed irrelevant to learning altogether, filed among the performance settings, and turn out to be the most consequential value in the configuration. The third exception is of a different kind: a mechanism believed necessary in general that proved necessary only in

²An Elo difference is a re-expression of the score of a match between two players: 100 points correspond to a score of about 0.64, 200 to 0.76, and 300 to 0.85. Every figure quoted in this chapter is the conversion of a match record reported in Chapter 8.

the wider of the two games.

Both errors are invisible to a measurement that applies the mechanism uniformly. Finding the two that were measured required, in one case a controlled comparison at matched seeds and in the other a deliberately asymmetric one. That is not a coincidence, and it is the subject of the next section.

9.3. Strength Measurement in a Self-Referential System

A system that learns by playing against itself certifies its own progress, and the most transferable result of this work concerns what that certification is worth. Every quantity the system computes about itself — the promotion score, the internal rating, the training loss — is measured against a reference that moves along with the thing being measured. This is not a defect to be repaired; it is what makes self-play possible. But it has a consequence, and Chapter 8 documents four distinct ways in which a strength measurement in such a system can appear to work while conveying nothing: a reference too weak to discriminate, a reference too strong to discriminate, a rating propagated along a chain of relative comparisons, and a defect applied identically to both sides of a comparison.

Two properties of that catalogue are worth stating. The first is that each of the four was encountered rather than anticipated: the list is a record of failures diagnosed after the fact, which is why it is short and why each entry has a mechanism attached rather than merely a name. The second is that nothing in it is specific to draughts, to chess, or to board games. Any system that generates its own training signal and certifies its own improvement is exposed to all four.

The protocol derived from them is correspondingly general: compare models directly rather than through intermediaries; choose references near the level being measured; make a comparison asymmetric when testing a setting both sides share; and keep one fixed external reference regardless, not to measure progress but to answer where the system stands.

Chapter 8 also supplies the evidence that these are not hypothetical concerns. On the very pairs the internal ladder estimates, playing the two models against each other gives a different answer: the chain overstates by up to a factor of 4, and at the last snapshot of both runs it reverses the sign, reporting a gain where the direct match finds a loss. The external reference fails in the opposite way. Given 50 games against each champion it could not separate them, though a direct match over 100 established a difference of 67

Elo — 36 wins against 17, with 47 draws.

Against these stands the positive result of the set. Run A is 164 Elo above the best model of the project's earlier configuration and Run B is 67 above Run A, which predicts 231 for Run B over that model; the direct measurement gives 205. The discrepancy of 26 Elo is smaller than the confidence interval of each of the three matches. Direct comparisons compose; chains of them do not, and the difference between the two cases is not which models are compared but whether the comparison passes through intermediate links.

9.4. Results on Italian Draughts

Chapter 8 reports the measurements in full; this section states only what they amount to. The system learns from no prior knowledge. The untrained network loses every game to either final champion, and by cycle 20 both runs already score 0.95 or more against the shallowest fixed-depth opponent. Both final champions beat every fixed-depth opponent the study could afford: against the deepest of them they take 30 of 30 games and 29 of 30.

The stronger of the two champions never beats a deliberately weakened version of a mature program, but draws 30 of 50 games against it. That places it 147 Elo below on the score as played, and as much as 241 below once the draws owed to the implemented no-progress rule are discounted. Against the best model of the project's previous configuration it is ahead by 205 Elo — 54 wins against 1 — and that model was trained for 300 cycles against its own 200.

Doubling the length of a run produces a measurably better player: 67 Elo, or 36 wins against 17 with 47 draws over 100 games. That gain is real and it is small — less than half of what correcting the measurement and training machinery bought at a third of the computation, and less than a quarter of what a single mis-set performance parameter would have cost.

The behavioural evidence is consistent with a player that learned to play soundly. Games lengthen from 121 half-moves in the first cycle to 159 and 169 in the last, which is what one expects as the blunders that end a game early become rarer. The proportion of drawn games collapses and then recovers, in a pattern that admits a natural reading: the network learns to convert an advantage before it learns to prevent one. Against an opponent within its reach the agent builds a material advantage in every one of 30 games and converts all but one of them; against one beyond its reach it never holds one for any length of time, and what improved across the four configurations is its capacity to survive

rather than to win.

9.5. Limitations

Six limitations bound the conclusions above, set out here in order of how far each qualifies what precedes it.

The reach of the ledger. Most rows of it rest on design evidence: decisions documented and motivated in the technical chapters, with two supported by measurement in draughts and one by an observation that was not controlled. That establishes what was rewritten and why it was rewritten; it does not establish that the same division would hold in a game carried to convergence. A chess run would move several rows from the first kind of evidence to the second, and could in principle move one of them the other way.

Behaviour as evidence. The title of this thesis speaks of strategic reasoning, and the evidence offered for its emergence is behavioural: scores against fixed opponents, the shape of the games, material balance in quiet positions. That is the evidence this literature normally offers, and it is what the measurements of Chapter 8 support. What the work does not contain is an analysis of the representations the network formed — no probe of what its planes encode, no test of whether a concept a player would name has a counterpart inside it. What is established here is that strategic play emerged, not that it is reasoned.

The cycle count and the learning rate. The two runs were configured to differ in a single parameter, the number of cycles, but that parameter also sets the length of the learning-rate cosine: Run B's learning rate decays more slowly from its first cycle, and at cycle 100 it is still training at 5.5 times Run A's rate. The 67 Elo between the champions therefore answers a practical question — whether a 200-cycle run is worth commissioning in preference to a 100-cycle one — and not the scientific one, whether additional training helps with everything else held fixed. Separating them would take a third run with the cycle count raised and the schedule held to the shorter one.

One run per configuration. Both champions are single samples of a stochastic process, and nothing here separates the 67 Elo between them from the variation that two runs of identical configuration might show. The same holds for every comparison that sets one whole run against another. What is controlled is the comparison made inside a run: the frozen generations are played against their own champion at the search budget of the run itself, parameter averaging was measured at identical configuration and identical seeds, and the batching parameter was measured on one network played against itself.

The draw rule as a model input. The implemented no-progress limit adjudicates earlier than the federal one, always in that direction, and the threshold is not an external convention but one of the network’s inputs. It closes some 40% of self-play games and accounts for nearly every draw either run produced, and it cannot be corrected without training again from the start. Its effect was measured rather than left unknown: the scores against the external engine are reported as bands, and those bands are the form in which this limitation already enters the results.

The promotion criterion. Chapter 8 identifies it as the principal constraint on the pipeline, and neither run varied it. The defence the pipeline already implements — refusing a candidate that regresses against a frozen earlier generation — was left inactive so that the two runs would differ in one parameter only. The constraint is therefore established and its remedy untested; Section 9.6 states what testing it would take.

9.6. Future Work

Three continuations follow from specific results rather than from general interest, and they are ordered by how much each would change what is known.

A third arm that changes the criterion rather than the duration. Neither run could have carried this out. The defence described in Section 9.5 — refusing a promotion whose candidate regresses against a frozen earlier generation — had to stay inactive in both, because switching it on in one of them would have made the two runs differ in two parameters and dissolved the comparison of Section 8.4; and that the criterion is the binding constraint is itself a finding of that comparison. Section 8.7 establishes the cost: the promotion match consumes 58 to 59% of the wall-clock time of a run and spends more than half of that without reaching a decision, while the promotions it does grant cluster immediately above parity. A run of the same length with the check active would answer two questions the present data cannot: how many promotions it blocks, and whether the champion it produces beats the champions of Runs A and B in a direct match.

The chess run, as a test rather than a completion. It was out of reach here for the reason given in Section 9.1, and its value has changed in the course of the work. It is no longer merely the missing half of the project: Section 8.6 makes a falsifiable prediction — that the cost of batching a tree search scales inversely with the branching factor of the game, so that a setting costly here would be nearly harmless in chess — and a chess run is the experiment that would refute or confirm it. Carried out twice, once with the initial scaffolding and once without, it would also settle the one row of Table 9.1 that rests on an uncontrolled observation.

Retraining under the federal draw rule. The divergence between the implemented rule and the federal one is itself a result of this work, and it was quantified rather than repaired because the threshold is one of the network’s inputs: correcting it costs a complete run and not a change to the counting logic. It is listed third not because it matters least but because the first two would change what is known, whereas this one would replace the bands of Section 8.5 with point estimates and let the endgames be learned under the rule that actually governs the game.

Two of the limitations in Section 9.5 have no continuation proposed here: the variability between runs of identical configuration, which would need repetitions rather than new arms, and the analysis of the representations the network formed, which belongs to a different kind of study.

9.7. Concluding Remarks

This thesis set out to build a self-learning system twice, on two games chosen to diverge exactly where an implementation is most sensitive, and to observe which parts of the process survived the transfer. Both instances are built to learn by playing against themselves, and the draughts one was carried far enough to be measured against opponents outside its own lineage.

They divided cleanly, though not exactly where one would have drawn the line in advance. The method — search-improved targets, training toward them, and a gate that admits a new champion only on evidence — moved from a game of about 35 legal moves per position to one of fewer than 6 unchanged. The engineering did not: almost nothing written to make chess fast is present in the draughts engine, and where the reason is not that the problem is smaller, it is that its rules are different.

The boundary between those two categories is where the finding of this work lies, and it does not fall where the designer drawing it assumes. One setting on the wrong side of it cost 293 Elo — against an identical copy of itself searching with the value in use, the network did not win a single game in 48 — and it was invisible to every instrument the system possessed, because the system applied it uniformly to everything it measured. Establishing that required a measurement built to be asymmetric in exactly the respect under suspicion: a general lesson about self-referential systems, and the one result here that would transfer to a system playing no game at all.

Bibliography

- [1] Claude E Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.
- [2] Louis Victor Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, University of Limburg, Maastricht, The Netherlands, 1994.
- [3] Federazione Italiana Dama. Regolamento tecnico del gioco della dama italiana. http://www.fid.it/regolamenti/2008/RegTec_CAP0_I.pdf. Aggiornamento 2008.
- [4] The Stockfish developers. Stockfish: A free and strong UCI chess engine. <https://stockfishchess.org>. Version 8.
- [5] David J Slate and Lawrence R Atkin. CHESS 4.5 — the Northwestern University chess program. In *Chess Skill in Man and Machine*, pages 82–118. Springer, 1977.
- [6] Donald E Knuth and Ronald W Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.
- [7] Albert L Zobrist. A new hashing method with application for game playing. Technical Report 88, Computer Sciences Department, University of Wisconsin, Madison, WI, 1970.
- [8] Murray Campbell, A Joseph Hoane, and Feng-hsiung Hsu. Deep Blue. *Artificial Intelligence*, 134(1-2):57–83, 2002.
- [9] Yu Nasu. Efficiently updatable neural-network-based evaluation functions for computer shogi. https://www.apply.computer-shogi.org/wcsc28/appeal/the_end_of_genesis_T.N.K.evolution_turbo_type_D/nnue.pdf, 2018. Appeal document, 28th World Computer Shogi Championship.
- [10] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

- [11] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT Press, second edition, 2018.
- [12] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.
- [13] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [14] Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.
- [15] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [16] Arthur L Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.
- [17] Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.
- [18] Arpad E Elo. *The rating of chessplayers, past and present*. Arco Pub., 1978.
- [19] Jonathan Schaeffer, Robert Lake, Paul Lu, and Martin Bryant. CHINOOK: The world man-machine checkers champion. *AI Magazine*, 17(1):21–29, 1996.
- [20] Jonathan Schaeffer. *One Jump Ahead: Challenging Human Supremacy in Checkers*. Springer, 1997.
- [21] Jonathan Schaeffer, Neil Burch, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. Checkers is solved. *Science*, 317(5844):1518–1522, 2007.
- [22] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [23] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel,

- et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [25] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games (CG 2006)*, volume 4630 of *Lecture Notes in Computer Science*, pages 72–83. Springer, 2006.
- [26] The LCZero authors. Leela Chess Zero. <https://lczero.org>.
- [27] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.
- [28] Fédération Internationale des Échecs. FIDE laws of chess. <https://handbook.fide.com>. Edition 2023.
- [29] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [30] Guillaume M J-B Chaslot, Mark H M Winands, and H Jaap van den Herik. Parallel Monte-Carlo tree search. In *Computers and Games (CG 2008)*, volume 5131 of *Lecture Notes in Computer Science*, pages 60–71. Springer, 2008.
- [31] Guillaume M J-B Chaslot, Mark H M Winands, H Jaap van den Herik, Jos W H M Uiterwijk, and Bruno Bouzy. Progressive strategies for Monte-Carlo tree search. *New Mathematics and Natural Computation*, 4(3):343–357, 2008.
- [32] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [33] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [34] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network

- training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456. PMLR, 2015.
- [35] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7132–7141, 2018.
- [36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1026–1034, 2015.
- [37] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- [38] Peter J Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [39] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [40] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.
- [41] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [42] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [43] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [44] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- [45] Boris T Polyak and Anatoli B Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
- [46] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generaliza-

- tion. In *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence*, 2018.
- [47] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. BLAKE2: simpler, smaller, fast as MD5. In *International Conference on Applied Cryptography and Network Security*, pages 119–135. Springer, 2013.
- [48] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.
- [49] Ed Gilbert. KingsRow. <https://edgilbert.org/ItalianCheckers/KingsRowItalian.htm>. Version 1.19e.

A | Hyperparameters and Configuration Manifest

This appendix lists the exact configuration of both engines. The two are documented separately because they were run very differently: the draughts pipeline produced every result in Chapter 8, over the two complete runs analysed there, while the chess pipeline is complete and functional but was never taken to convergence, so its table records the configuration as built rather than one validated by a finished run.

Table A.1 is therefore the one Chapter 8 refers to. The two runs share every value in it except the cycle count, which is marked; as Section 8.4 explains, that single setting also determines the learning-rate schedule, so the runs differ in one setting but in two respects.

Table A.1: Configuration of the Italian draughts pipeline, for both runs.

Parameter	Value	Description
Pipeline & Orchestration		
DAMA_CYCLES	100 / 200	Self-play cycles. The only value that differs between Run A and Run B ; it also sets the length of the learning-rate decay.
DAMA_GAMES	400	Self-play games generated per cycle.
DAMA_BUFFER_SAMPLES	600,000	Replay window, in positions — about 10 cycles' worth.
<i>ply cap</i>	300	Hard cap on self-play game length; truncated games are scored by material balance rather than as draws.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
Network & Training		
DAMA_CHANNELS	96	Width of the residual trunk.
DAMA_BLOCKS	6	Residual blocks; 1.13 M parameters in total.
DAMA_USE_SE	on	Squeeze-and-excitation recalibration inside each block.
DAMA_EPOCHS	2	Passes over the replay window per cycle.
DAMA_BATCH	1024	Samples per forward/backward pass.
DAMA_LR	1×10^{-3}	Base learning rate, Adam.
DAMA_LR_SCHEDULE	on	Cosine decay across the whole run, in addition to the cosine internal to each training call.
DAMA_LR_FINAL	0.10	Final learning rate as a fraction of the base, reached at the last cycle.
DAMA_EMA	off	Exponential moving average of weights, disabled: measured as harmful here, since each cycle is a short directional fine-tune and averaging it leaves the candidate behind the champion.
DAMA_VALUE_DISCOUNT	0.99	Per-ply discount applied to the value target.
Search & Exploration		
DAMA_SIMS	400	Simulations per move, in self-play and in the promotion match alike.
DAMA_C_PUCT	1.5	Exploration constant.
DAMA_MCTS_BATCH	8	Leaf positions gathered before each network evaluation. Not a performance setting: see Section 8.6.
<i>Dirichlet α</i>	1.5	Root noise concentration during self-play; fixed for the whole run rather than annealed.
<i>Dirichlet ε</i>	0.25	Root noise mixing weight.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
<i>self-play temperature</i>	12	Opening plies sampled from the visit counts in self-play; played greedily afterwards.
DAMA_ARENA_TEMP_PLIES	10	Opening plies sampled from the visit counts, to diversify the promotion match.
Promotion Criterion		
DAMA_ARENA_GAMES	200	Nominal promotion-match length, before the sequential test.
DAMA_SPRT	on	Sequential probability ratio test, deciding promotion.
DAMA_SPRT_MAX_GAMES	400	Cap. Reached without a decision in 37 of Run A's cycles and 75 of Run B's (Section 8.7).
DAMA_SPRT_ALPHA	0.05	Type-I error rate.
DAMA_SPRT_BETA	0.05	Type-II error rate.
DAMA_PROMOTE_MIN	0.55	Score threshold, used when the sequential test is disabled.
DAMA_ANTIDRIFT	off	Fixed-reference gate: refuses a promotion whose candidate regresses against an older frozen generation. Implemented and verified, left inactive so that the two runs differ in one parameter only. Discussed in Section 8.7.
Periodic Measurement		
DAMA_STRENGTH_EVERY	20	Cycles between measurements against fixed-depth opponents.
DAMA_STRENGTH_GAMES	60	Games per fixed-depth measurement, at depths 4 and 6.
DAMA_AB_DEEP_DEPTH	8	Depth of the slower fixed opponent.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
DAMA_AB_DEEP_EVERY	60	Cycles between depth-8 measurements: 1 point in Run A, 3 in Run B. The rest of that column in Table 8.3 was filled afterwards by replaying the frozen generations.
DAMA_AB_DEEP_GAMES	20	Games per depth-8 measurement during the run.
DAMA_GEN_EVERY	10	Cycles between frozen champion snapshots. These are what Sections 8.3 and 8.4 replay.
DAMA_GEN_GAMES	100	Games per rung of the internal generation ladder.
DAMA_GEN_ANCHORS	3	Previous generations each new one is played against.

Adjudication

<i>no-progress draw</i>	80	Half-moves without a capture or a promotion, after which a draw is declared. Diverges from the federal rule and is an input to the network: see Section 8.7.
<i>resignation</i>	none	No resignation and no quickdraw logic; every self-play game runs to a terminal state or to the ply cap.
<i>opening book</i>	none	Diversity comes from root noise and temperature sampling alone.

Execution

DAMA_CPP_THREADS	24	Parallel workers on the production machine, three quarters of its logical cores. Verified not to affect the samples produced.
DAMA_SERVER_BATCH	256	Maximum inference batch assembled across concurrent games.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
DAMA_DEVICE	cuda	Device for training, search and evaluation run through the fast execution path.

The chess configuration follows. It was arrived at through the same calibration process, but the values below were never exercised by a run carried to convergence, so they should be read as the intended configuration rather than as a validated one. Where the two tables differ — the opening book, the resignation logic, the branching-factor limits, the factored policy targets — the differences are those Chapters 1 to 7 describe and motivate.

Table A.2: Configuration of the chess pipeline, as built.

Parameter	Value	Description
Pipeline & Orchestration		
CYCLES	1000	Total number of AlphaZero self-play iterations.
EXPLORE_GAMES	1000	Number of self-play games generated per cycle.
ARENA_GAMES	96	Number of matches to evaluate the candidate network.
MAX_PLIES	200	Hard cap on game length to prevent infinite loops.
REPLAY_K	6	Number of previous cycles retained in the training buffer.
MAX_SAMPLES_PER_GAME	180	Maximum states kept per game, prioritising informative plies.
Network Training		
EPOCHS	1	Passes over the dataset per training phase.
BATCH_SIZE	4096	Number of samples per training forward/backward pass.
LR	1×10^{-3}	Base learning rate for the AdamW optimiser.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
WEIGHT_DECAY	1×10^{-4}	L_2 regularisation penalty.
VALUE_LOSS_WEIGHT	2.0	Scaling factor applied to the SmoothL1 (Huber) value loss.
GRAD_CLIP_NORM	1.0	Maximum gradient norm for clipping.
WARMUP_FRAC	0.08	Fraction of steps used for the linear learning rate warmup.
EMA_DECAY	0.999	Decay rate for the exponential moving average of weights.
POLICY_SMOOTH_EPS	0.03	Label smoothing epsilon applied over legal moves.
MCTS & Exploration		
EXPLORE_SIMULATIONS	600	MCTS simulations per move during data generation.
ARENA_SIMULATIONS	600	MCTS simulations per move during candidate evaluation.
C_PUCT	1.25	Upper Confidence Bound exploration constant.
EXPLORE_TOPK	24	Branching factor limit during exploration MCTS.
ARENA_TOPK	16	Branching factor limit during arena MCTS.
DIRI_ALPHA_START	0.30	Initial Dirichlet noise parameter (decaying to 0.10).
DIRI_EPS_START	0.25	Initial Dirichlet mix-in fraction (decaying to 0.10).
USE_OPENING_BOOK	True	Enables hard-coded opening lines for early diversity.
BOOK_PROB	0.60	Probability of playing a book move during the first 12 plies.
Adjudication & Early Stopping		
RESIGN_AFTER_PLIES	24	Minimum game length before resignation is permitted.

Continued on next page

(Continued from previous page)

Parameter	Value	Description
RESIGN_VALUE	−0.90	Value threshold below which the engine considers the position lost.
RESIGN_CONSEC_PLY	8	Number of consecutive plies below threshold required to resign.

List of Figures

1.1	The chess board model	10
1.2	Move representation in the chess environment	11
1.3	One-move lifecycle in the chess environment	13
1.4	Reversible transitions in the chess environment	15
1.5	Move generation in Italian draughts	17
2.1	Search flow, chess	24
2.2	Search flow, draughts	26
3.1	Encoding of an example chess position	32
4.1	Policy-value network, chess	39
4.2	Policy-value network, draughts	42
5.1	Inference path, both engines	46
5.2	Move indexing, chess	47
5.3	Move indexing, draughts	49
6.1	Training flow, chess	52
6.2	Training flow, draughts	55
6.3	Learning-rate schedules in draughts	57
7.1	The self-play loop in chess	62
7.2	Exploration schedules in chess	64
7.3	Temperature schedules, chess and draughts	67
8.1	Training loss per cycle in draughts	74
8.2	Self-play game length and draw rate in draughts	76
8.3	Chained ladder against direct matches, both runs	81
8.4	Peak material advantage per game in quiet positions	89
8.5	Speed and strength against search batch size	93
8.6	Legal moves per position in draughts	94
8.7	Promotion-match scores per cycle, both runs	96

List of Tables

3.1	Input planes, chess	31
3.2	Input planes, draughts	34
4.1	Network heads, chess	40
4.2	Network heads, draughts	42
6.1	Parameter averaging in draughts	58
8.1	The two runs in draughts	71
8.2	Configuration shared by the two draughts runs	72
8.3	Score against fixed-depth opponents in draughts	73
8.4	Four modes of failure for strength measurement	78
8.5	Relative and fixed measures of one model in draughts	79
8.6	Chained ladder against direct matches, Run A	80
8.7	Chained ladder against direct matches, Run B	80
8.8	Final champion against its own snapshots, Run A	83
8.9	Final champion against its own snapshots, Run B	83
8.10	Run A against Run B, final champions	84
8.11	Effect of cycles, corrections and batch size in Elo	85
8.12	Both champions against an earlier 300-cycle model	85
8.13	Both champions against KingsRow	88
8.14	Material balance in quiet positions against KingsRow	89
8.15	Effect of the draw rule in draughts	90
8.16	Cost of search batching in draughts	93
8.17	Summary of promotion-match scores, both runs	95
8.18	Promotion outcomes by how the test ended, both runs	96
8.19	Summary of the results in draughts	99
9.1	Components shared and rewritten between chess and draughts	104
A.1	Configuration of the draughts pipeline	119
A.2	Configuration of the chess pipeline	123

Acknowledgements

First and foremost, I would like to thank my supervisor, Prof. Marcello Restelli, for the opportunity to develop this thesis project. Heartfelt thanks goes to Danieli Automation for providing the computational resources necessary to carry out the simulations. Within the company, I wish to express my most sincere gratitude to Marco Fabiani: his constant availability, invaluable technical support, and the genuine interest he showed in my research have been a fundamental source of encouragement, making him an excellent point of reference throughout this journey.

As I conclude my Bachelor's degree and after an entire year dedicated to this work, I would like to spare a thought for those who have been a constant source of motivation for me. I deeply thank my friends, a precious presence in my daily life, and my family, for their continuous and essential support.

